Rozdział 4

Procesor

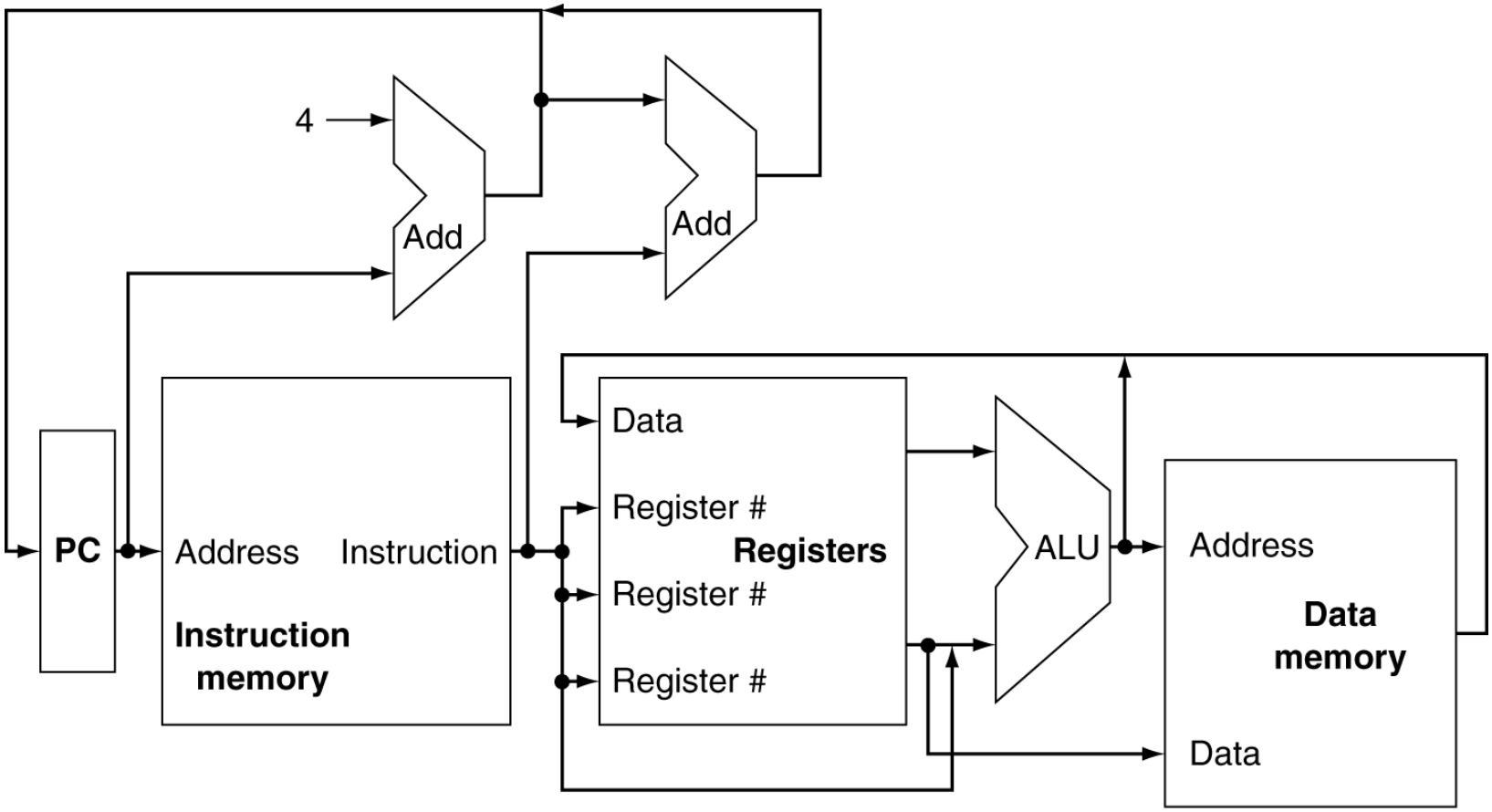
Wprowadzenie

- Czynniki wpływające na wydajność CPU
  - Liczba instrukcji
    - Określona przez ISA i zweryfikowana przez kompilator
  - CPI i czas cyklu
    - Określone przez sprzęt CPU
- Będziemy analizować dwie implementacje MIPS
  - Wersja uproszczona
  - Bardziej realistyczna wersja potokowa
- Podzbiór instrukcji ukazujący główne aspekty
  - Operujące na pamięci: `lw`, `sw` (= LDR, STR)
  - Arytmetyczne/logiczne: `add`, `sub`, `and`, `or`, `slt` (set on less than)
  - Sterujące: `beq`, `j` (jump, = B)

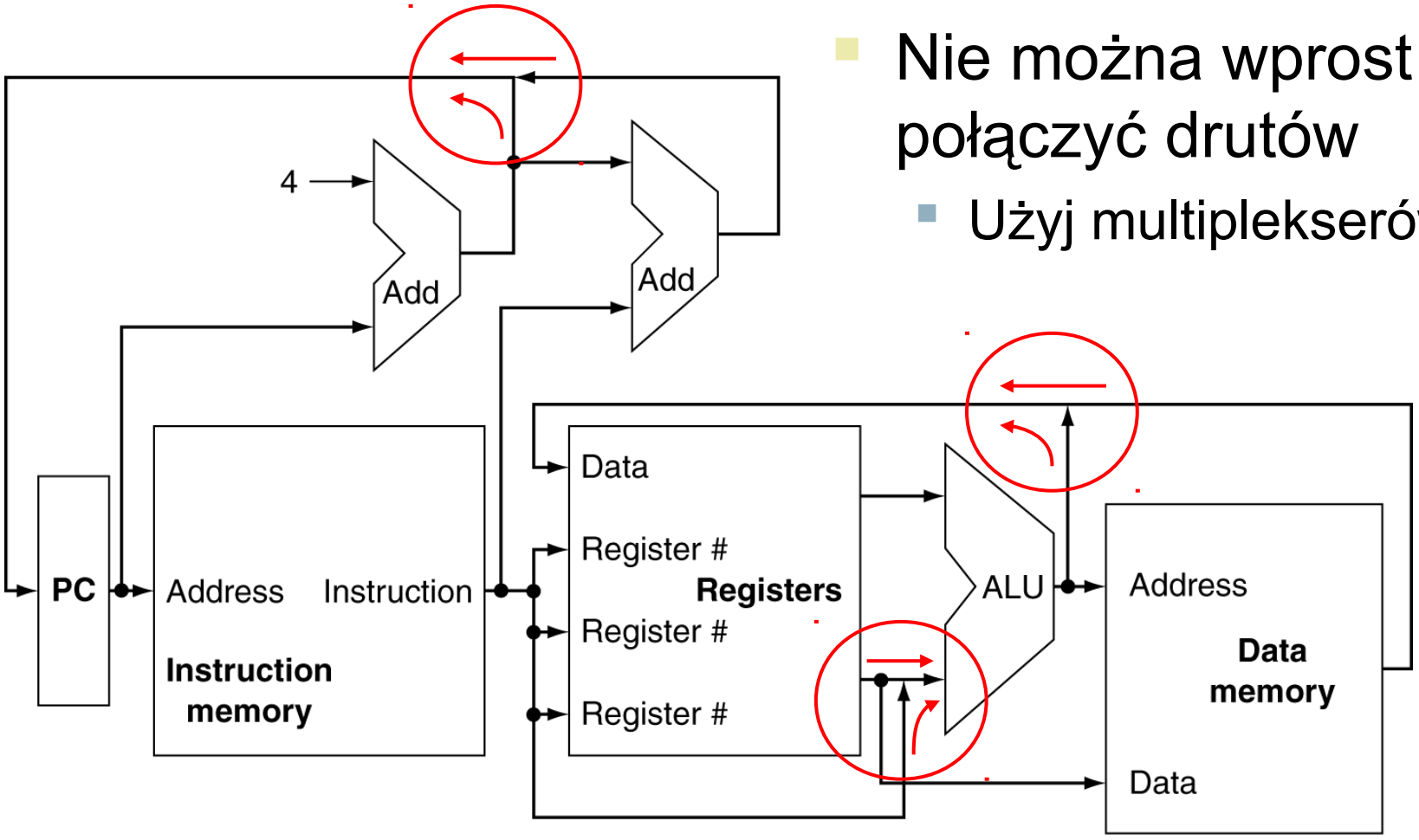
Wykonywanie instrukcji

- PC → pamięć instrukcji, załaduj (fetch) instrukcję
- Pola rejestrów → rejestry, czytaj rejestry
- Zależnie od typu instrukcji
  - Użyj ALU do obliczania
    - Wyniku operacji arytmetycznej
    - Adresu pamięci dla load/store
    - Adresu docelowego instrukcji skoku
  - Uzyskaj dostęp do pamięci danych jeśli load/store
  - PC ← adres docelowy skoku lub PC + 4

Szkic działania CPU

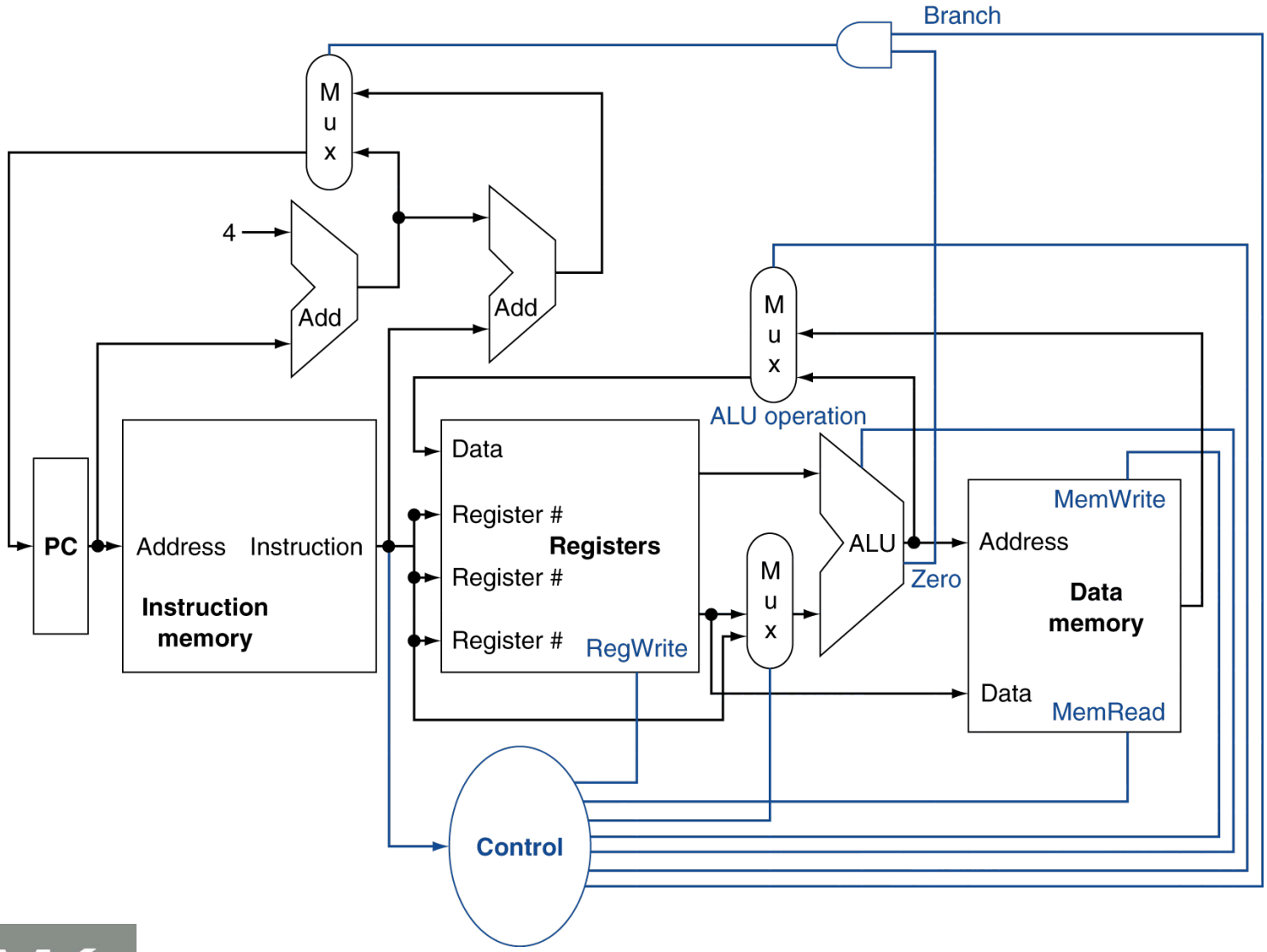


Multipleksery



- Nie można wprost połączyć drutów
  - Użyj multiplekserów

Sterowanie

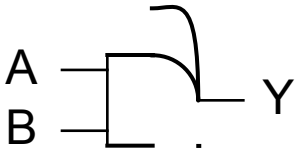

Branch
M
u
x
4
Add
Add
M
u
x
ALU operation
PC
Address
Instruction
Instruction
memory
Data
Register #
Registers
Register #
Register #
RegWrite
M
u
x
ALU
Zero
MemWrite
Address
Data
memory
Data
MemRead
Control

Projektowanie logiki

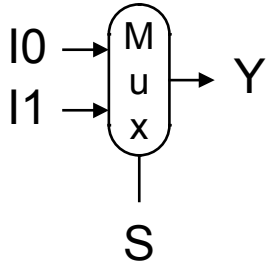
- Informacja kodowana jest binarnie
  - Niskie napięcie = 0, Wysokie napięcie = 1
  - Jeden przewód na bit
  - Dane wielobitowe kodowane na szynach wieloprzewodowych
- Element kombinacyjny
  - Działa na danych
  - Wyjście jest funkcją wejścia
- Elementy stanowe (sekwencyjne)
  - Przechowują informację

Elementy kombinacyjne

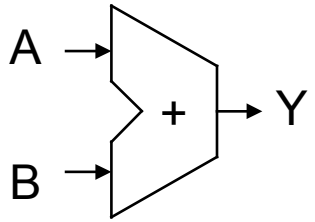
- Bramka AND
  - Y = A & B



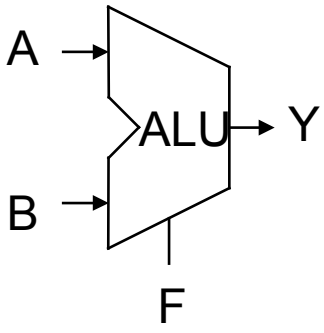
- Multiplekser
  - Y = S ? I1 : I0



- Sumator
  - Y = A + B

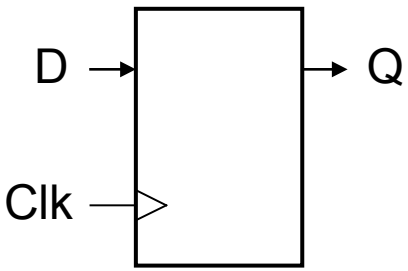


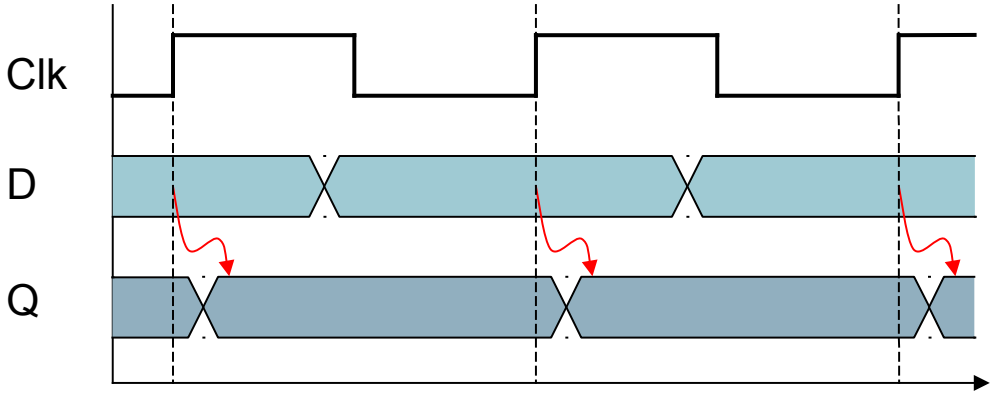
- Jednostka arytmetyczno/Logiczna
  - Y = F(A, B)



Elementy sekwencyjne

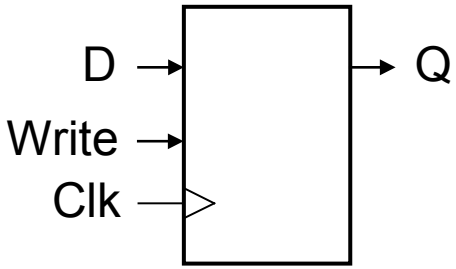
- Rejestr: zachowuje dane w układzie
  - Używa sygnału zegarowego w celu określenia momentu aktualizacji zachowanych danych
  - Wyzwalany zboczem: aktualizuj gdy Clk zmienia się z 0 na 1

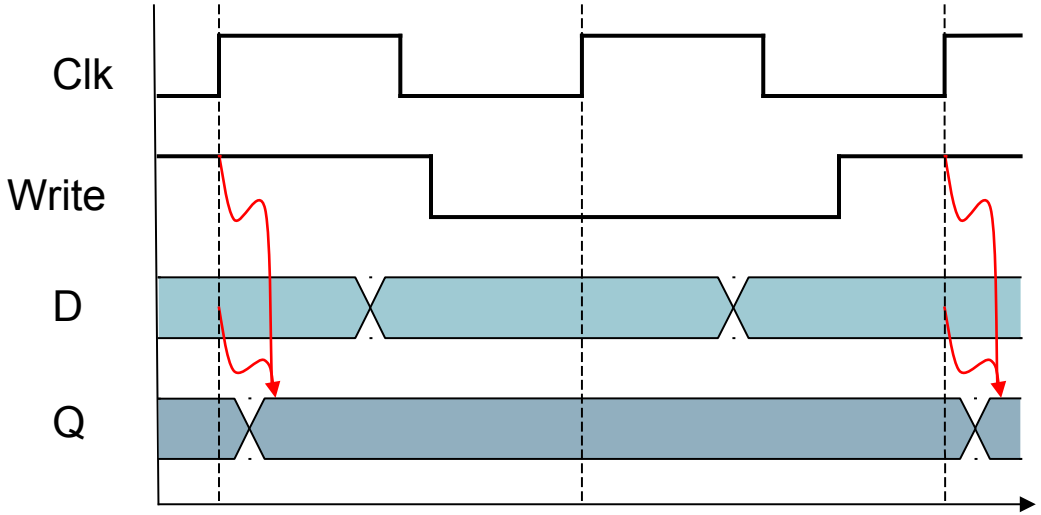




Elementy sekwencyjne

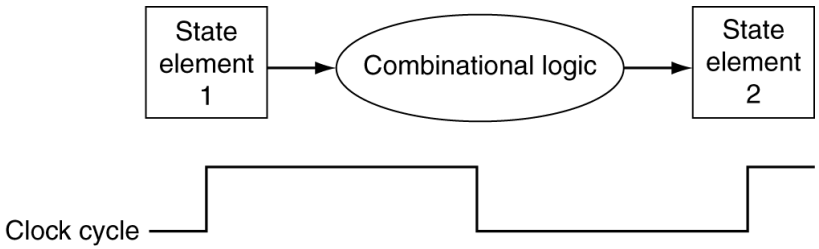
- Rejestry z kontrolą zapisu
  - Aktualizują na zboczu zegara tylko kiedy linia sterowania zapisu jest 1
  - Używane gdy zachowane dane są potrzebne później

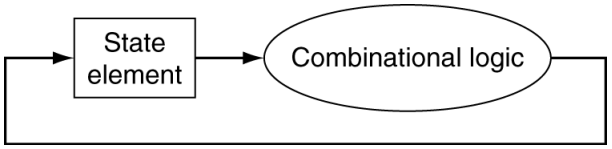




Metodologia synchronizacji

- Logika kombinacyjna przetwarza dane w trakcie cykli zegara
  - Pomiędzy zboczami zegara
  - Wejście z elementów stanu, wyjście do elementów stanu
  - Największe opóźnienie determinuje okres zegara

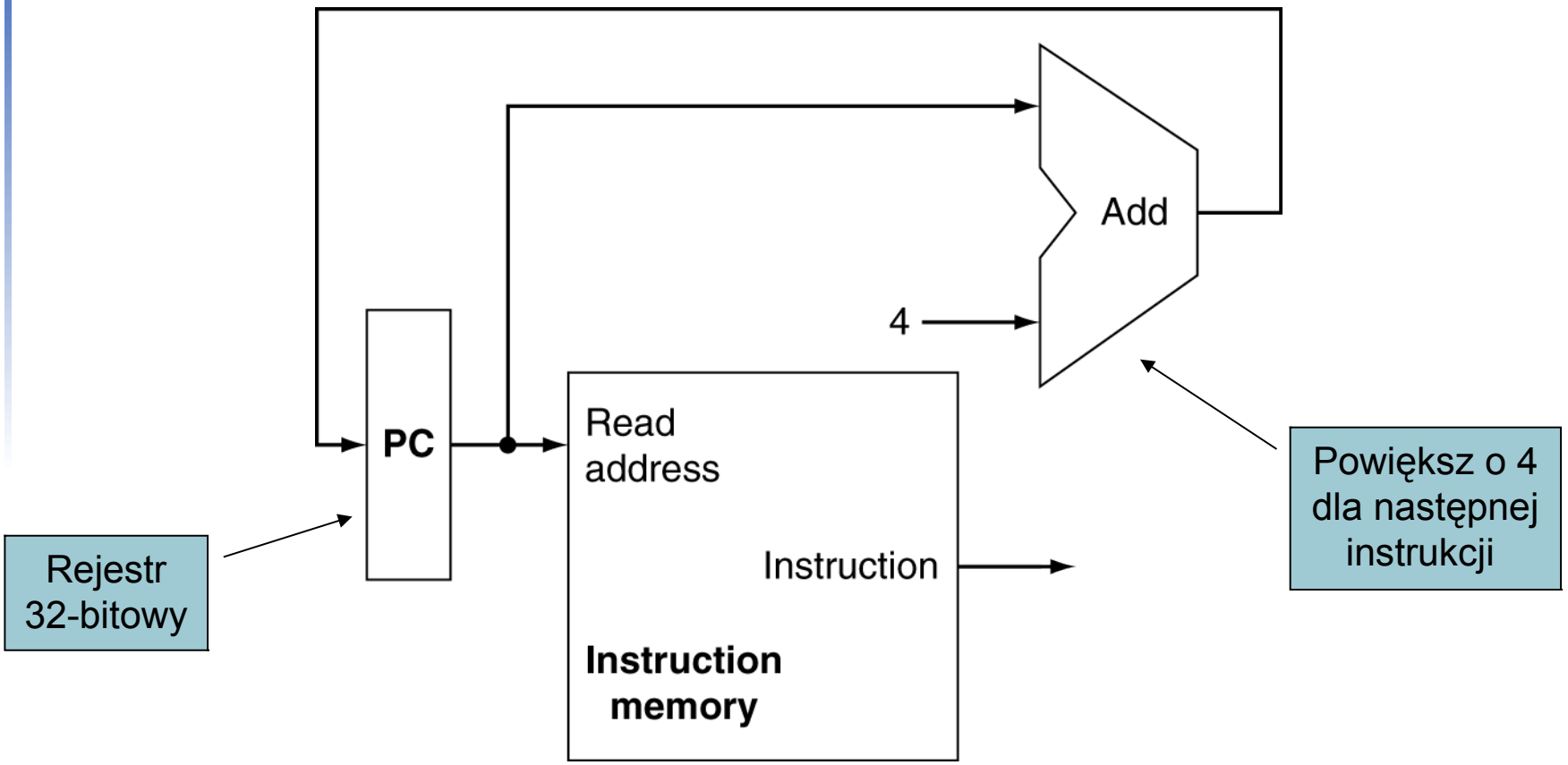




Budowanie ścieżki danych

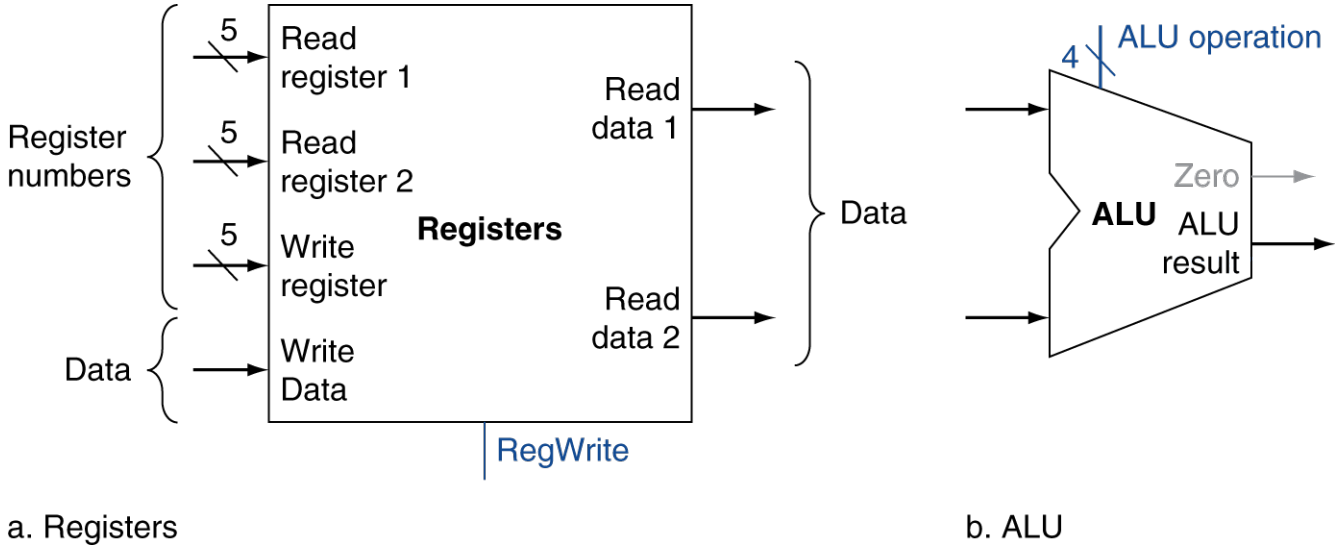
- Ścieżka danych (datapath)
  - Elementy przetwarzające dane i adresy w CPU
    - Rejestry, ALU, mux'y, pamięci, …
- Budujemy ścieżkę danych MIPS'a stopniowo
  - Przez uściślanie projektu szkicowego

Ładowanie instrukcji



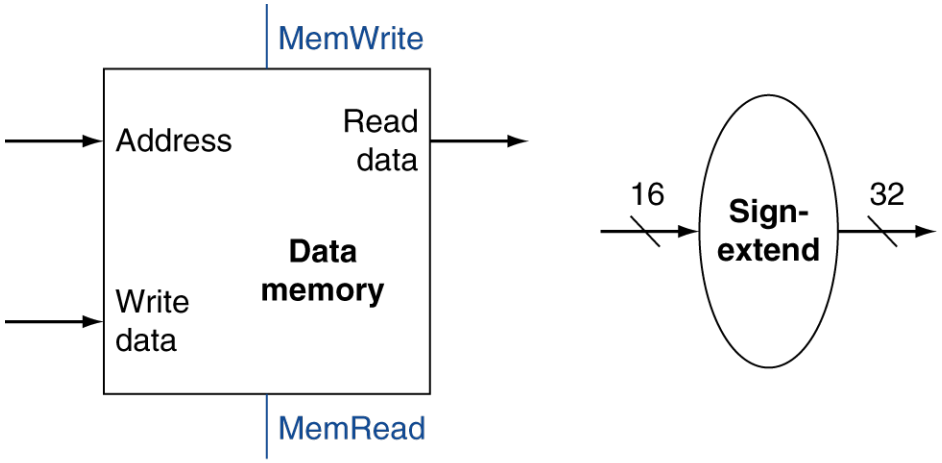
Instrukcje R (R-Format)

- Odczytej dwa operandy rejestrowe
- Wykonaj operację arytmetyczną/logiczną
- Zapisz do rejestru wynikowego



Instrukcje Load/Store

- Odczytaj operandy rejestrowe
- Oblicz adres używając 16-bitowego przesunięcia
  - Użyj ALU, przesunięcie (offset) rozszerz znakowo
- Load: Czytaj pamięć i zaktualizuj rejestr
- Store: Zapisz wartość z rejestru do pamięci



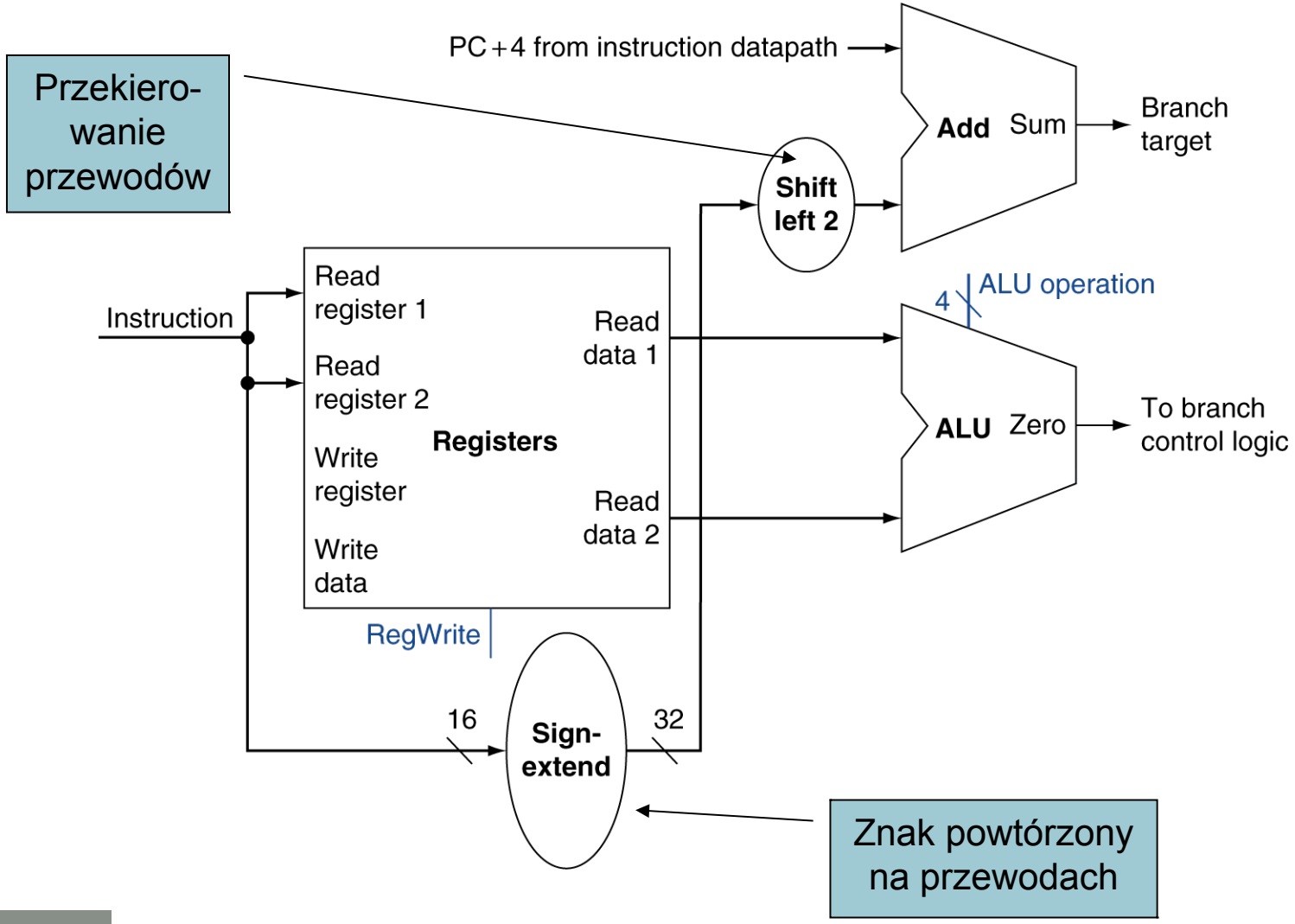
a. Data memory unit

b. Sign extension unit

Instrukcje skoku (branch)

- Czytaj operandy rejestrowe
- Porównaj operandy
  - Użyj ALU, odejmij i sprawdź czy wynik = 0
- Oblicz adres docelowy (target) skoku
  - Rozszerz znakowo przesunięcie (displacement)
  - Przesuń w lewo o 2 miejsca (word displacement)
  - Dodaj do PC: PC + 4
    - Już wykonane przy ładowaniu instrukcji

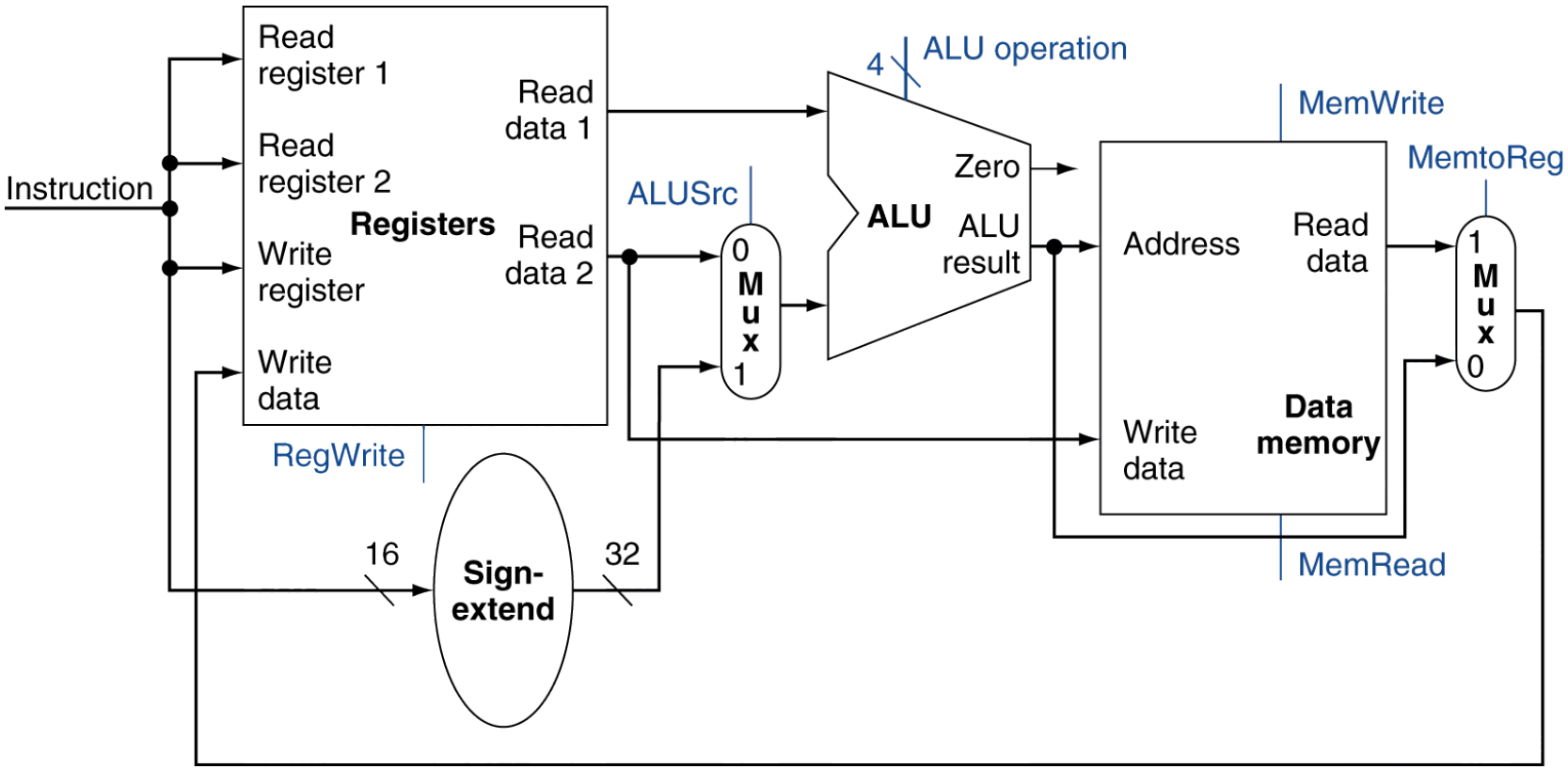
Instrukcje skoku



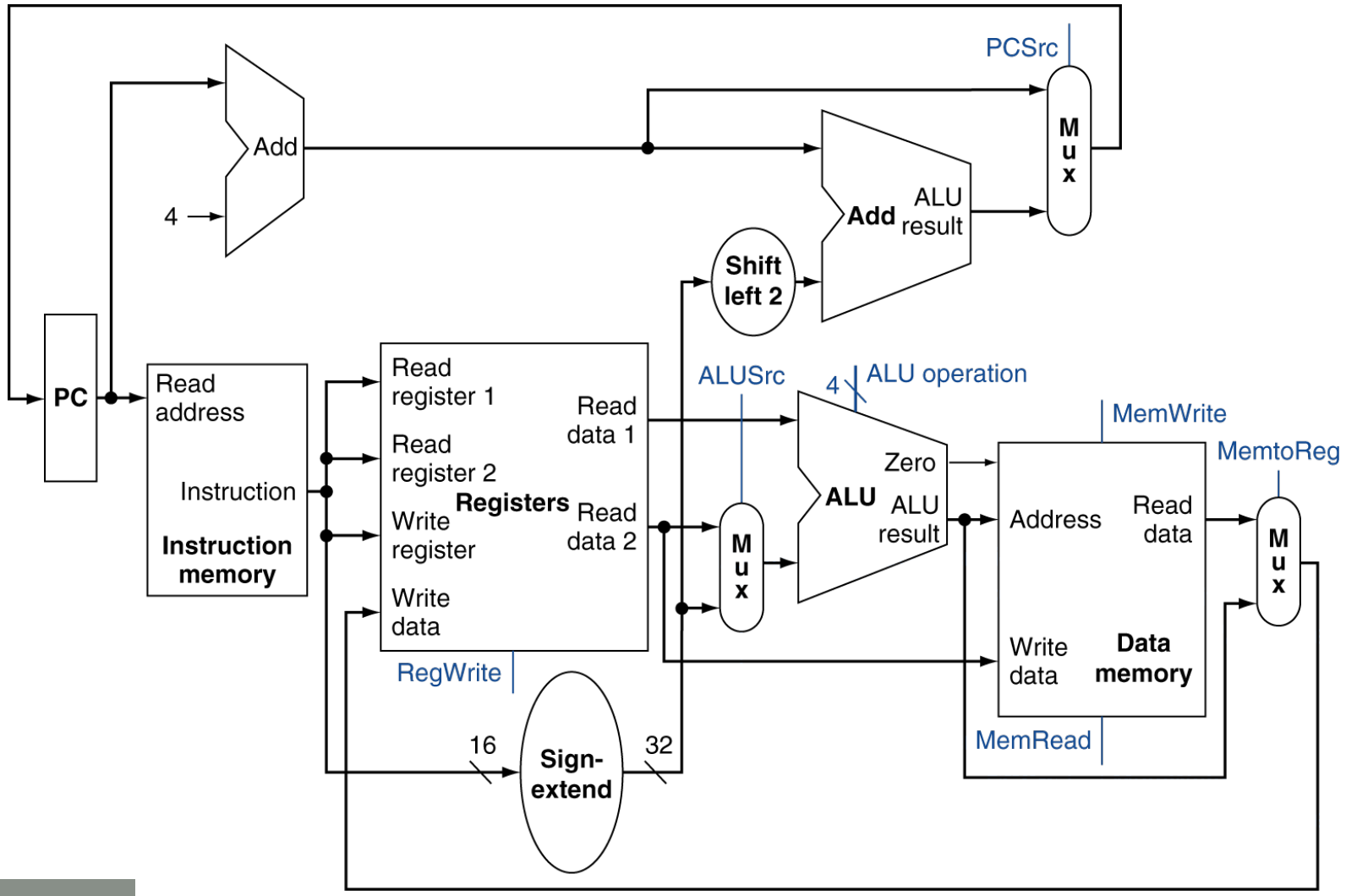
Łączenie elementów

- Prototypowa ścieżka danych wykonuje instrukcję w jednym cyklu zegara
  - Każdy element ścieżki danych może w danej chwili wykonywać tylko jedną funkcję
  - Tak więc potrzebujemy osobnych pamięci na instrukcje i dane
- Użyj multiplekserów tam, gdzie dla różnych instrukcji są używane różne źródła danych

Scieżka danych R/Load/Store



Pełna ścieżka danych



Sterowanie ALU

- ALU jest używane do
  - Load/Store: F = dodaj
  - Branch: F = odejmij
  - R-type: F zależy od pola funct

Sterowanie ALU	Funkcja
0000	AND
0001	OR
0010	Dodaj
0110	Odejmij
0111	Ustaw jeśli mniej niż (set-on-less-than)
1100	NOR

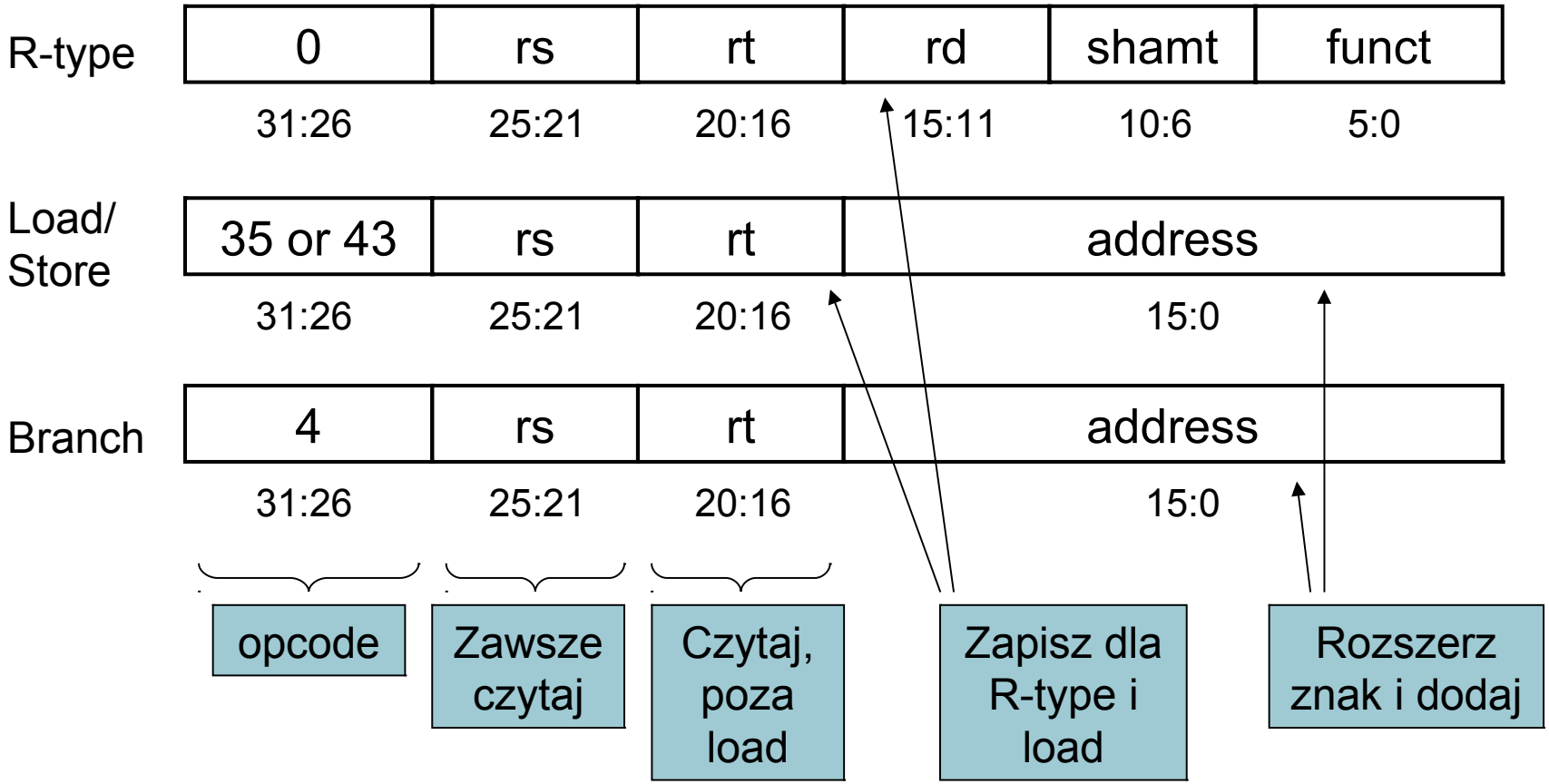
Sterowanie ALU

- Przyjmijmy 2-bitowe ALUOp otrzymane z pola opcode
  - Logika kombinacyjna generuje sterowanie ALU

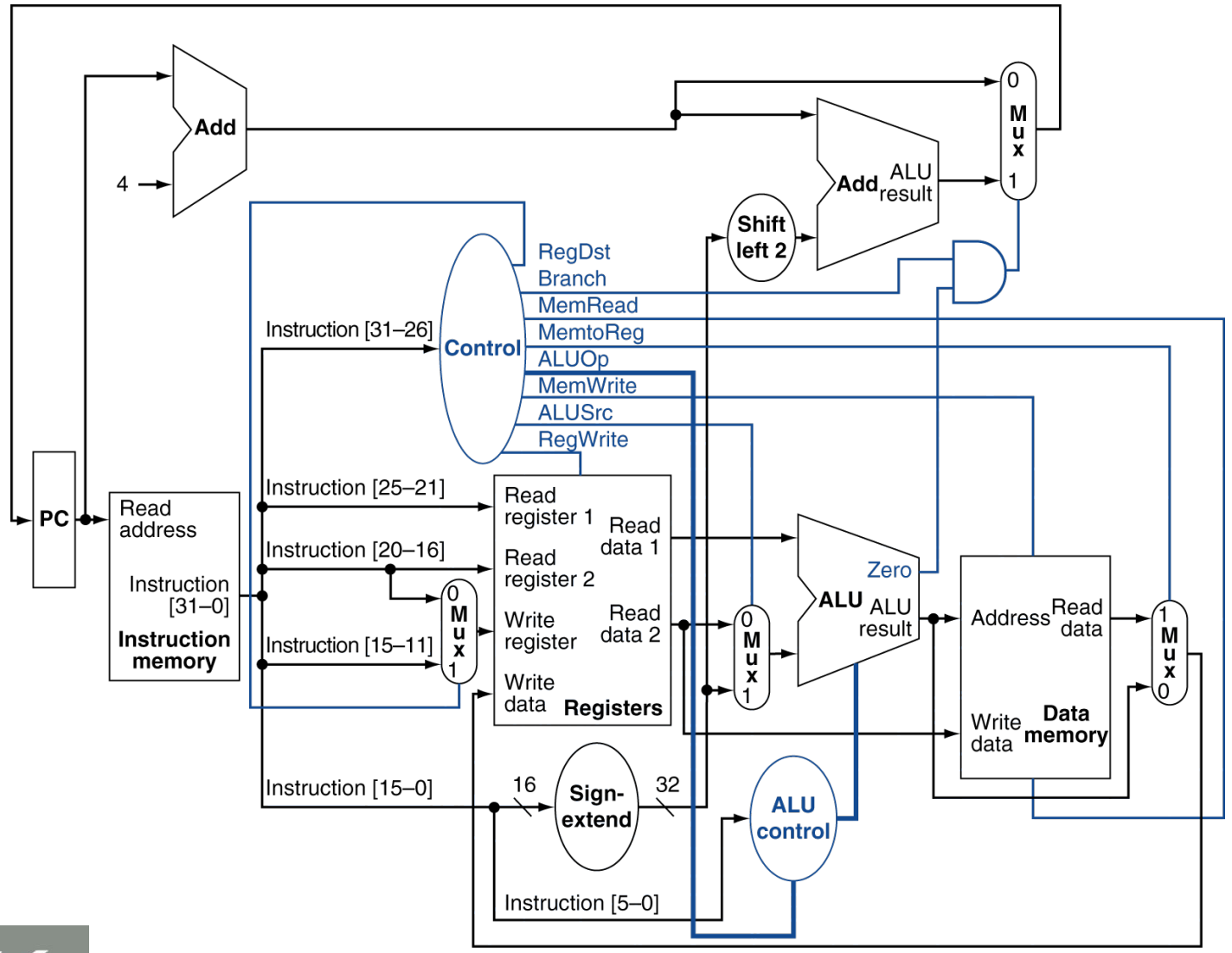
opcode	ALUOp	Operacja	funct	Funkcja ALU	Sterowanie ALU
lw	00	Ładuj słowo	XXXXXX	dodaj	0010
sw	00	Chowaj słowo	XXXXXX	dodaj	0010
beq	01	Skok jeśli równe	XXXXXX	odejmij	0110
R-type	10	dodaj	100000	dodaj	0010
		odejmij	100010	odejmij	0110
		AND	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	set-on-less-than	0111

Główna jednostka sterująca

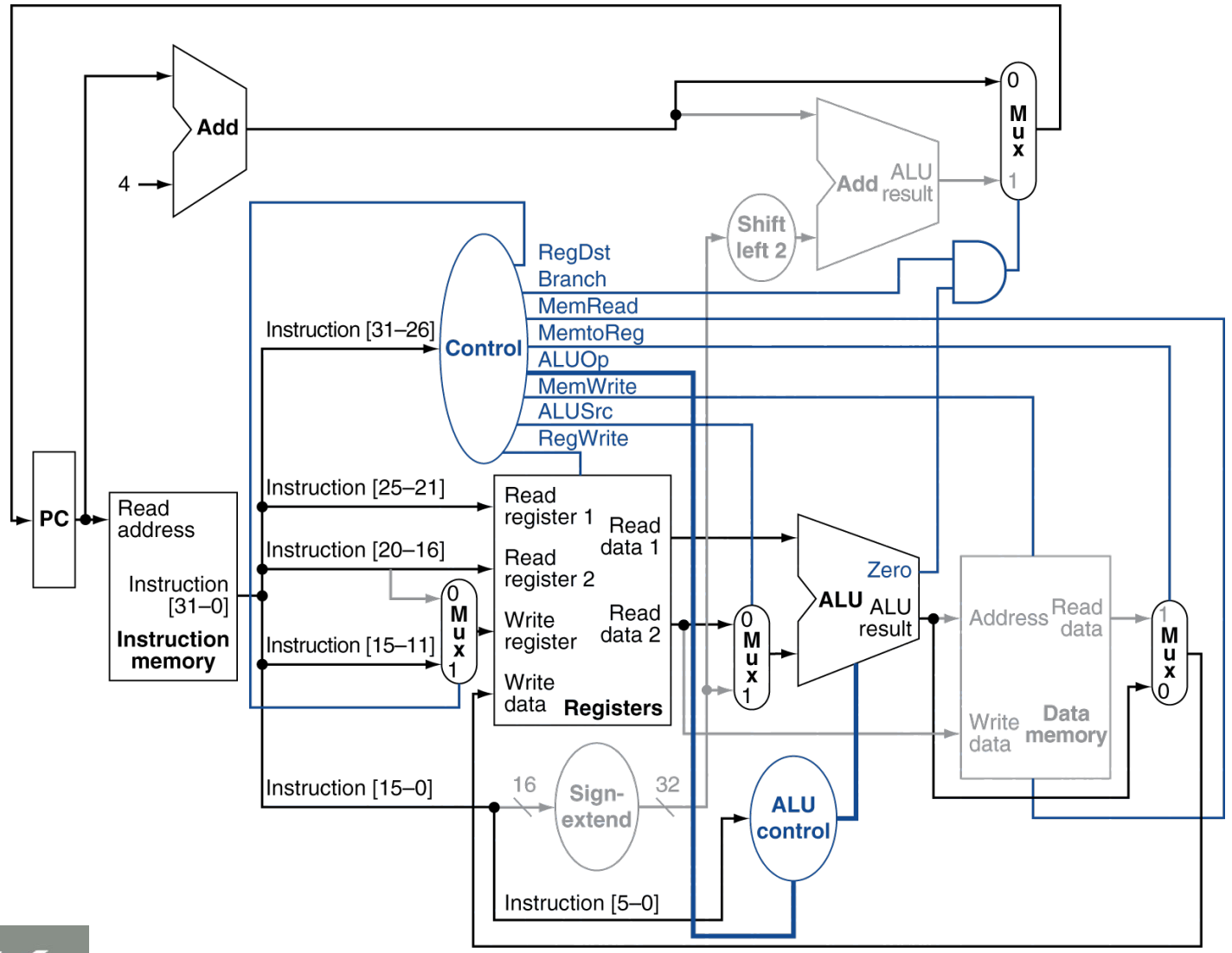
- Sygnały sterowania otrzymane z instrukcji



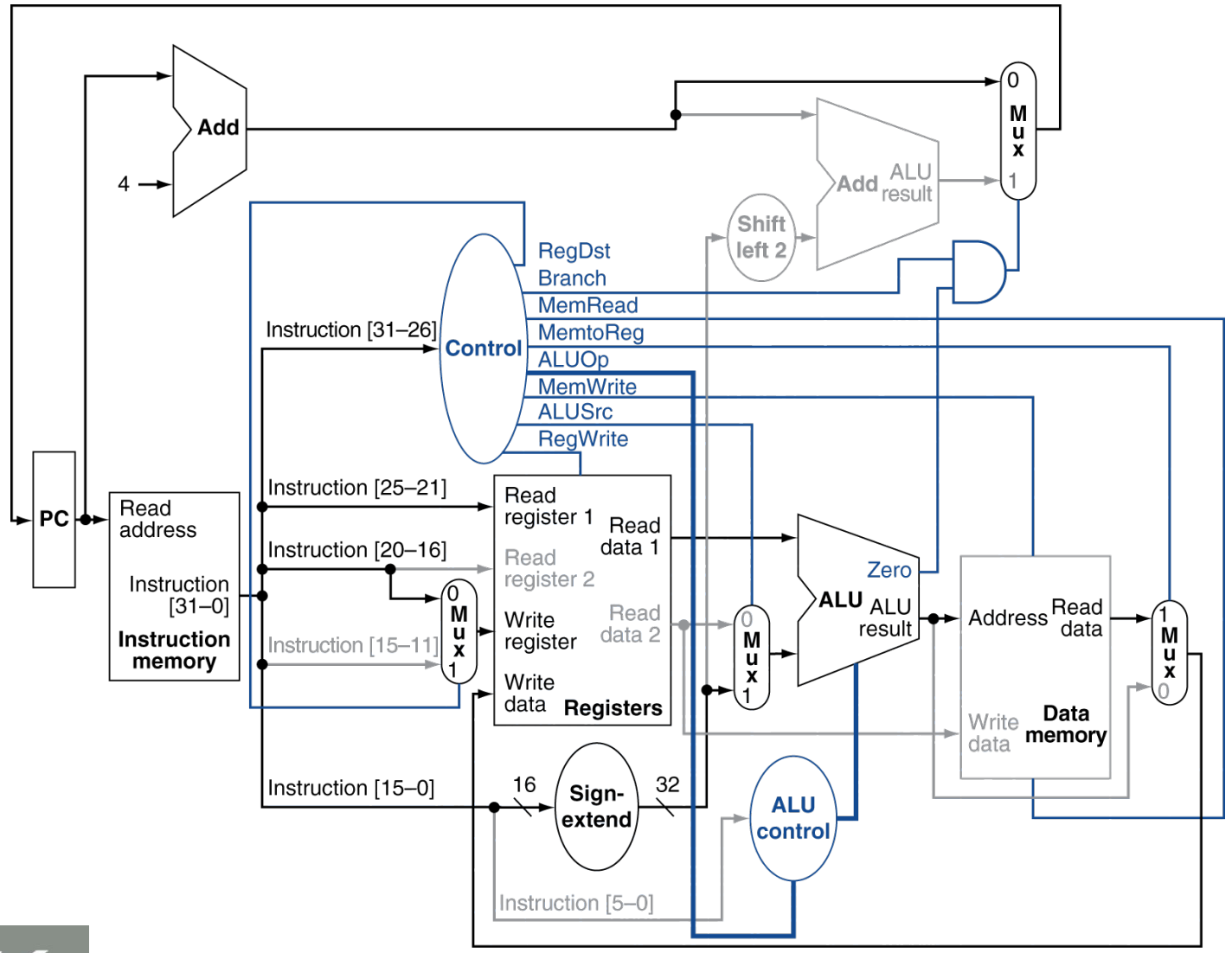
Scieżka danych + sterowanie



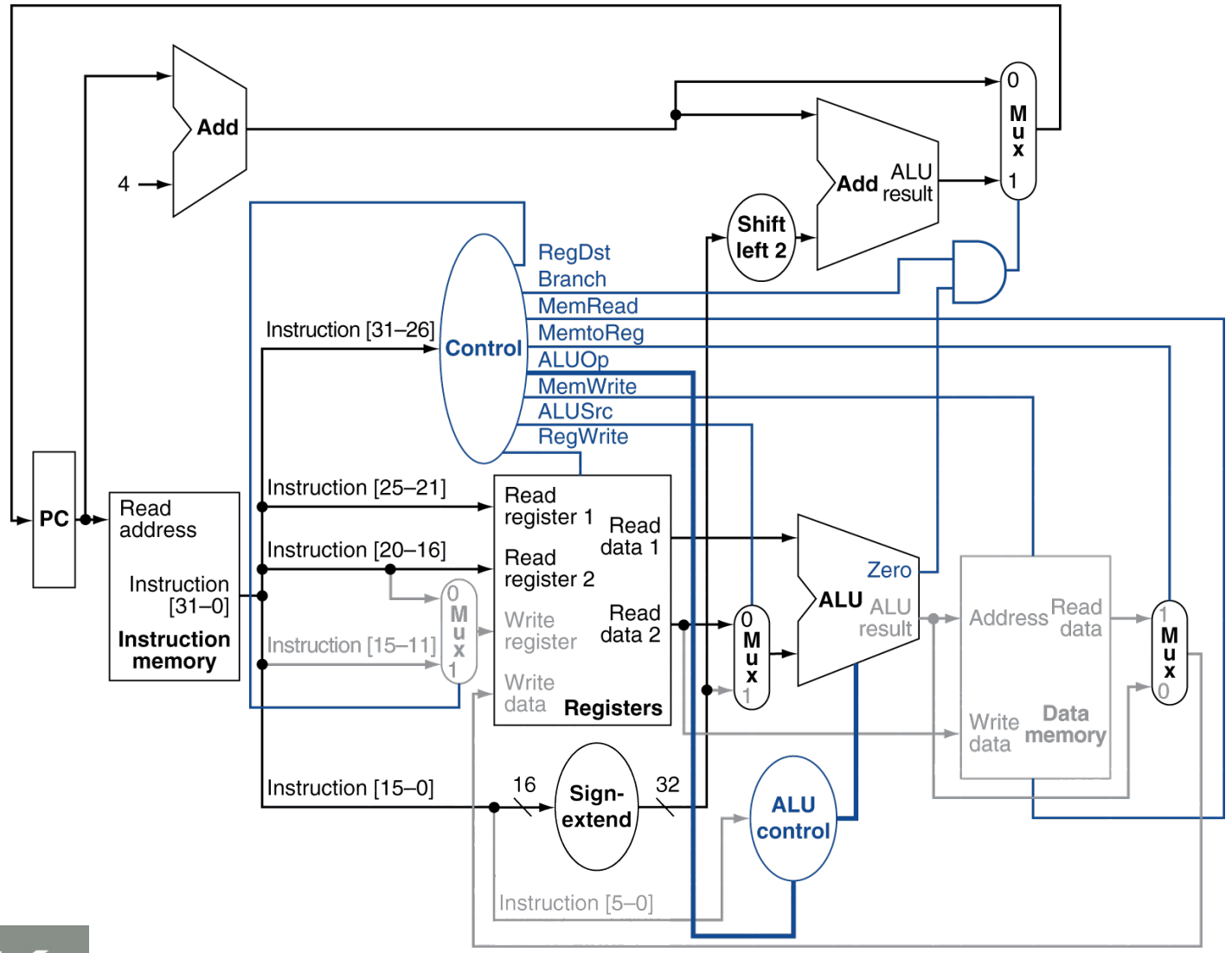
Instrukcja R-Type



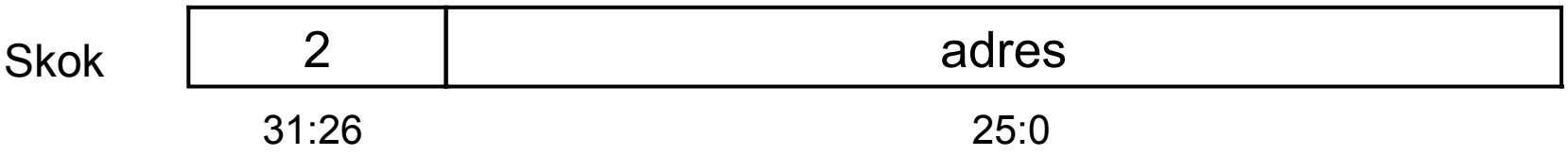
Instrukcja Load



Instrukcja skocz gdy równe

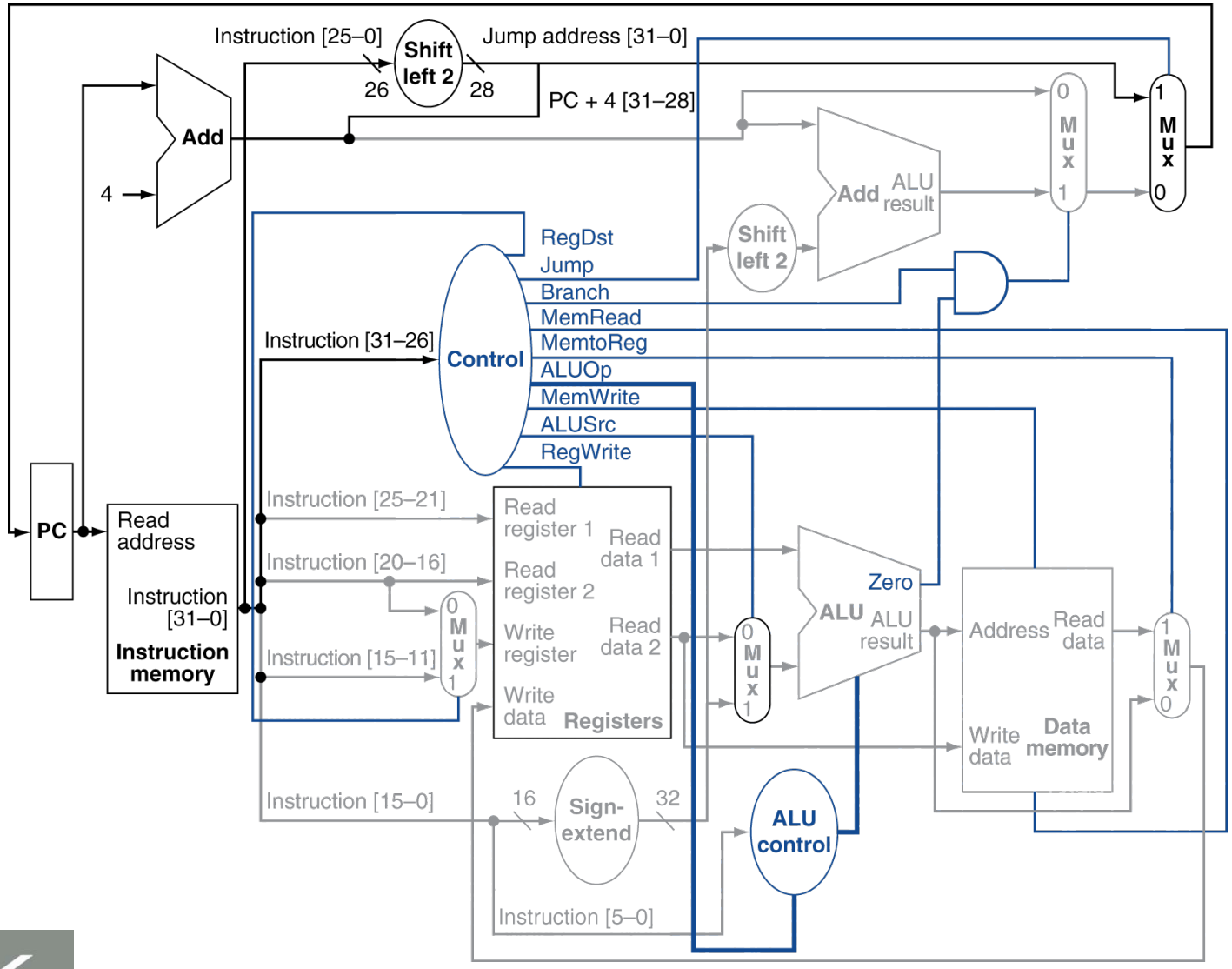


Skok bezwarunkowy



- Skok operuje na adresie słowa
- Uaktualnij PC konkatenacją
  - 4 górnych bitów poprzedniego PC
  - 26 bitami adresu skoku
  - 00
- Konieczny dodatkowy sygnał sterujący dekodowany z opcode

Scieżka danych ze skokami

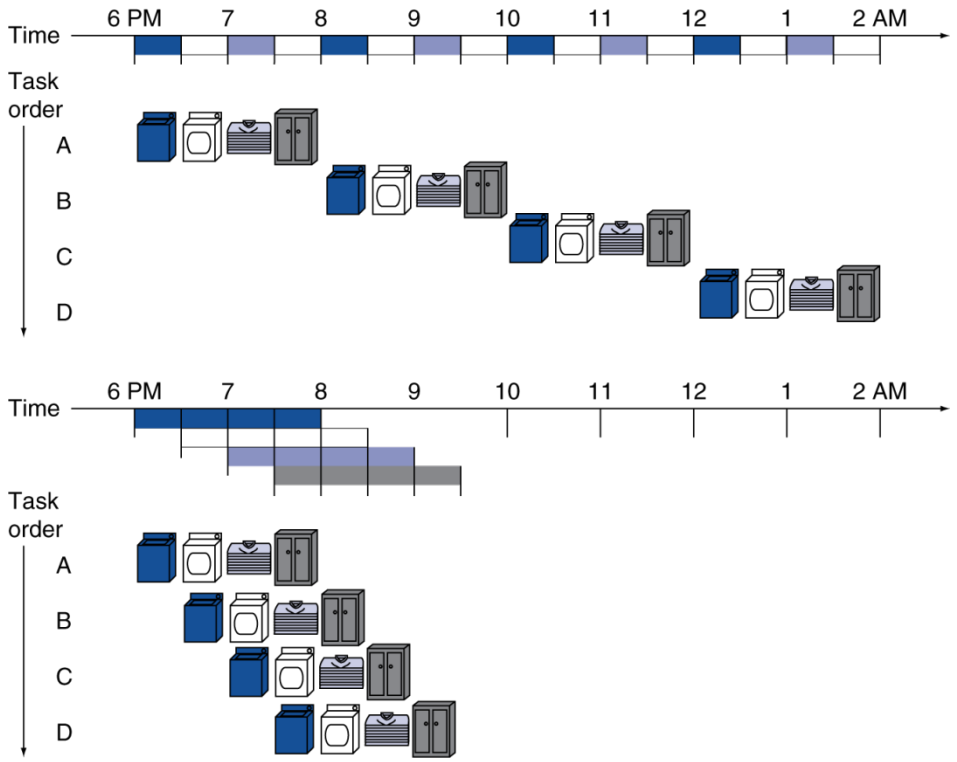


Co z wydajnością

- Największe opóźnienie określa okres zegara
  - Ścieżka krytyczna: instrukcja load
  - Pamięć instrukcji → rejestry → ALU → pamięć danych → rejestry
- Dopasowywanie okresu zegara do bieżącej instrukcji niewykonalne
- Podważa zasadę projektowania
  - Czyń typowy przypadek szybkim
- Poprawimy wydajność przez potokowość

Potokowość - analogia

- Potok w pralni: wykonanie „na zakładkę”
  - Równoległość poprawia wydajność



- Cztery ładunki:
  - Przyspieszenie = 8/3.5 = 2.3
- Non-stop:
  - Przyspieszenie (speedup) = 2n/0.5n + 1.5 ≈ 4 = liczba etapów

Potok w MIPS'ie

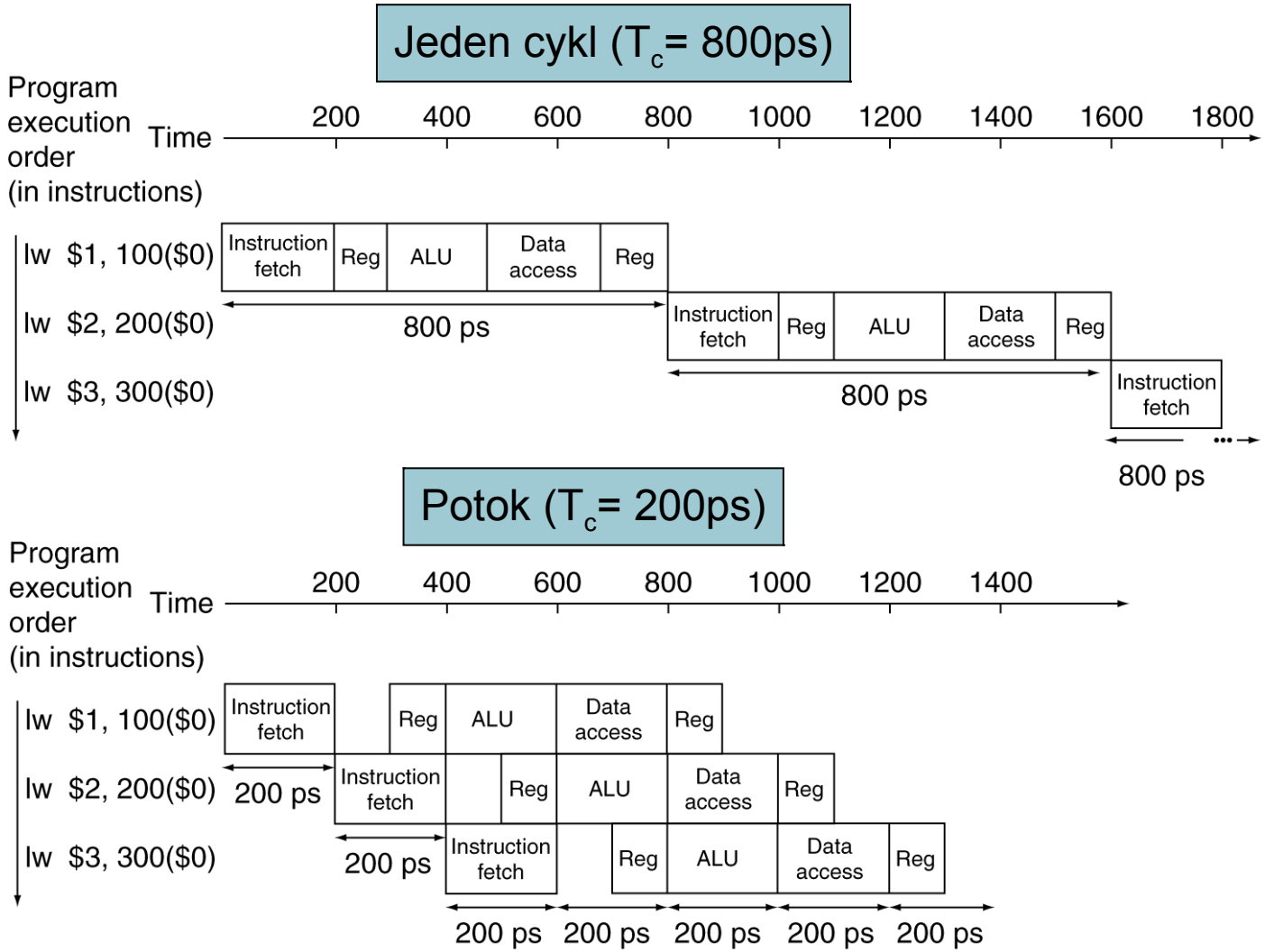
- Pięć etapów, krok na etap
  1. IF: Ładowanie (fetch) instrukcji z pamięci
  2. ID: Dekodowanie instrukcji & odczyt rejestrów
  3. EX: Wykonanie operacji lub obliczanie adresu (execute)
  4. MEM: Transfer operandu do/z pamięci
  5. WB: Zapis wyniku w rejestrze (write back)

Wydajność potoku

- Przyjmijmy czasy etapów
  - 100ps na zapis lub odczyt rejestru
  - 200ps na pozostałe etapy
- Porównaj potokową ścieżkę danych ze ścieżką w jednym cyklu zegara

Instr	Instr fetch	Odczyt rejestru	ALU op	Dostęp pamięci	Zapis rejestru	Czas całkowity
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Wydajność potoku



Przyspieszenie przez potok

- Jeśli wszystki etapy są zbalansowane
  - tj. wszystkie trwają tak samo długo
  - Czas pomiędzy instrukcjami$_{\text{potokowy}}$
    = $\frac{\text{Czas pomiędzy instrukcjami}_{\text{bez potoku}}}{\text{Liczba etapów}}$
- Jeśli nie zbalansowane, to przyspieszenie jest mniejsze
- Przyspieszenie bo większa przepustowość
  - Czas oczekiwania (latency, czas pojedynczej instrukcji) nie zmniejsza się

Potok i projektowanie ISA

- MIPS ISA zaprojektowano dla potokowości
  - Wszystkie instrukcje są 32 bitowe
    - Łatwiej ładować i dekodować w jednym cyklu
    - porównaj x86: 1- do 17-bajtowe instrukcje
  - Mało i regularne formaty instrukcji
    - Dekodowanie i odczyt rejestrów w jednym kroku
  - Adresowanie load/store
    - Obliczanie adresu w 3-cim etapie, dostęp do pamięci w etapie 4-tym
  - Wyrównanie operandów w pamięci
    - Dostęp do pamięci zajmuje tylko jeden cykl

Zagrożenia (Hazards)

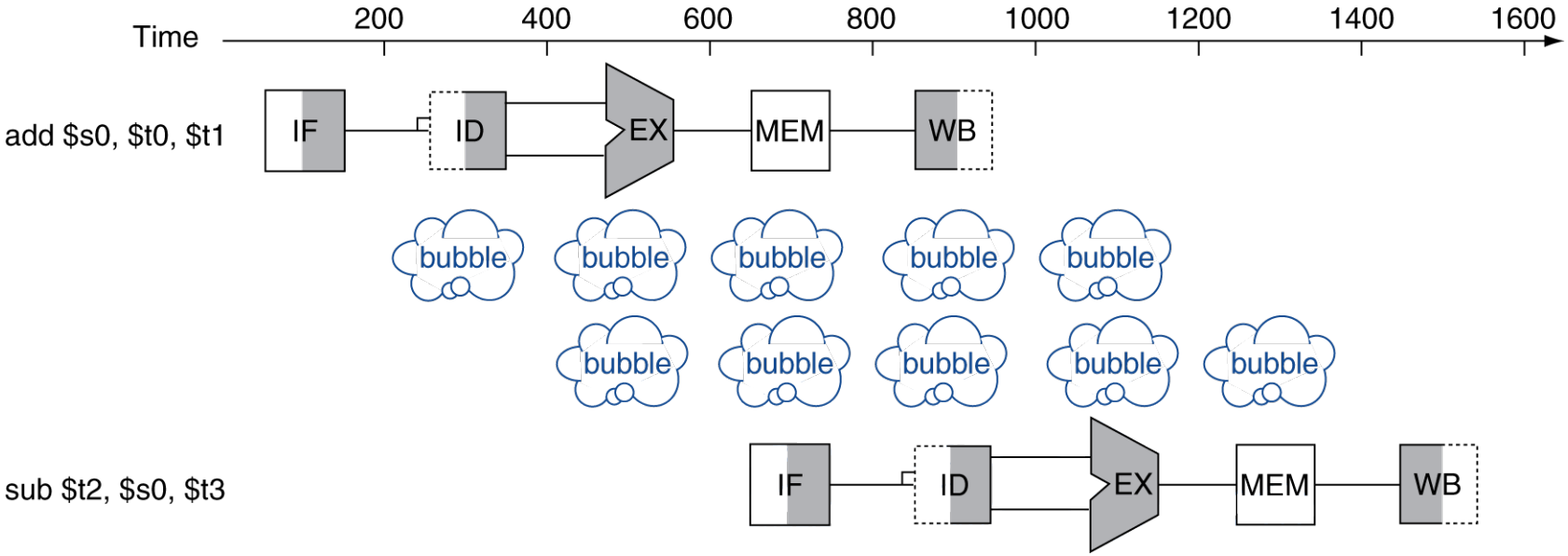
- Sytuacje w których w następnym cyklu nie może zostać wykonana instrukcja
- Zagrożenia strukturalne
  - Potrzebne zasoby są zajęte
- Zagrożenia danych
  - Konieczność oczekiwania na zapis lub odczyt danych przez poprzednią instrukcję
- Zagrożenia sterowania
  - Decyzja odnośnie podejmowanej akcji zależy od poprzedzającej instrukcji

Zagrożenia strukturalne

- Konflikt użycia zasobów
- W potoku MIPS z jedną pamięcią (dotyczy pierwszych ARM'ów)
  - Load/store wymaga dostępu do danych
  - Ładowanie instrukcji musiałoby się zatrzymać (*stall)* w tym cyklu
    - Wywołałoby to potokową „bańkę" ("bubble")
- Też potokowa ścieżka danych wymaga oddzielnych pamięci instrukcji/danych
  - Lub osobnych pamięci podręcznych (caches)

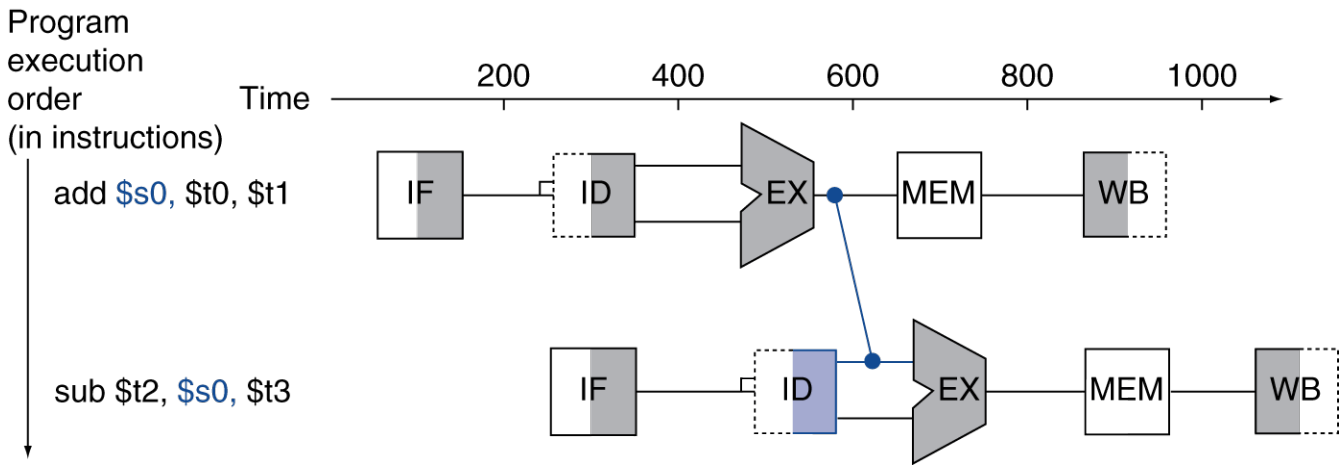
Zagrożenia danych

- Instrukcja zależy od uzyskania dostępu do danych w poprzedniej instrukcji
  - `add  $s0, $t0, $t1`
    `sub  $t2, $s0, $t3`



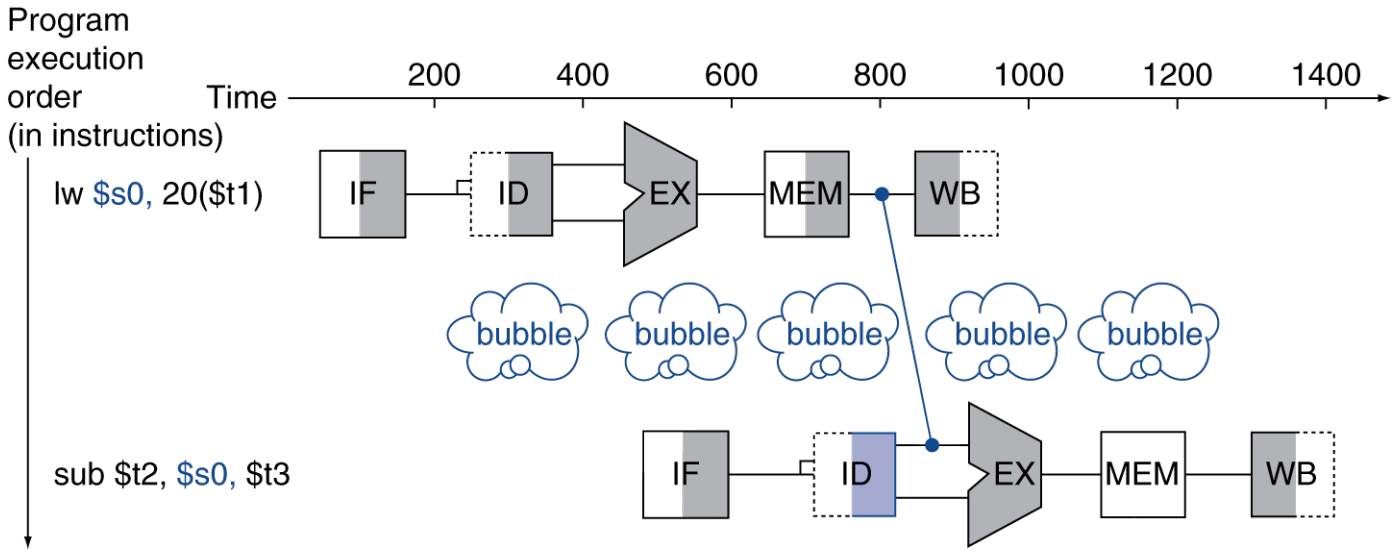
Przekierowanie (omijanie)

- Użyj wyniku gdy zostanie obliczony
  - Nie czekaj aż zostanie zapisany do rejestru
  - Wymaga dodatkowych połączeń w ścieżce danych (przekierowanie – forwarding)



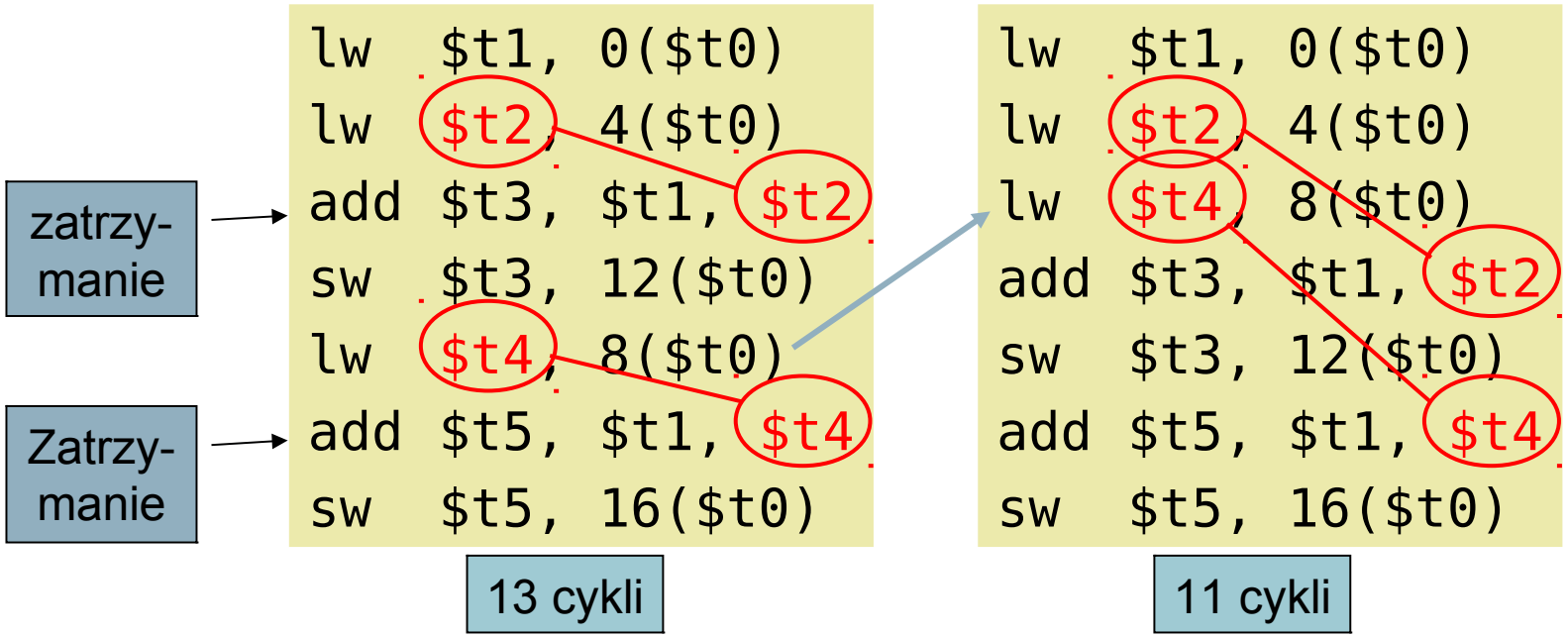
Zagrożenia danych – Load

- Nie zawsze da się uniknąć zatrzymania przez przekierowanie
  - Jeśli wartości nie obliczono na czas
  - Nie można przekierowywać wstecz w czasie!



Programowanie a zatrzymania

- Przeorganizuj kod w celu uniknięcia użycia wyniku load w następnej instrukcji
- Kod C: `A = B + E; C = B + F;`

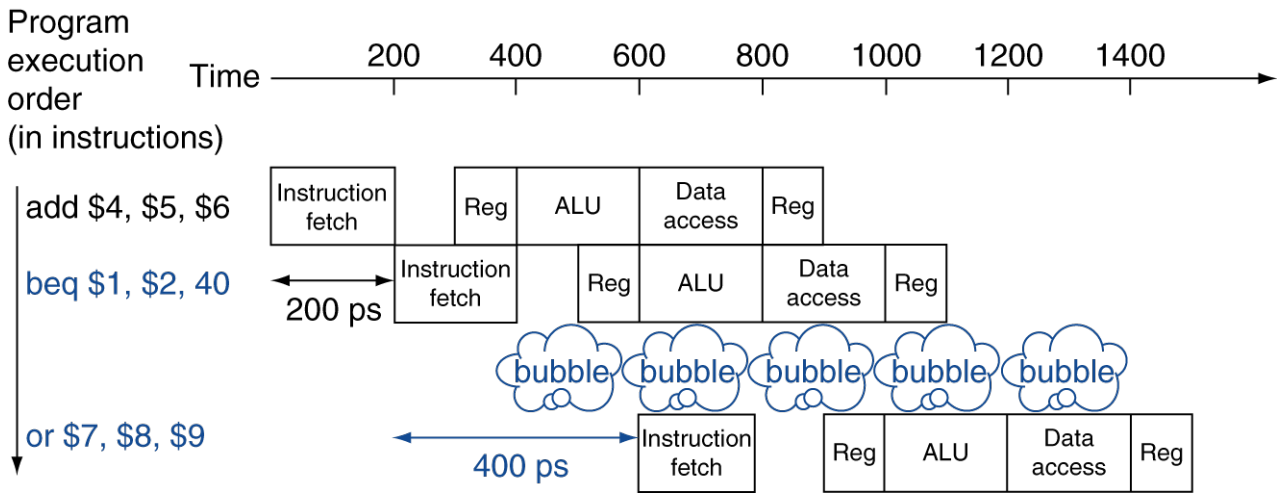


Zagrożenia sterowania

- Skoki realizują przebieg sterowania
  - Ładowanie następnej instrukcji zależy od wyliczenia adresu docelowego skoku
  - Potok nie zawsze może załadować poprawną instrukcję
    - Bo ciągle pracuje nad etapem ID skoku
- W potoku MIPS
  - Konieczność wczesnego porównywania rejestrów i obliczania adresu docelowego skoku
  - Dodaj sprzęt do wykonania tego w etapie ID

Zatrzymanie przy skoku

- Czekaj z ładowaniem kolejnej instrukcji aż pojawi się adres docelowy skoku

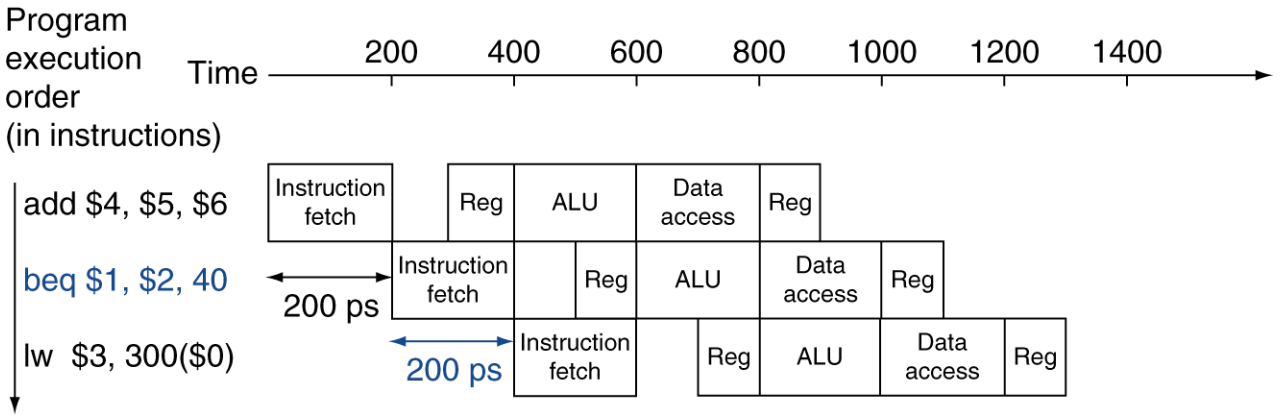


Predykcja skoku

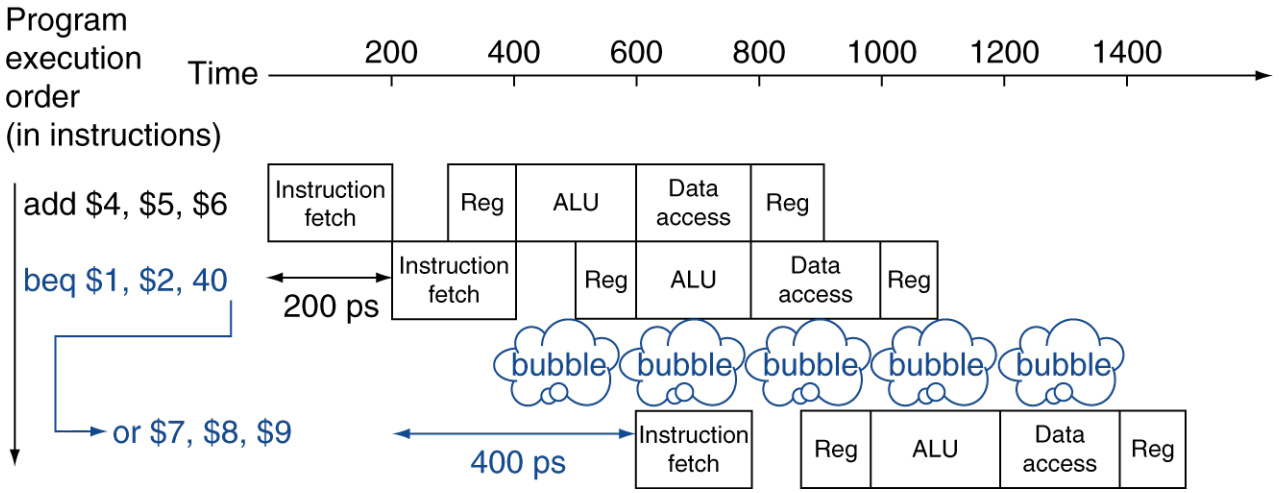
- Dłuższy potok nie jest w stanie określić skoku dostatecznie wcześnie
  - Koszt zatrzymań staje się nieakceptowalny
- Zgadywanie wyniku instrukcji skoku
  - Zatrzymanie tylko gdy predykcja jest błędna
- W potoku MIPS
  - Można przewidywać, że skoku nie będzie
  - Ładuj bez opóźnienia instrukcję następującą po skoku

MIPS: predykcja skoku nie ma

Predykcja poprawna



Predykcja niepoprawna



Bardziej realistyczna predykcja

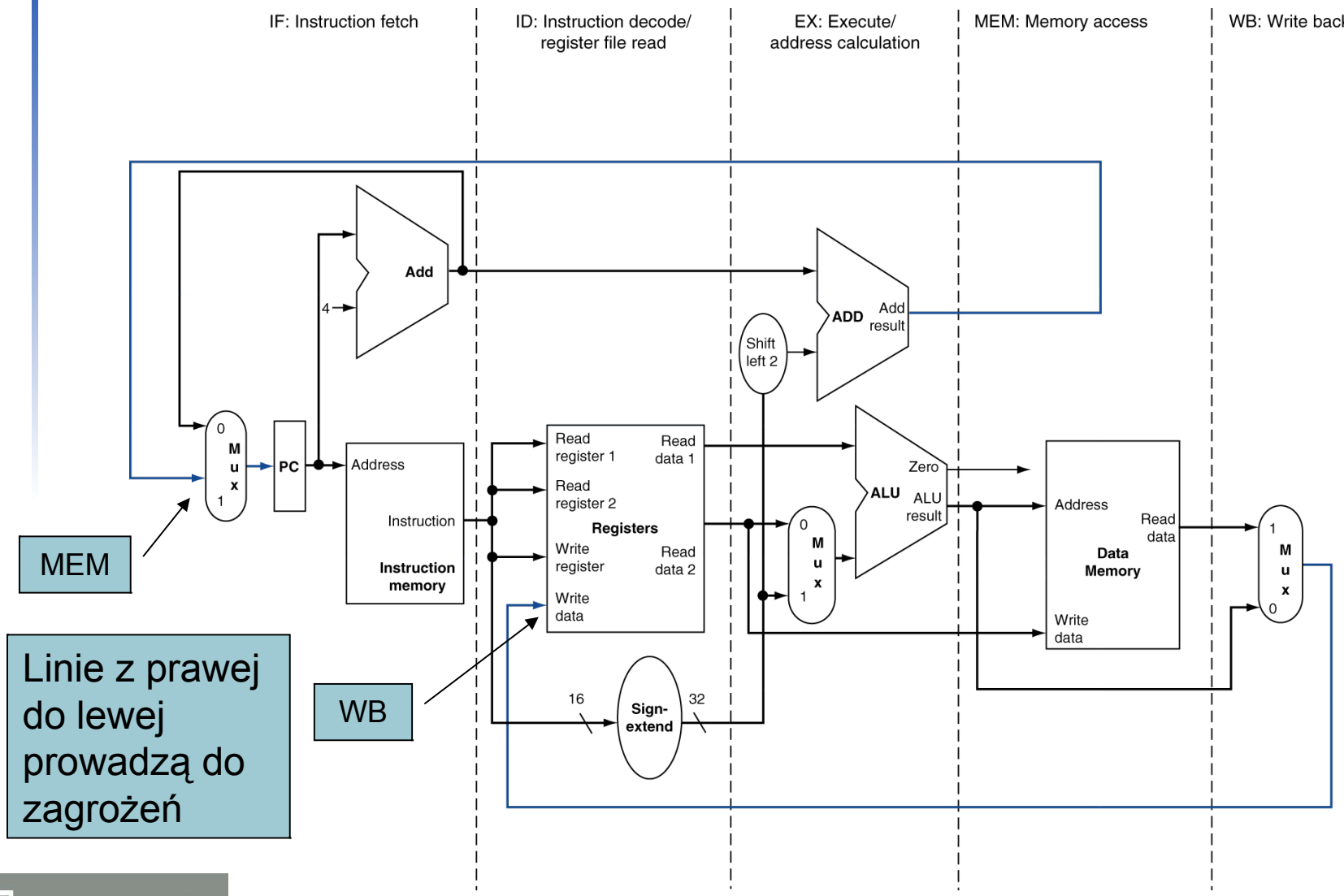
- Statyczna predykcja skoku
  - Oparta na typowym wyniku skoku
  - Przykład: skoki w pętlach i instrukcjach if
    - Oczekuj skoków wstecz
    - Przewiduj: skoki w przód nie wykonują się
- Dynamiczna predykcja skoków
  - Sprzęt bada faktyczne wykonania skoków
    - np. śledzi historię ostatnich skoków
  - Zakłada, że obecny trend utrzyma się
    - Jeśli predykcja błędna, zatrzymanie przy przeładowaniu instrukcji i uaktualnienie historii

Potok – podsumowanie

The BIG Picture

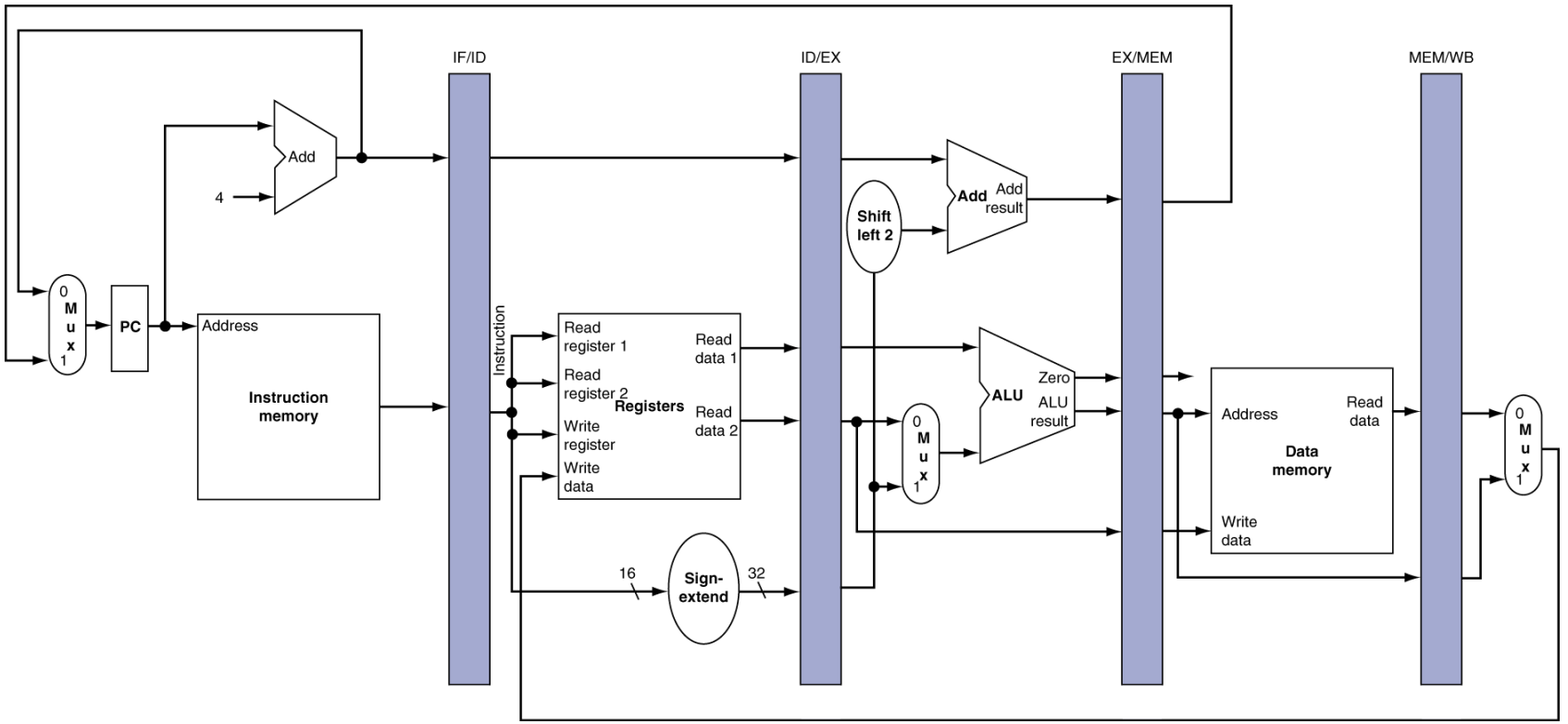
- Potok poprawia wydajność przez zwiększenie przepustowości instrukcji
  - Wykonuje wiele instrukcji równolegle
  - Każda instrukcja ma ten sam czas wykonania
- Podlega zagrożeniom
  - Strukturalnym, danych, sterowania
- Projektowanie listy instrukcji wpływa na złożoność implementacji potoku

Potok a ścieżka danych MIPS



Rejestry potoku

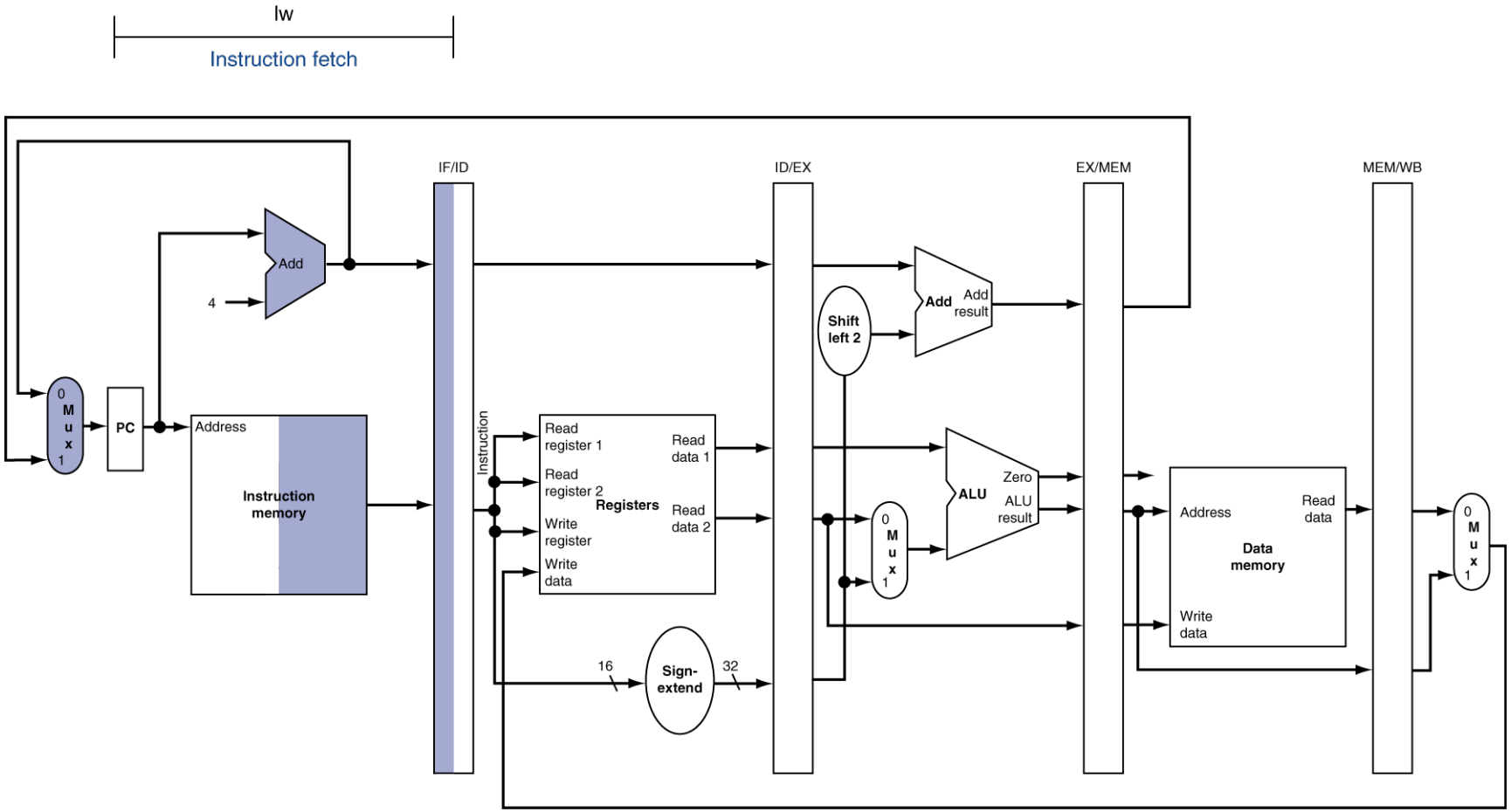
- Konieczne są rejestry pomiędzy etapami
  - Do pamiętania informacji z poprzedniego cyklu



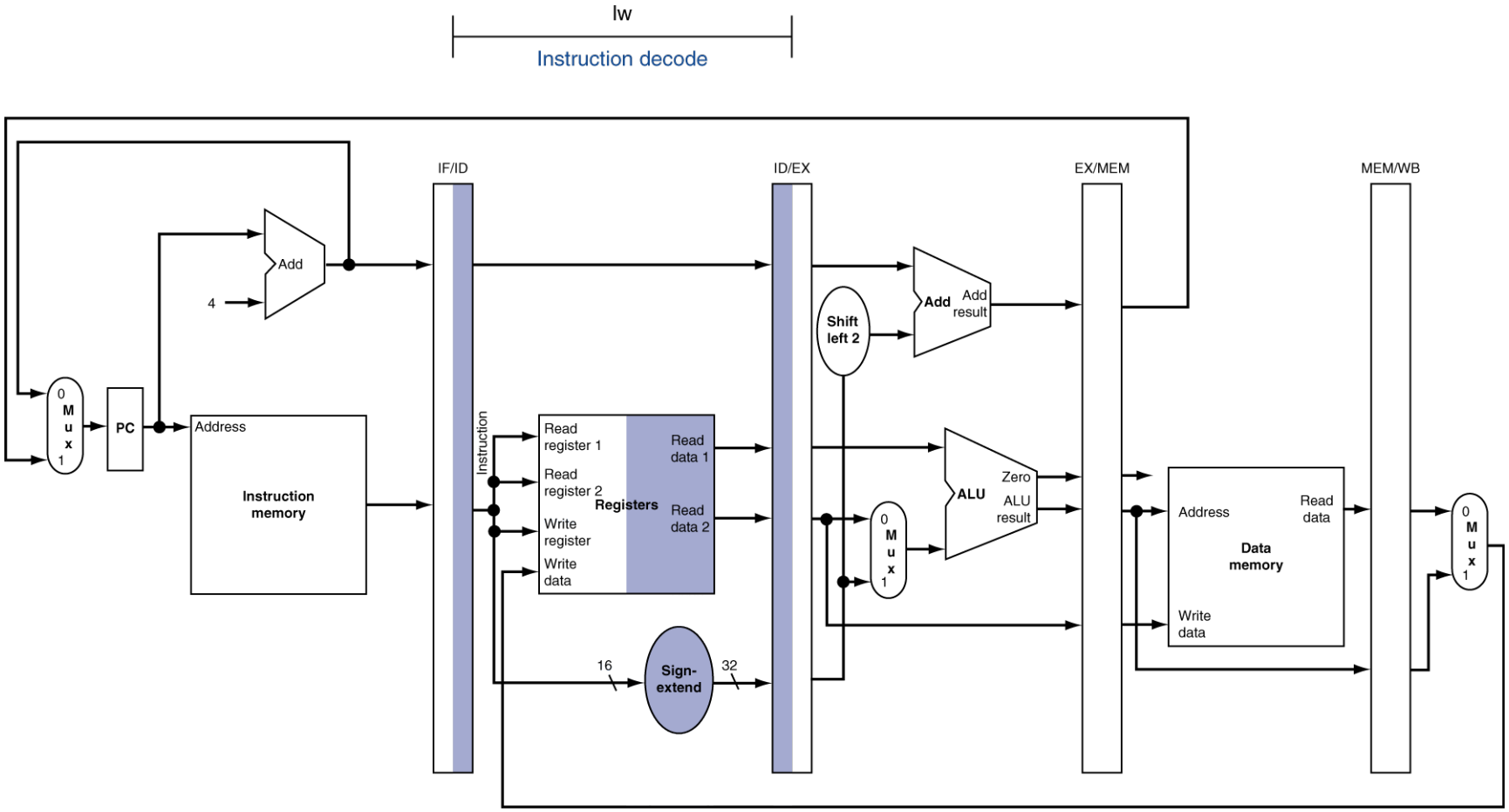
Funkcjonowanie potoku

- Przepływ instrukcji cykl po cyklu przez ścieżkę danych z potokiem
  - Diagram potoku w jednym cyklu zegarowym
    - Pokazuje użycie potoku w jednym cyklu
    - Wskazuje użyte zasoby
  - wobec diagramu dla wielu cykli
    - Graf operacji w czasie
- Spójrzmy na diagramy w jednym cyklu dla instrukcji load & store

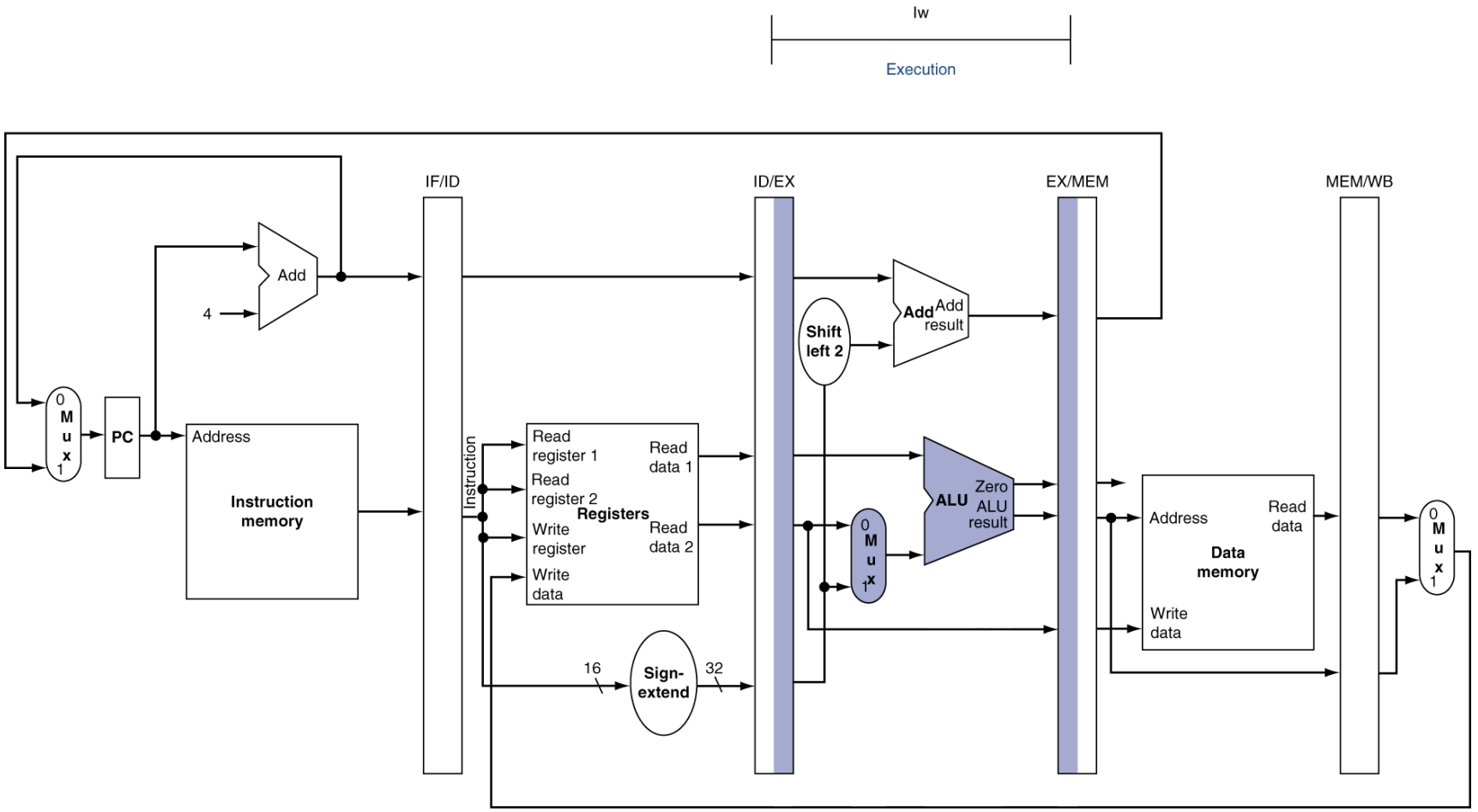
IF dla Load, Store, …



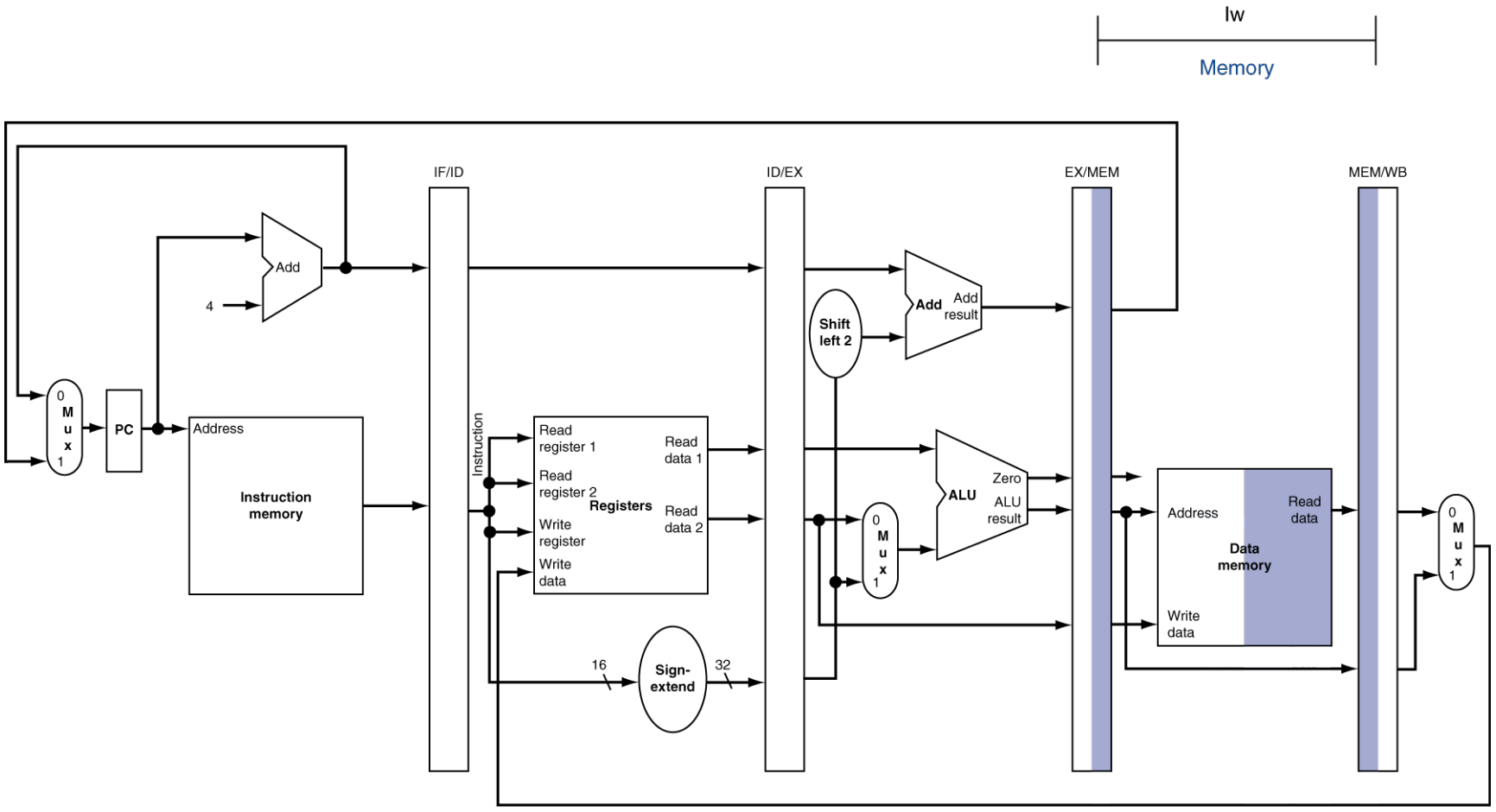
ID dla Load, Store, …



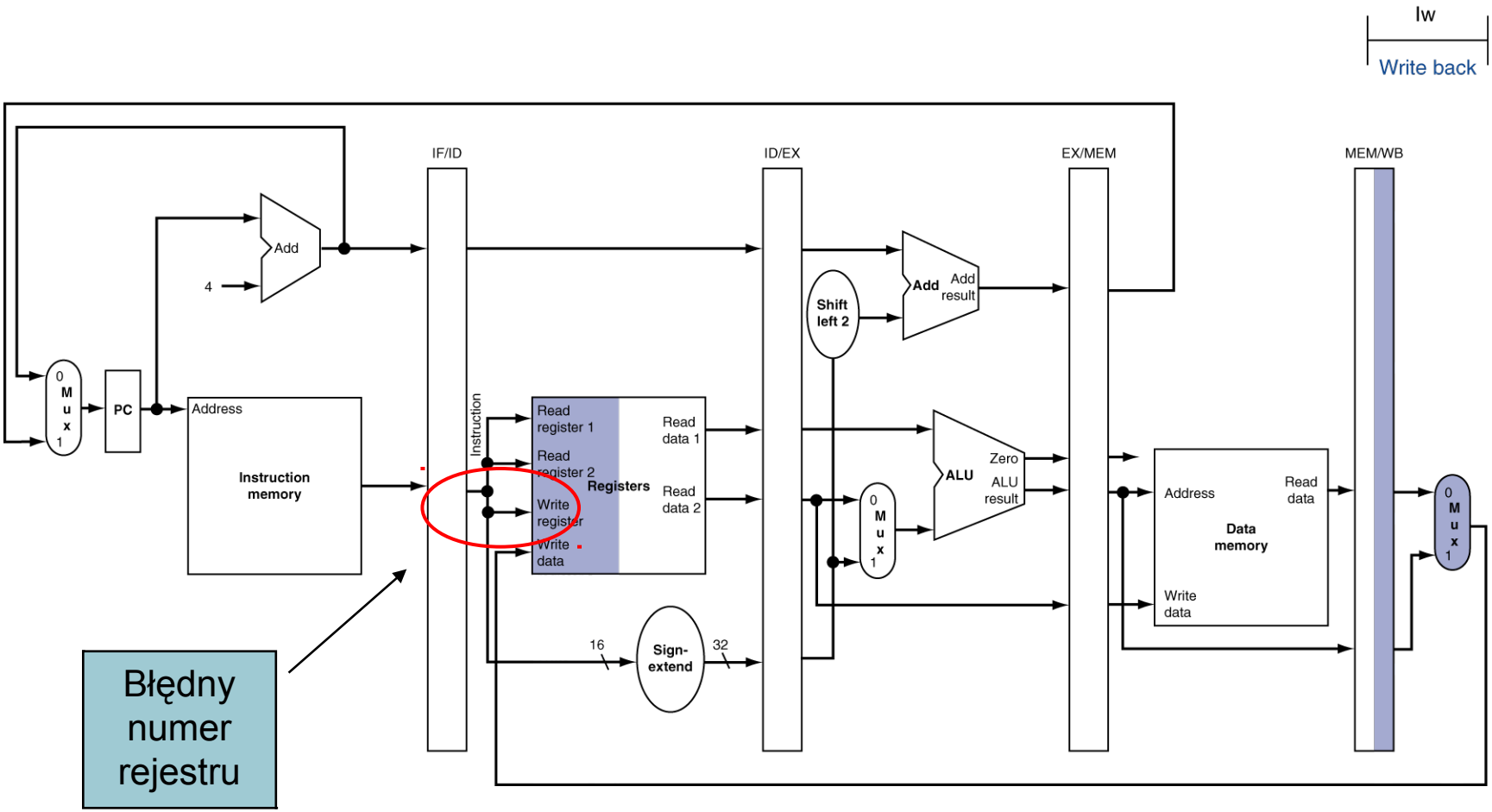
EX dla Load


lw
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
16
Sign-extend
32

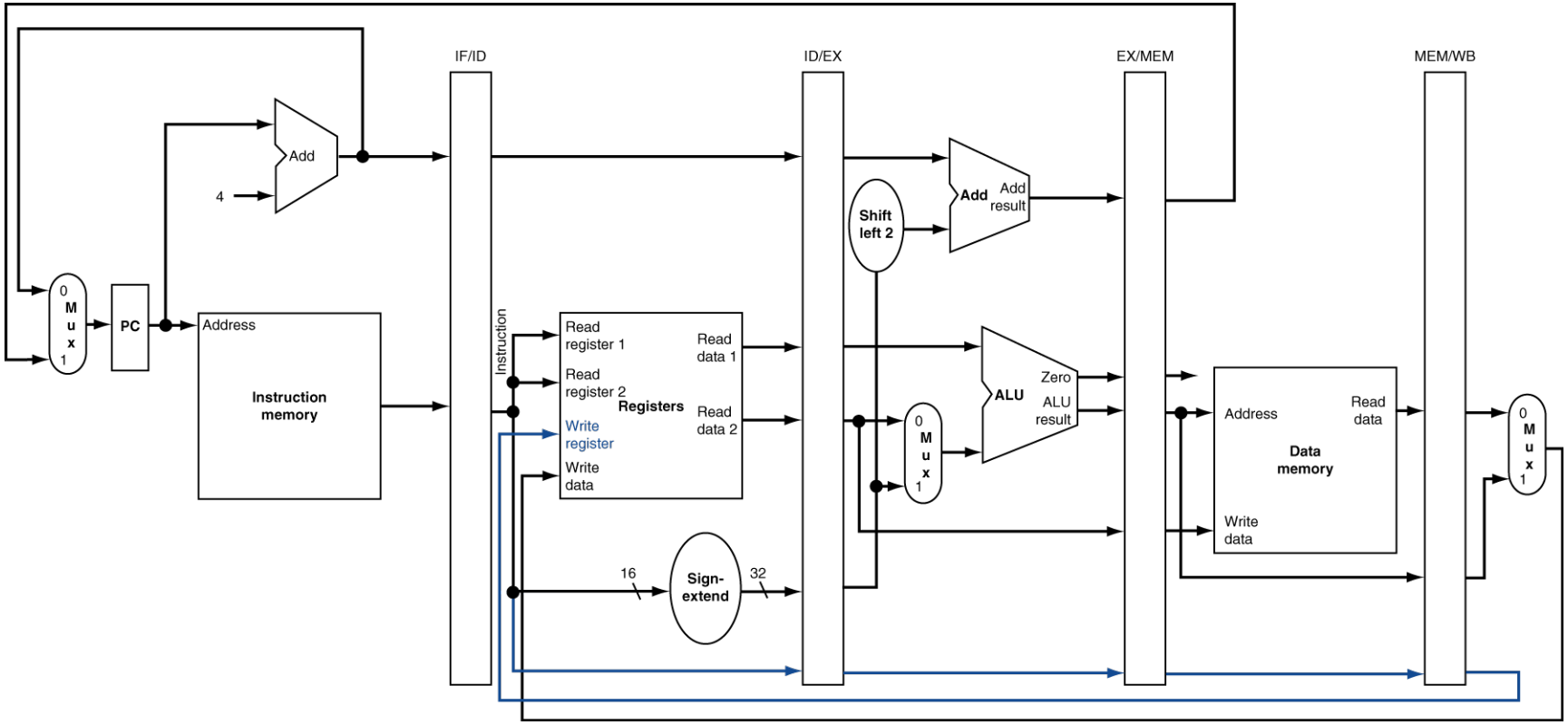
MEM dla Load


lw
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign-extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Data memory
Read data
Write data

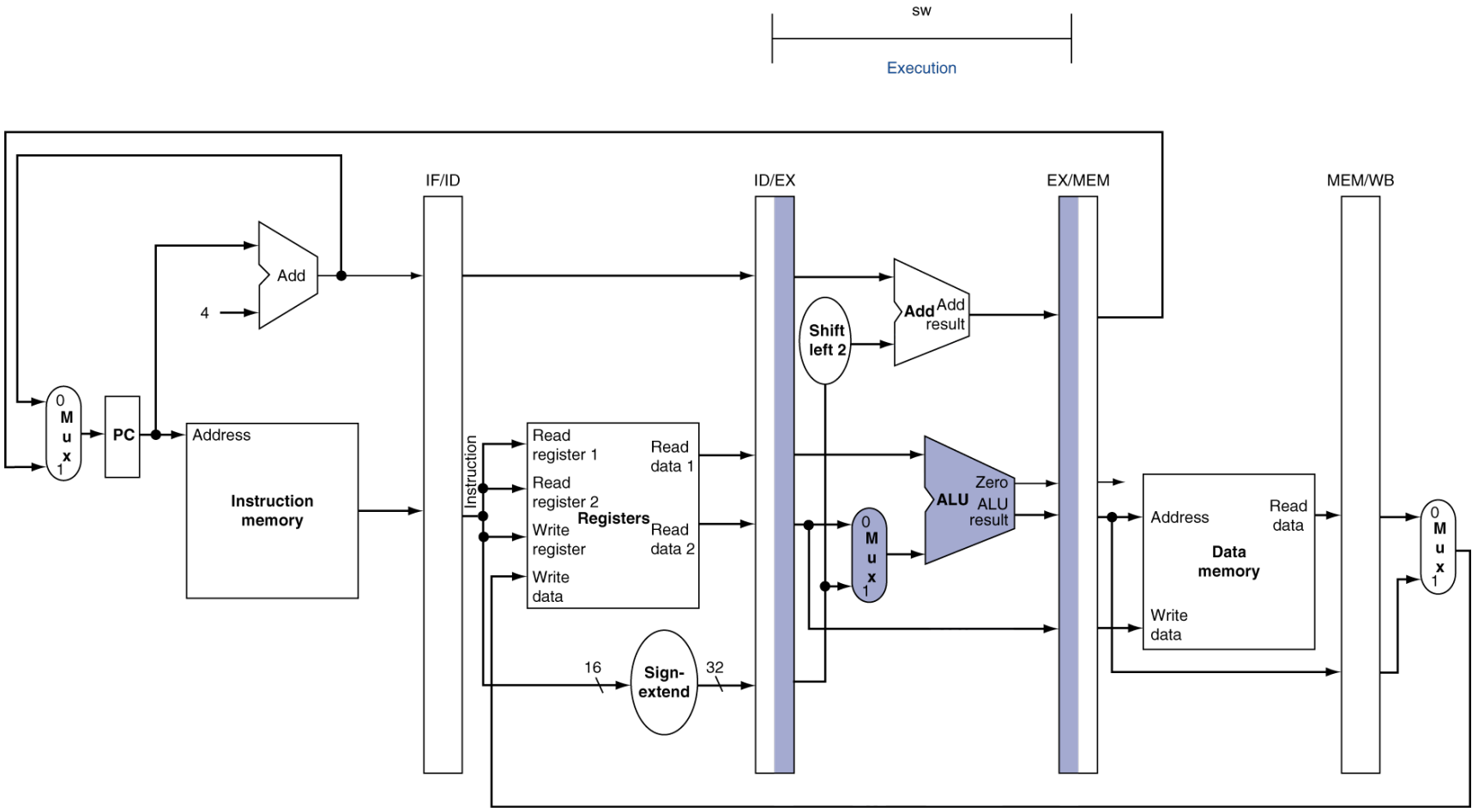
WB dla Load


lw
Write back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
16
Sign-extend
32
Błędny numer rejestru

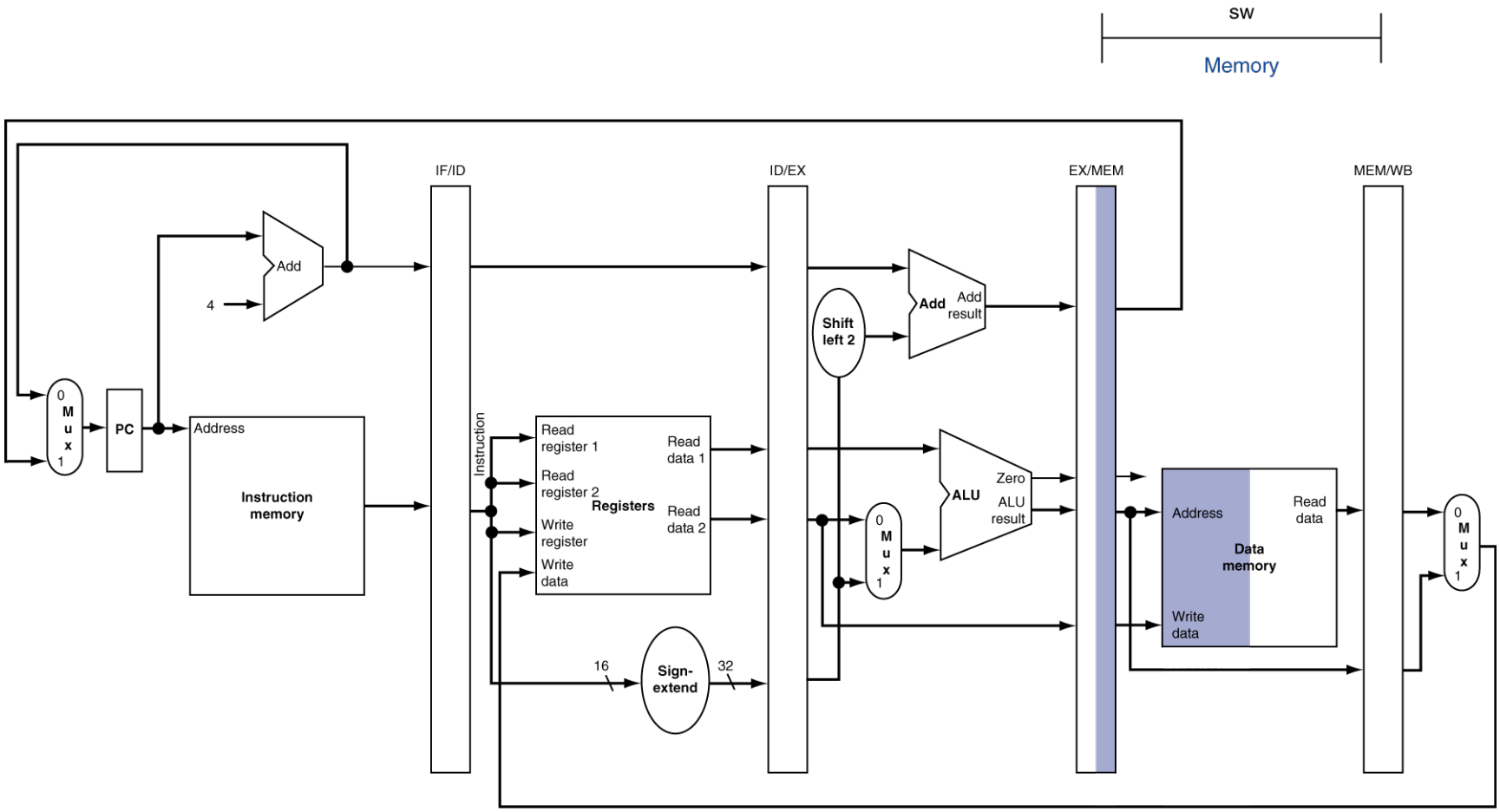
Poprawiona ścieżka danych



EX dla Store


sw
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign-extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data

MEM dla Store



WB dla Store

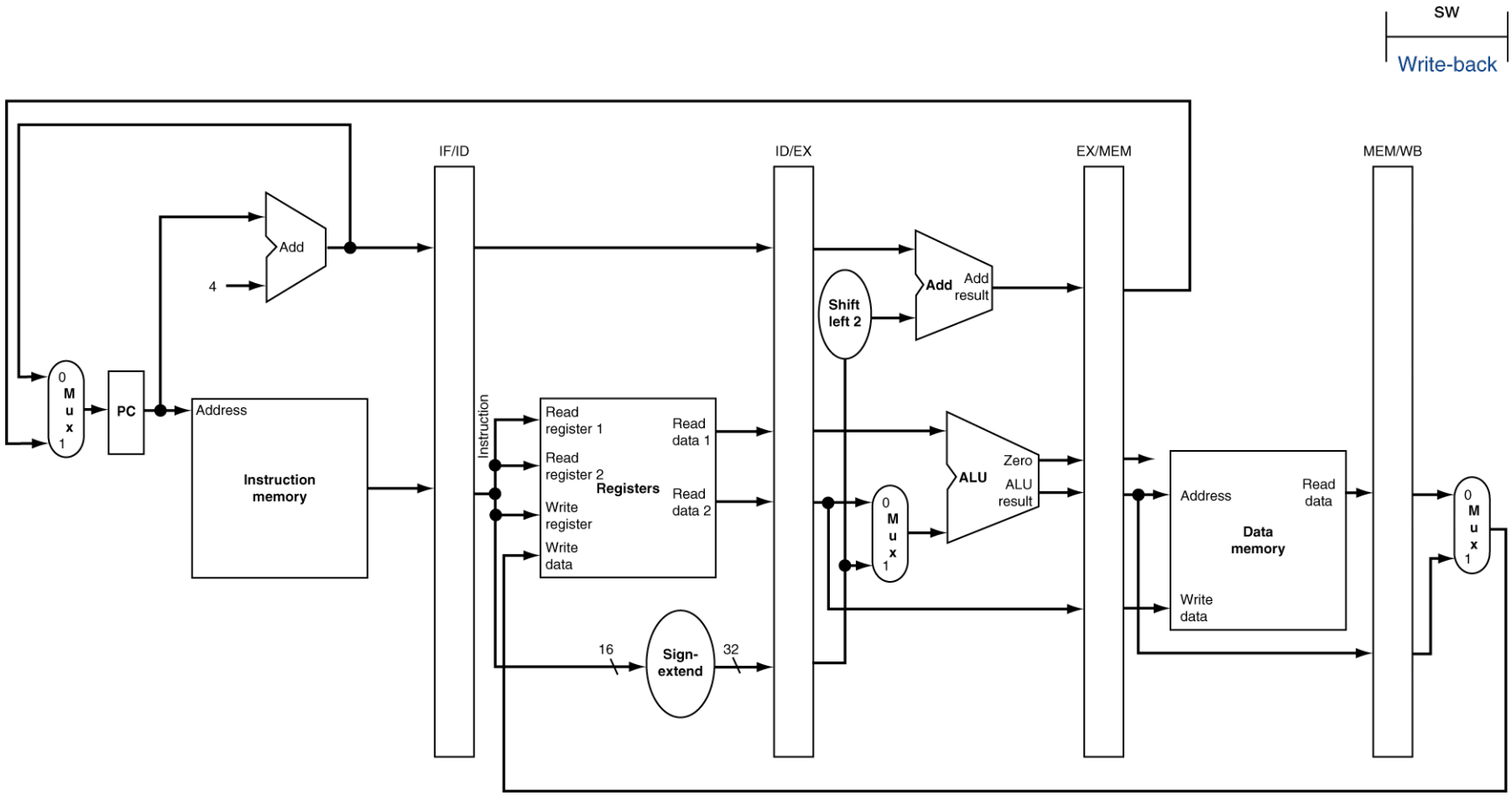

SW
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign-extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data

Diagram potoku – wiele cykli

- Postać ukazująca użycie zasobów

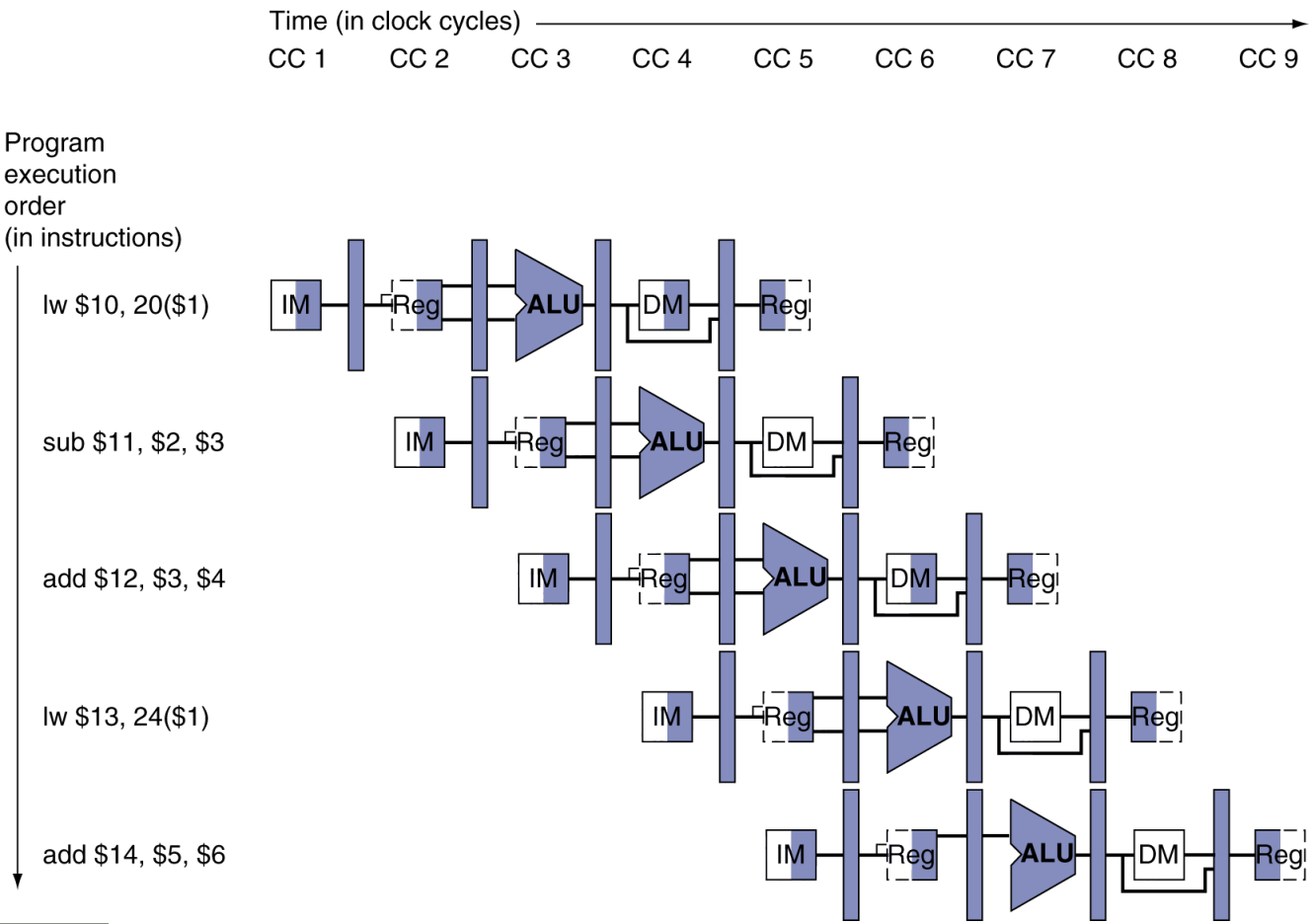


Diagram potoku – wiele cykli

- Postać tradycyjna

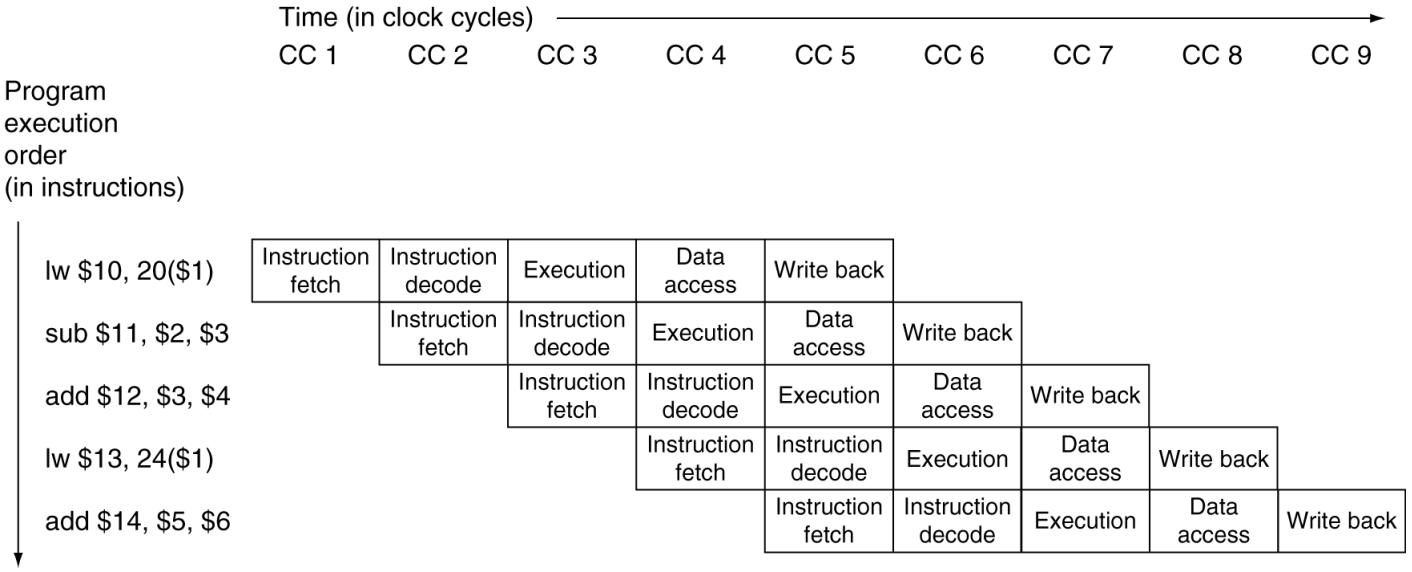
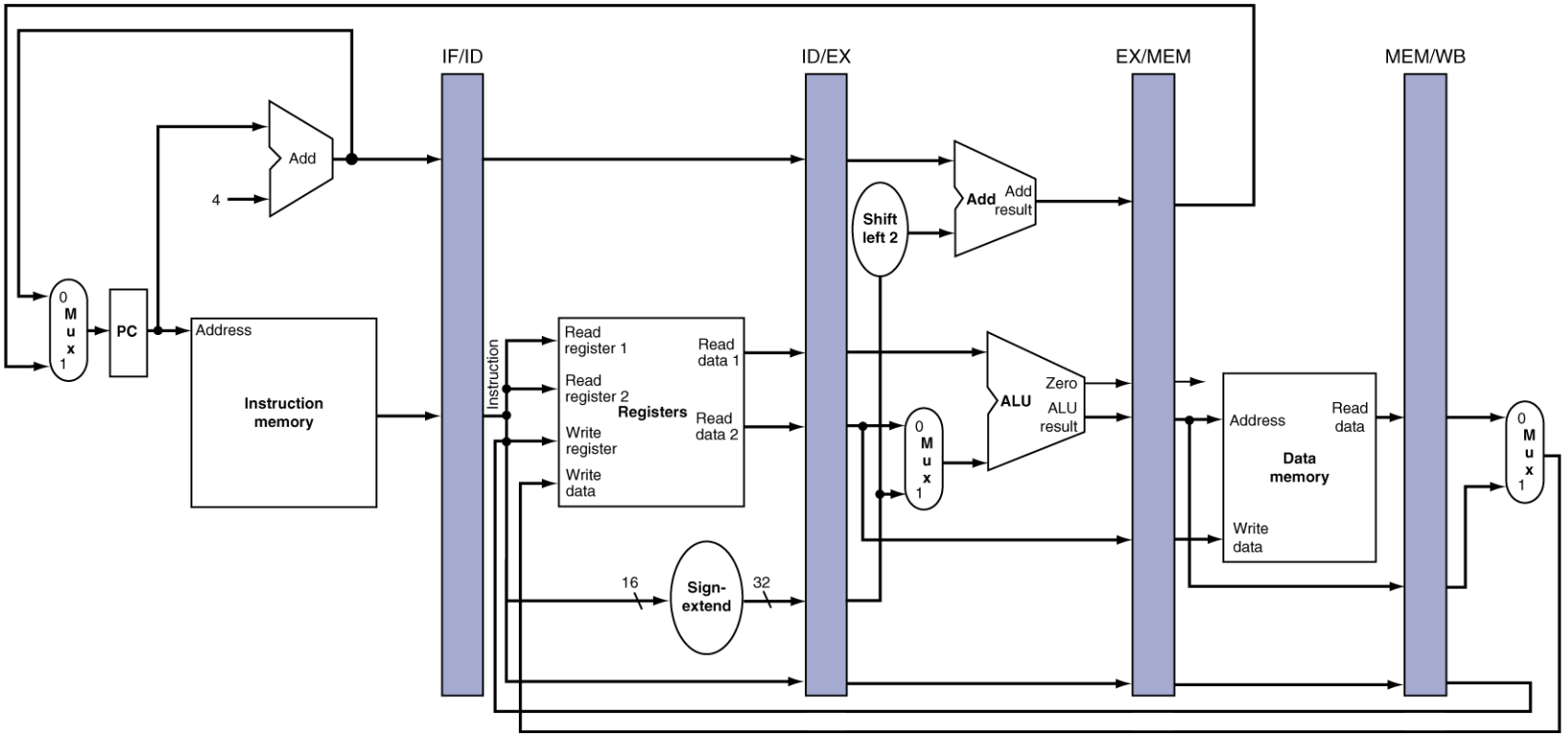


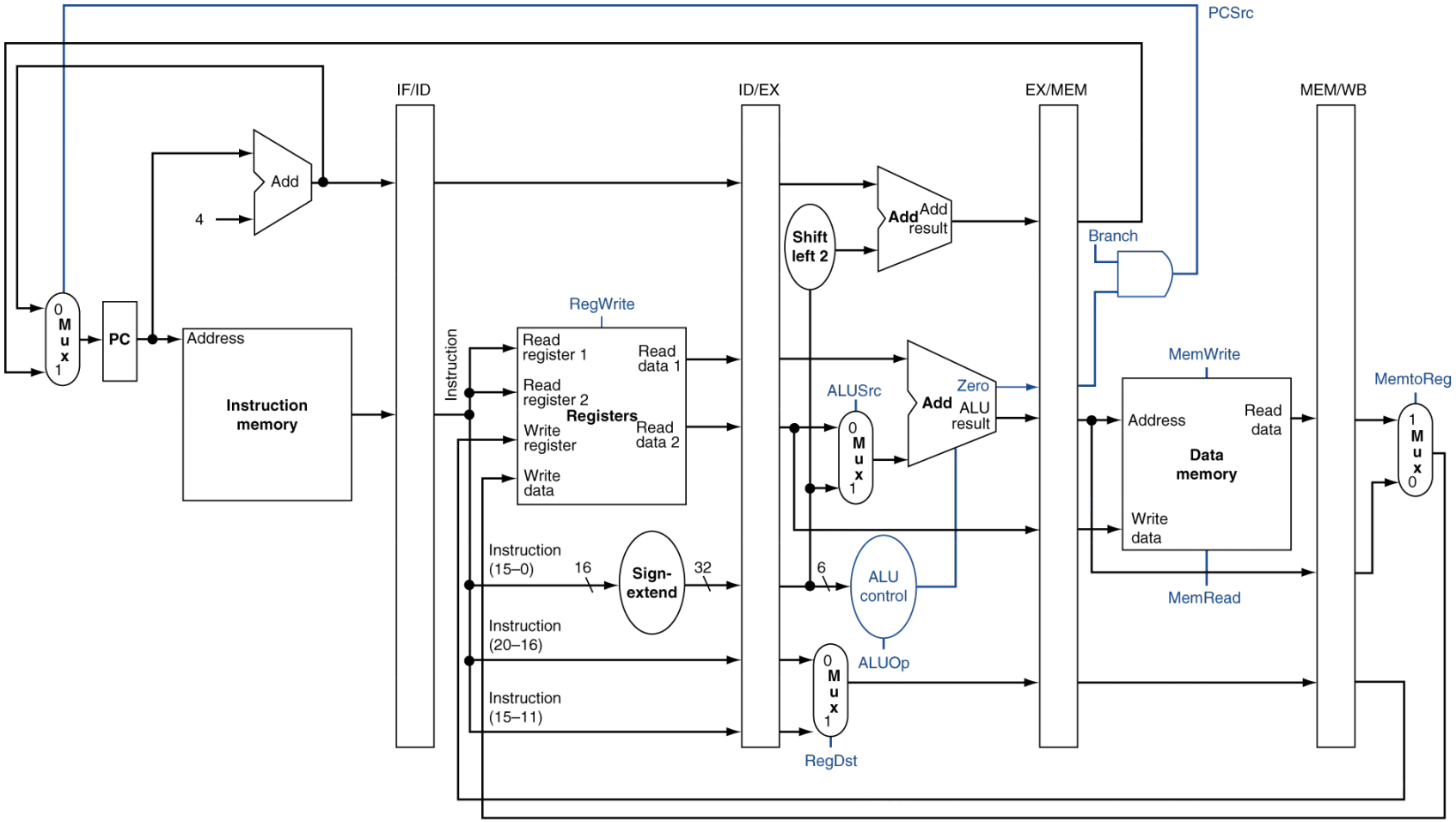
Diagram potoku – jeden cykl

- Stan potoku w określonym cyklu

add $14, $5, $6	lw $13, 24 ($1)	add $12, $3, $4	sub $11, $2, $3	lw $10, 20($1)
Instruction fetch	Instruction decode	Execution	Memory	Write-back

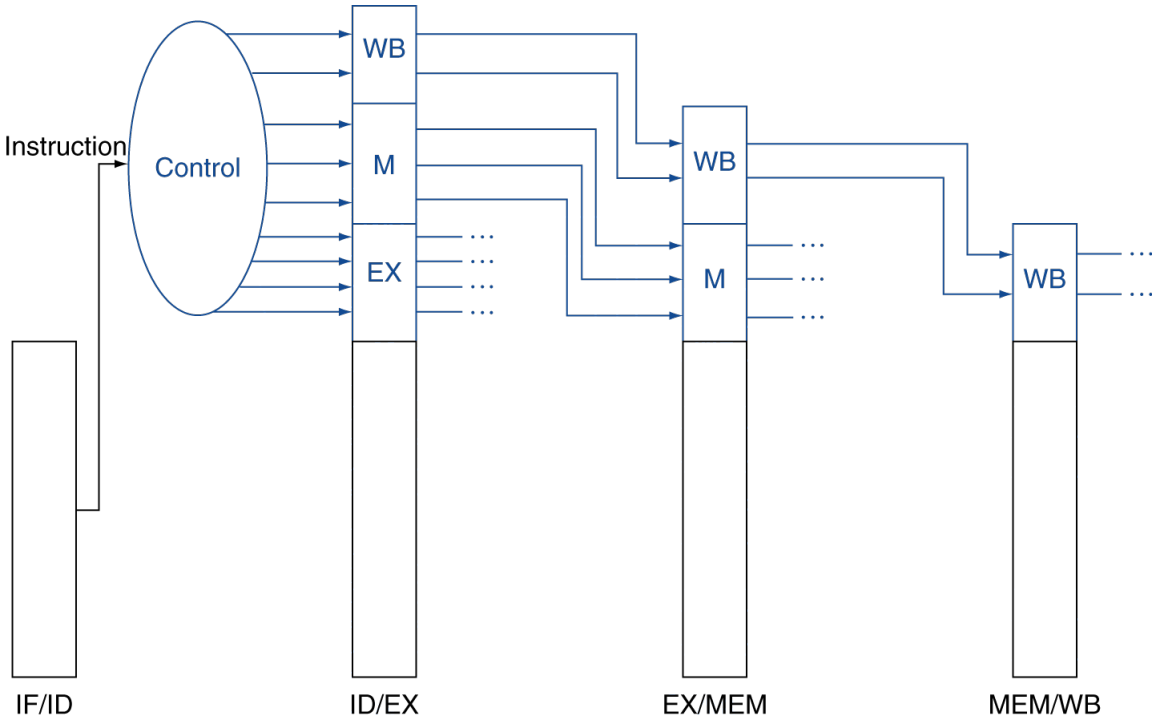


Sterowanie potoku (szkic)

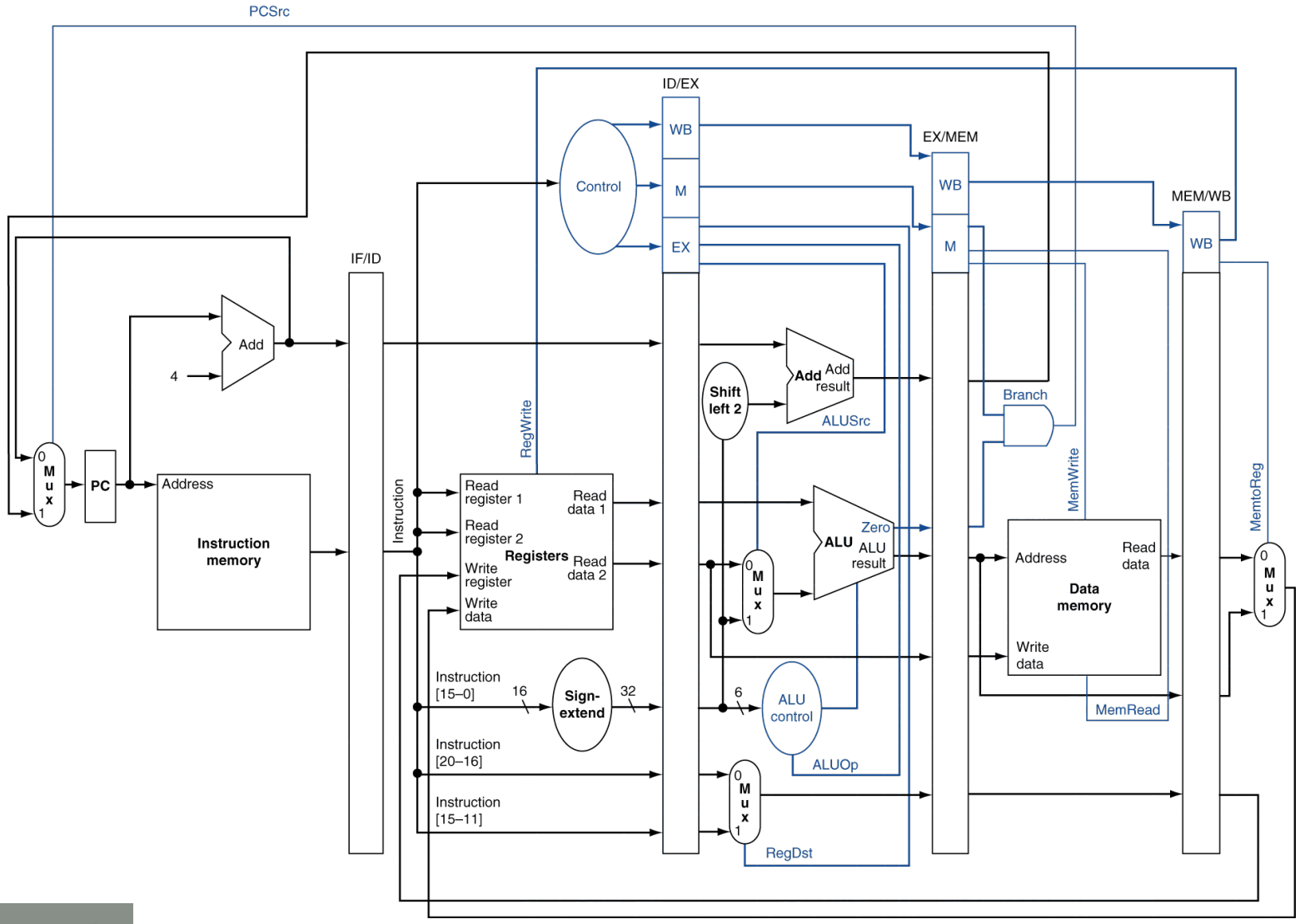


Sterowanie potoku

- Sygnały sterujące wyliczone z instrukcji
  - Jak w implementacji jednocyklowej



Sterowanie potoku



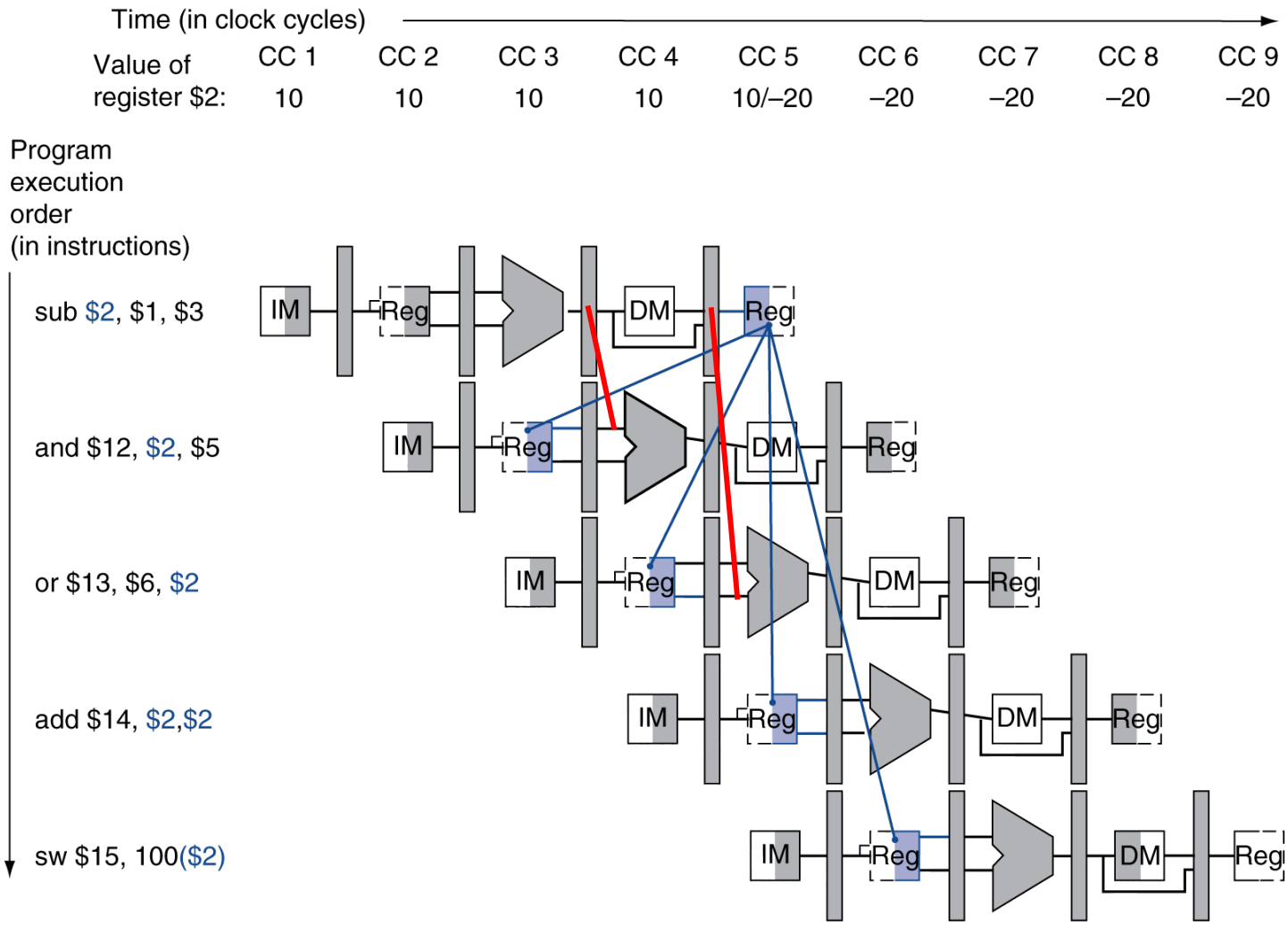
Zagrożenia danych – instr. ALU

- Rozważ następującą sekwencję:

```
sub $2, $1,$3
and $12,$2,$5
or  $13,$6,$2
add $14,$2,$2
sw  $15,100($2)
```

- Można uniknąć zagrożenia przez przekierowanie (forwarding)
  - Jak wykryć konieczność przekierowania?

Zależności & Przekierowanie



Jak wykryć przekierowanie

- Przesyłaj numery rejestrów wzdłuż potoku
  - np. ID/EX.RegisterRs = numer rejestru Rs znajdyjący się w rejestrze potoku ID/EX
- Numery rejestrów z operandami ALU w stanie EX są dane przez
  - ID/EX.RegisterRs, ID/EX.RegisterRt
- Zagrożenia danych gdy
  1a. EX/MEM.RegisterRd = ID/EX.RegisterRs
  1b. EX/MEM.RegisterRd = ID/EX.RegisterRt
  2a. MEM/WB.RegisterRd = ID/EX.RegisterRs
  2b. MEM/WB.RegisterRd = ID/EX.RegisterRt

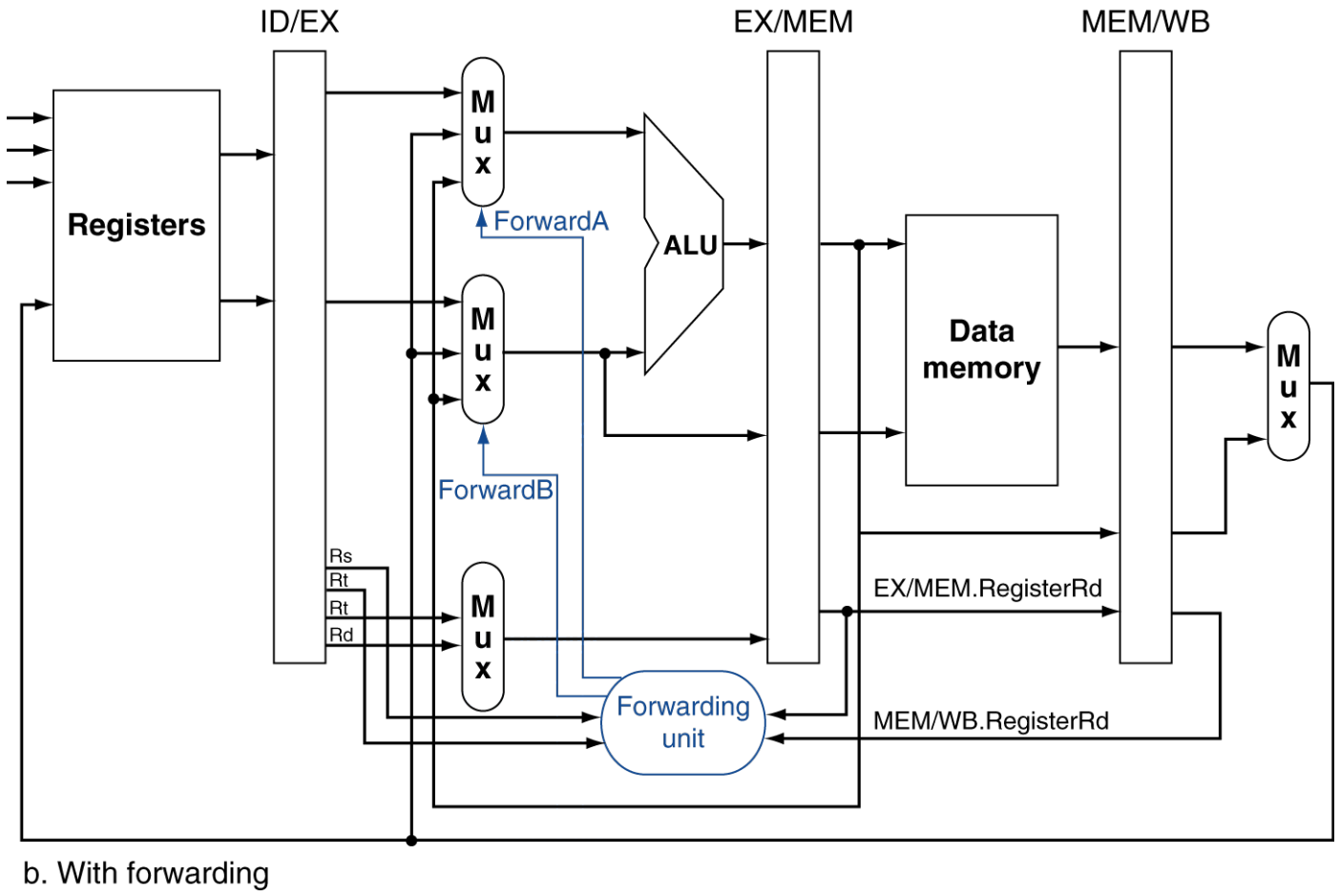
Fwd z rej. potoku EX/MEM

Fwd z rej. potoku MEM/WB

Jak wykryć przekierowanie

- Ale tylko gdy przekierowywana instrukcja zapisuje wynik do rejestru!
  - EX/MEM.RegWrite, MEM/WB.RegWrite
- I tylko gdy Rd dla tej instrukcji nie jest $zero
  - EX/MEM.RegisterRd ≠ 0, MEM/WB.RegisterRd ≠ 0

Ścieżki przekierowania



b. With forwarding

Warunki przekierowania

- Zagrożenie w EX
  - if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
    ForwardA = 10
  - if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
    ForwardB = 10
- Zagrożenie w MEM
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
    ForwardA = 01
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
    ForwardB = 01

Podwójne zagrożenie danych

- Rozważ sekwencję:

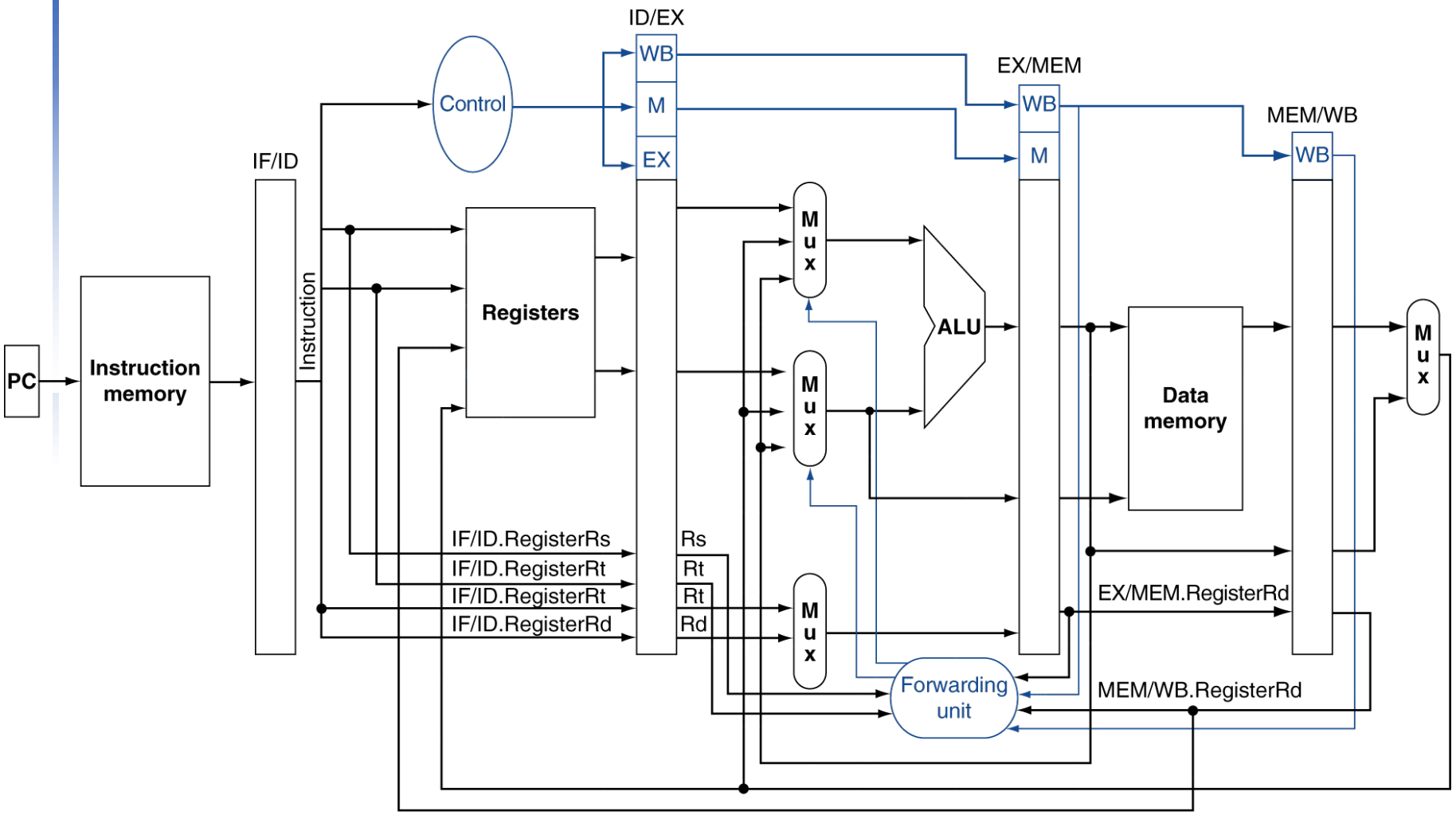
```
add $1,$1,$2
add $1,$1,$3
add $1,$1,$4
```

- Oba zagrożenia wystąpią
  - Chcemy użyć najświeższy wynik
- Zrewiduj warunek zagrożenia w MEM
  - Przekierowywuj tylko jeśli warunek zagrożenia w EX jest równy fałsz

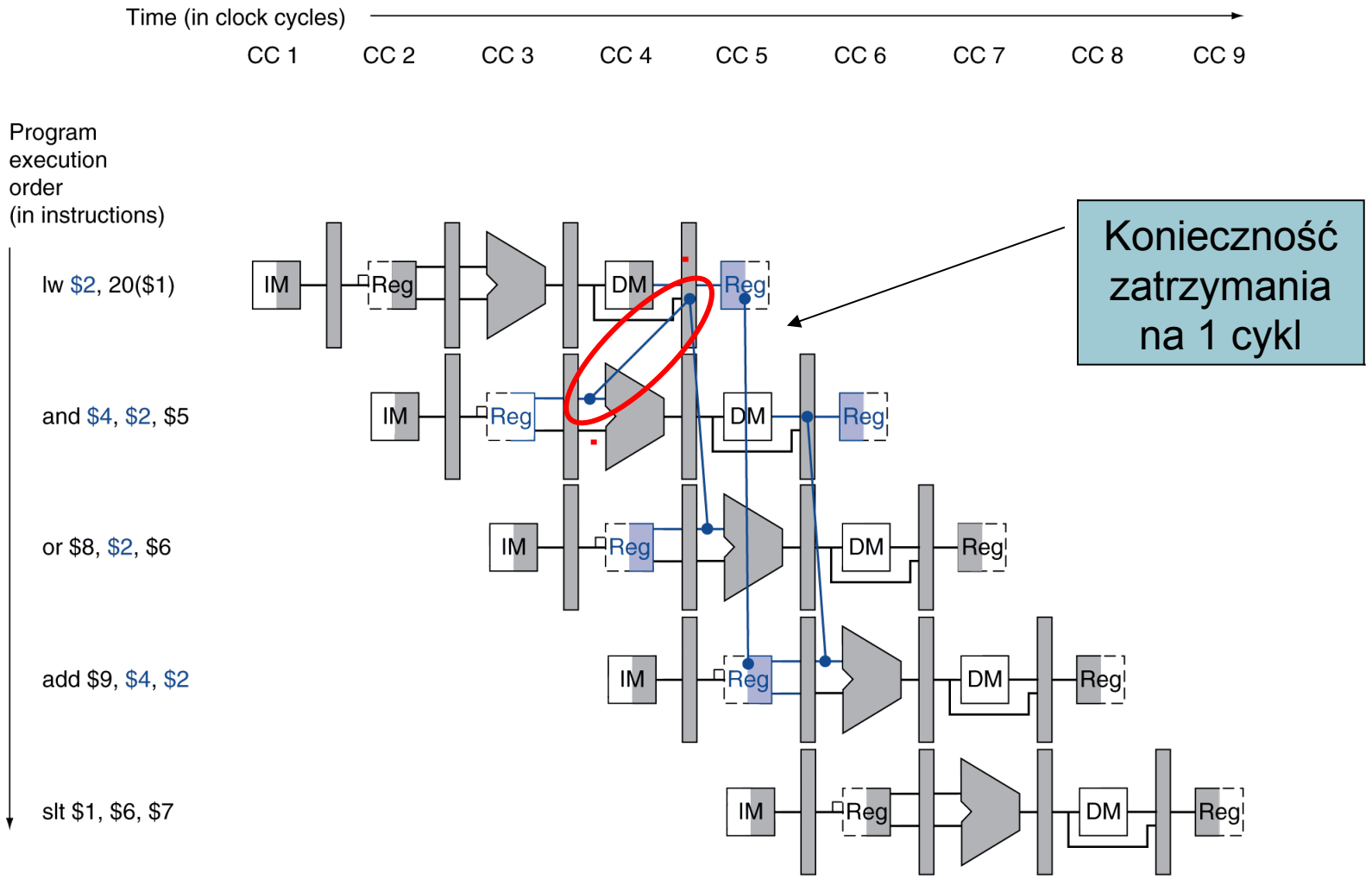
Rewizja warunku przekierowania

- Zagrożenie w MEM
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
    ForwardA = 01
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
    ForwardB = 01

Ścieżka d. z przekierowaniem



Zagrożenie danych przy Load



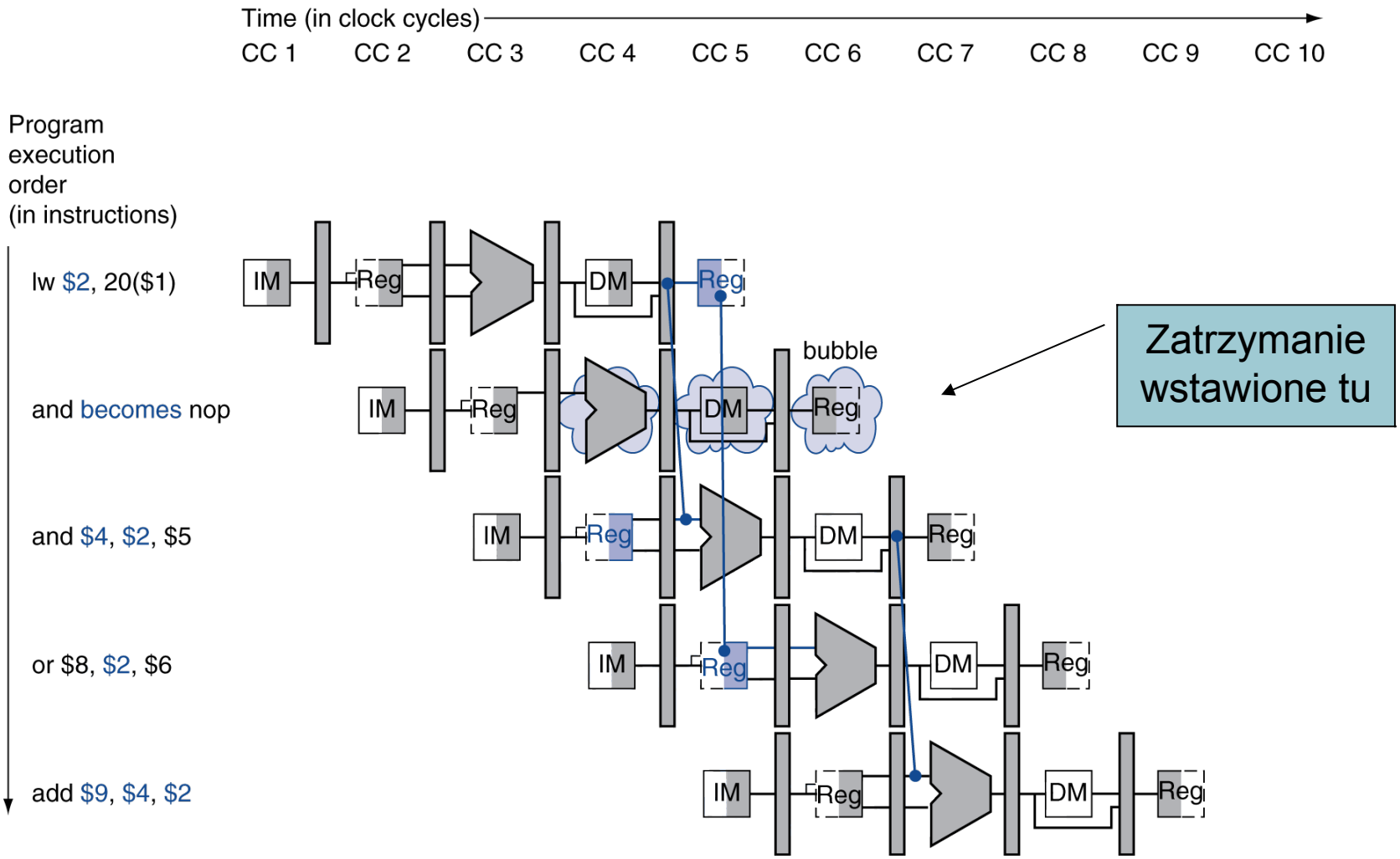
Zagrożenie danych przy Load

- Sprawdź kiedy instrukcja korzystająca z Load jest w stanie ID
- Numery rejestrów będącymi operandami ALU w stanie ID są dane przez
  - IF/ID.RegisterRs, IF/ID.RegisterRt
- Zagrożenie danych przy Load gdy
  - ID/EX.MemRead and
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or
    (ID/EX.RegisterRt = IF/ID.RegisterRt))
- Jeśli wykryte, zatrzymaj potok (stall) i wstaw bańkę (bubble)

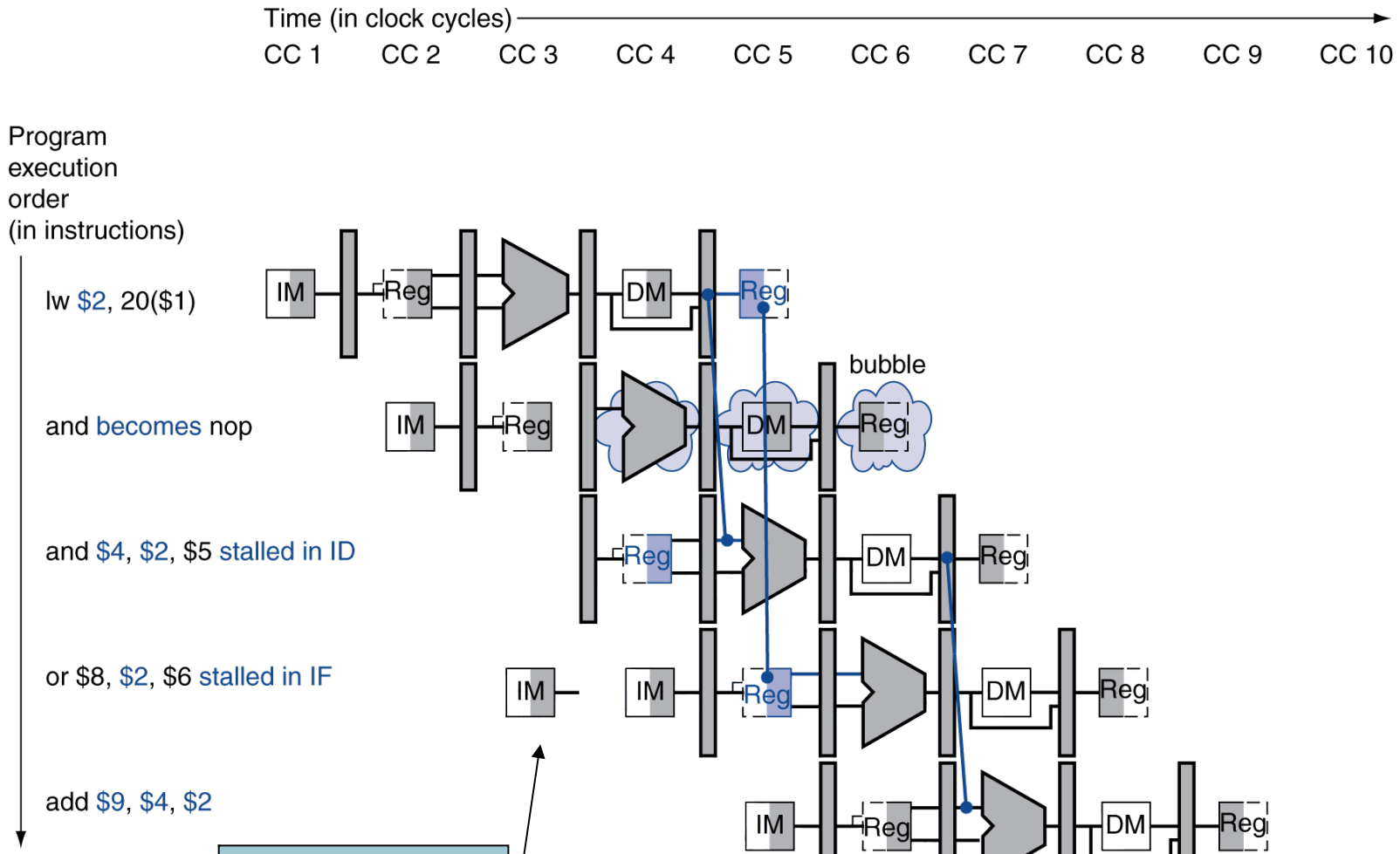
Jak zatrzymać potok

- Ustaw sygnały sterujące w rejestrze ID/EX na 0
  - EX, MEM i WB wykonują `nop` (no-operation)
- Zapobież aktualizacji PC i rejestru IF/ID
  - Instrukcja za Load jest ponownie dekodowana
  - Dalsza instrukcja jest ponownie ładowana
  - 1-cyklowe zatrzymanie pozwala korzystać z danych stanu MEM instrukcji `lw`
    - Można dalej przekierować je do stanu EX

Zatrzymanie/Bańka w potoku

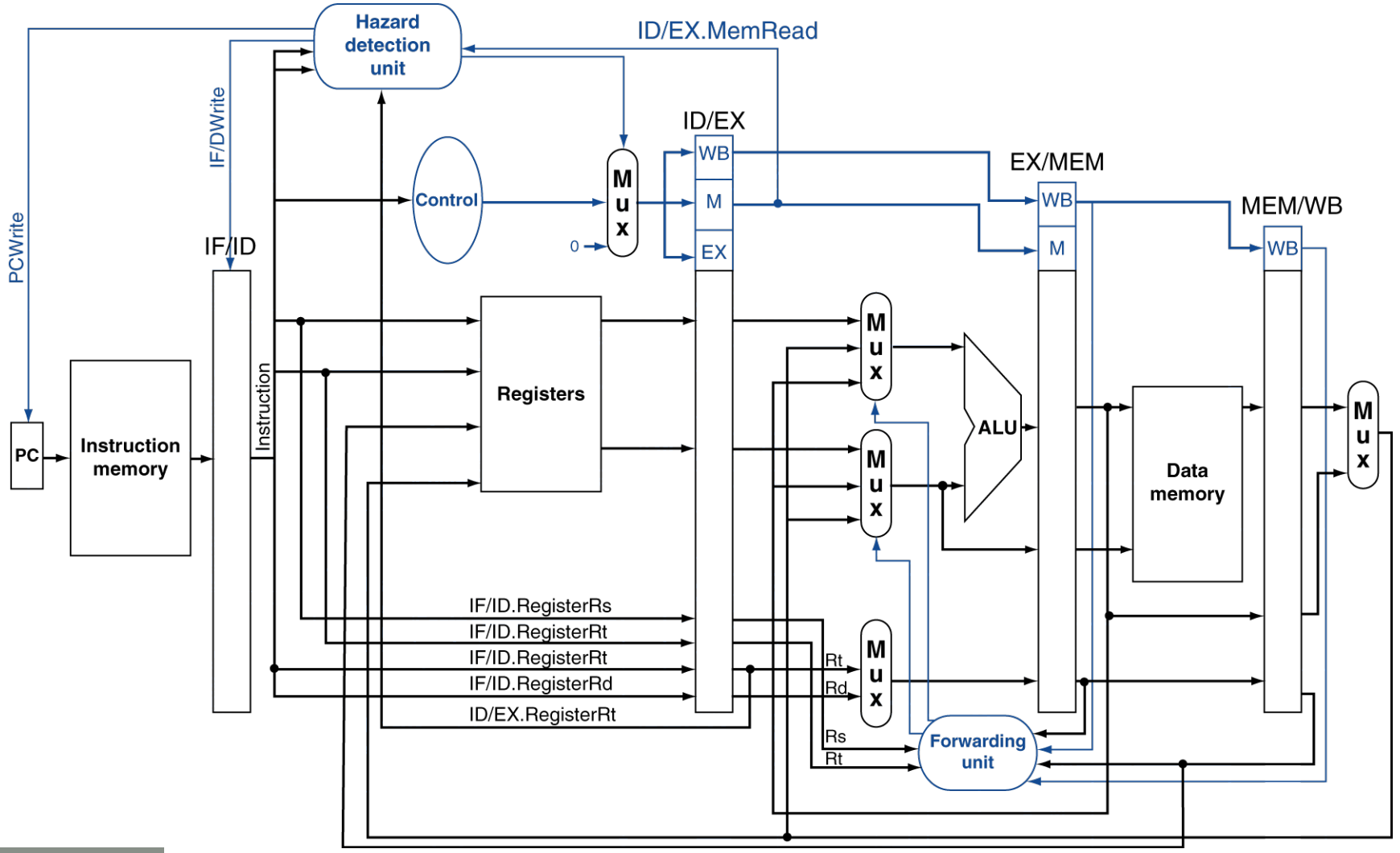


Zatrzymanie/Bańka w potoku



Albo dokładniej…

Ścieżka d. z detekcją zagrożeń



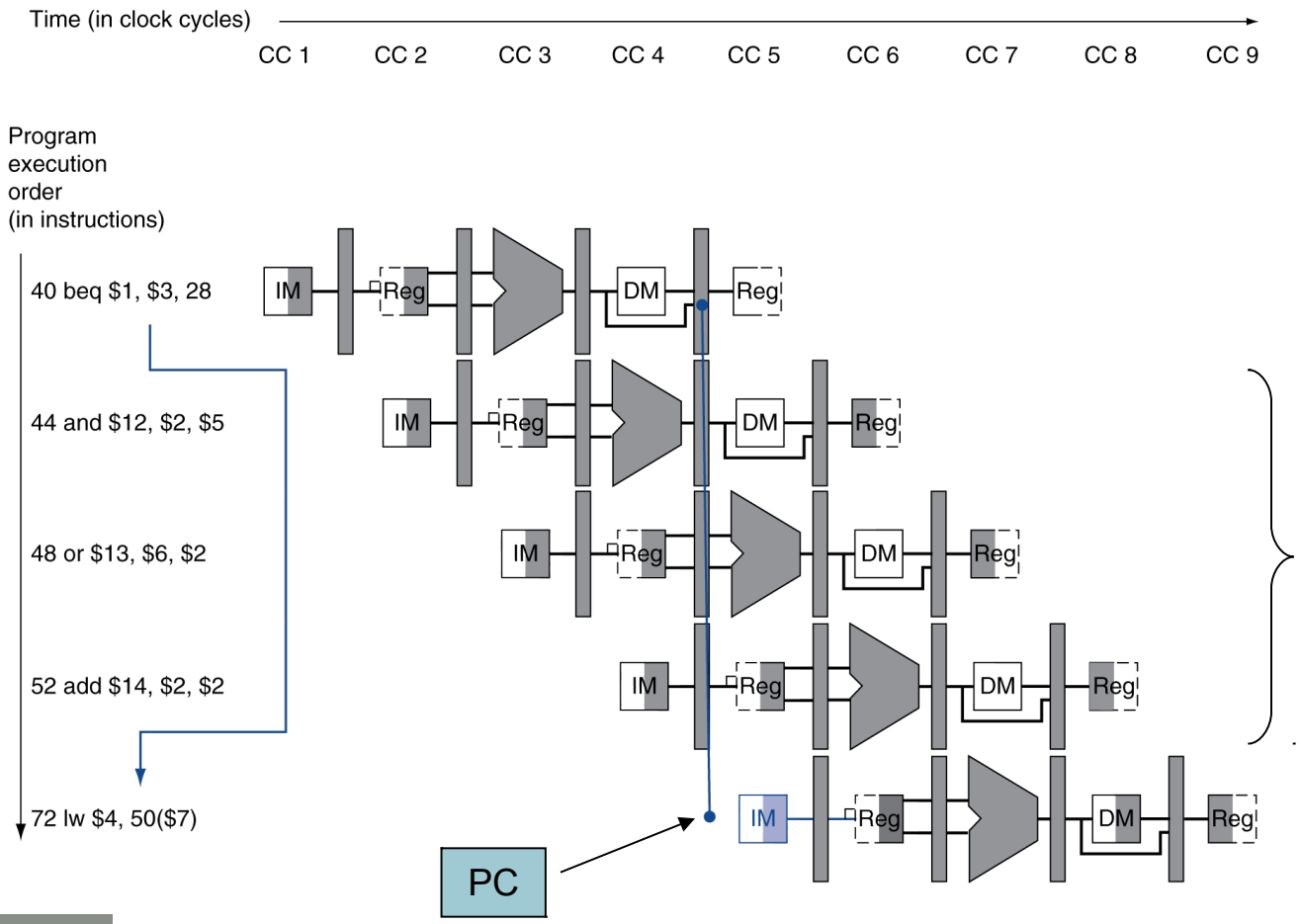
Zatrzymania a wydajność

The BIG Picture

- Zatrzymania zmniejszają wydajność
  - Ale są konieczne dla otrzymania prawidłowych wyników
- Kompilator może tak zorganizować kod by uniknąć zagrożeń i zatrzymań
  - Musi znać strukturę potoku

Zagrożenia przy skokach

- Jeśli wynik skoku określony w MEM

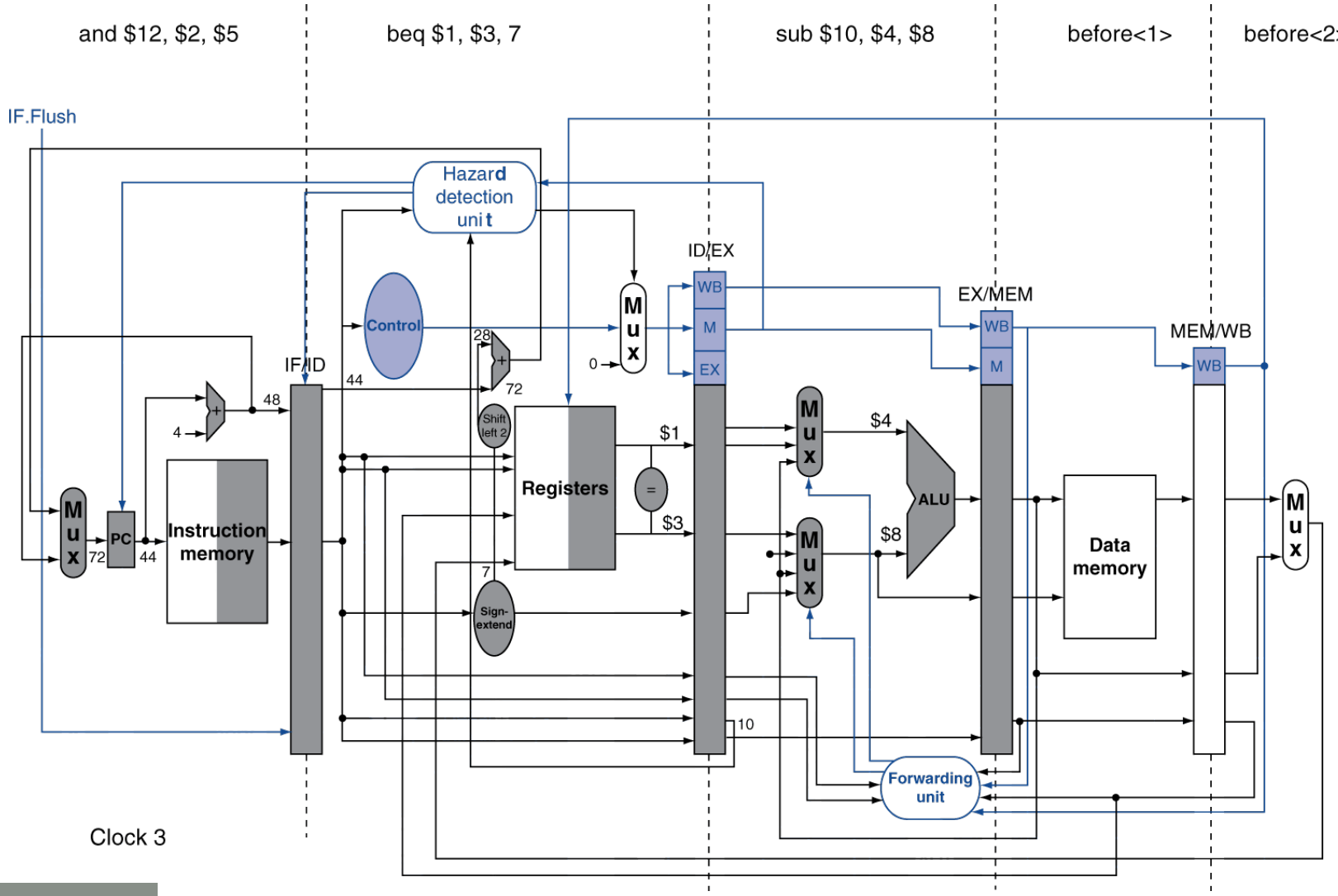


Skracanie opóźnienia skoku

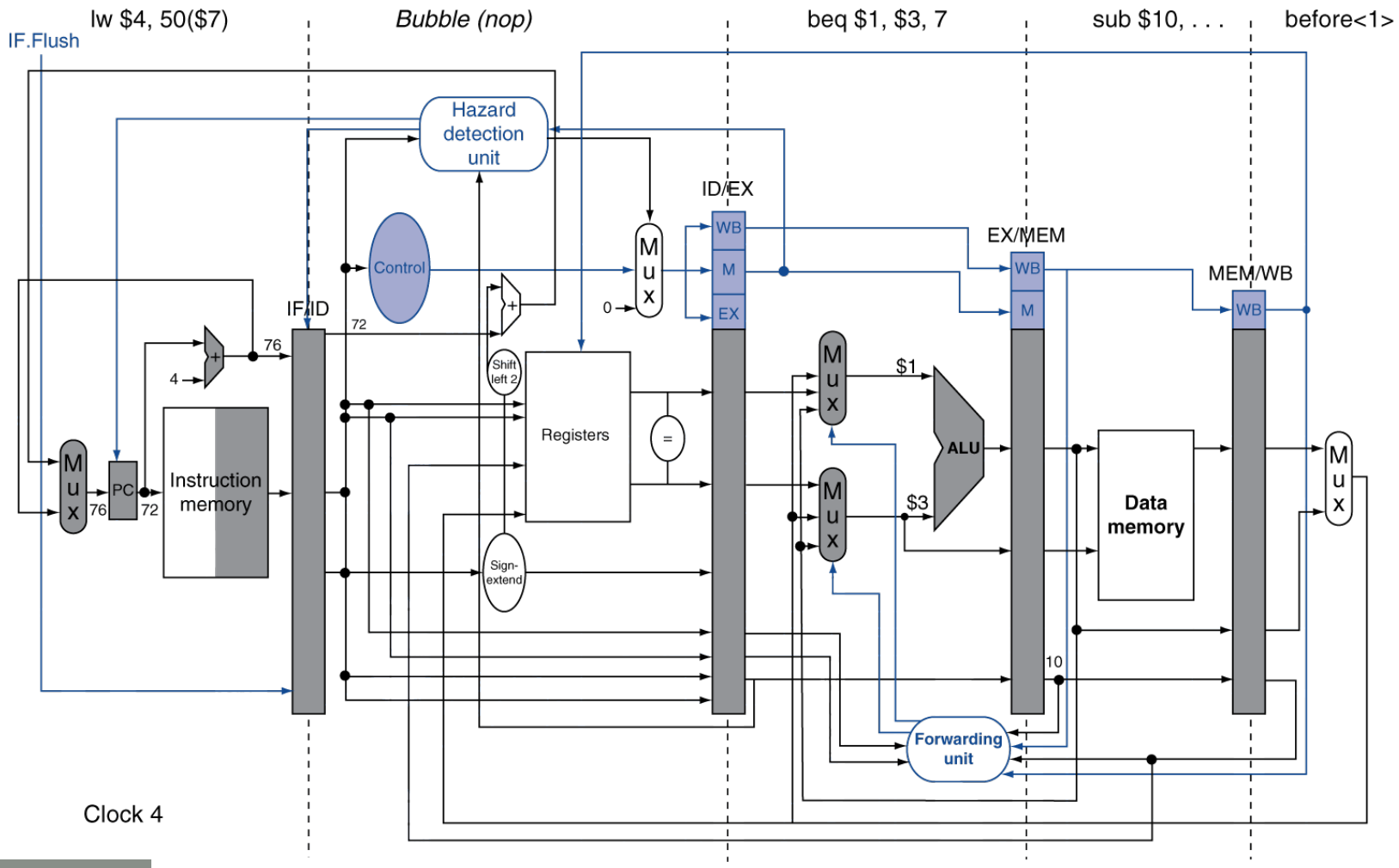
- Przesuń sprzęt determinujący wynik do stanu ID
  - Sumator dla adresu docelowego
  - Układ porównujący rejestry
- Przykład: skok następuje

```
36:   sub   $10, $4, $8
40:   beq   $1,  $3, 7
44:   and   $12, $2, $5
48:   or    $13, $2, $6
52:   add   $14, $4, $2
56:   slt   $15, $6, $7
      ...
72:   lw    $4, 50($7)
```

Przykład: skok następuje



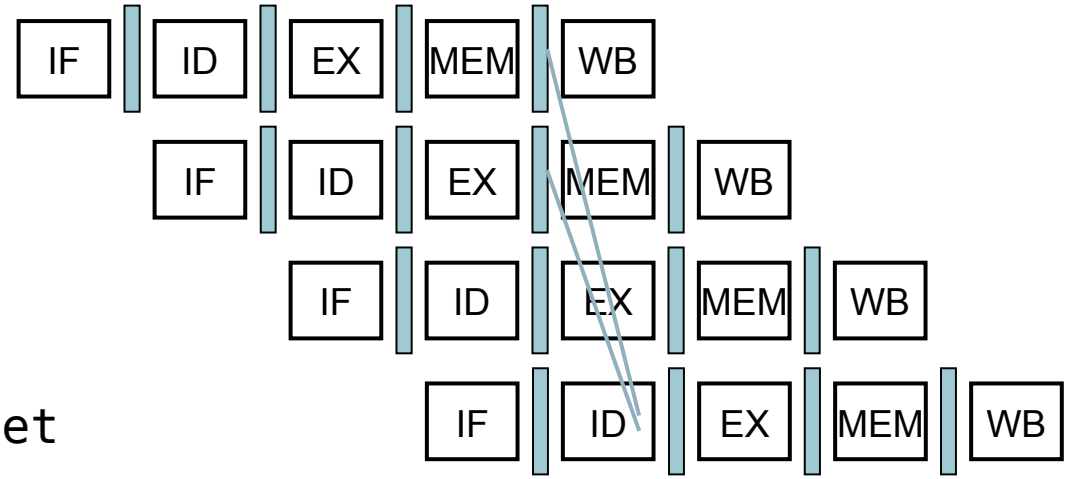
Przykład: skok następuje



Zagrożenia danych a skoki

- Jeśli badany rejestr jest ładowany w drugiej lub trzeciej instrukcji ALU wstecz

```
add $1, $2, $3
add $4, $5, $6
…
beq $1, $4, target
```

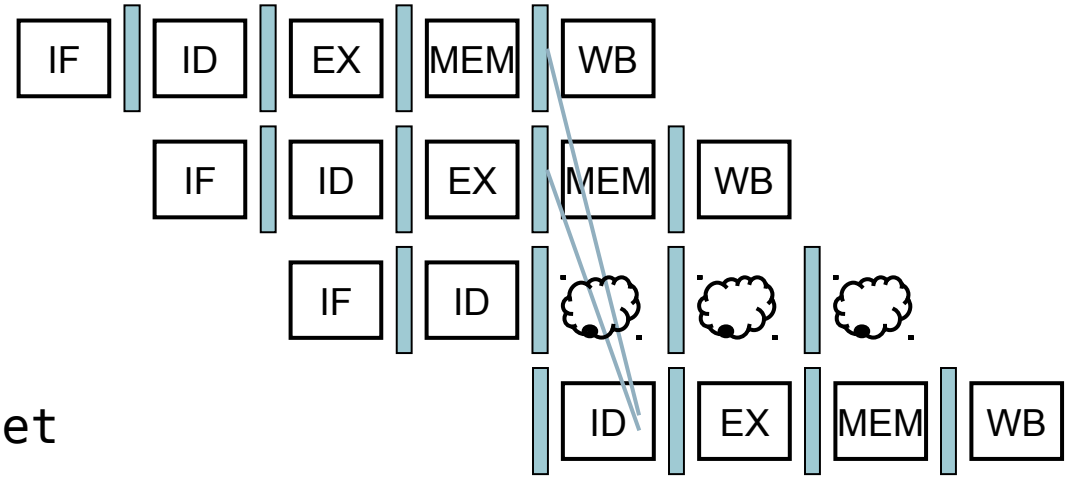


- Można to rozwiązać przez przekierowanie

Zagrożenia danych a skoki

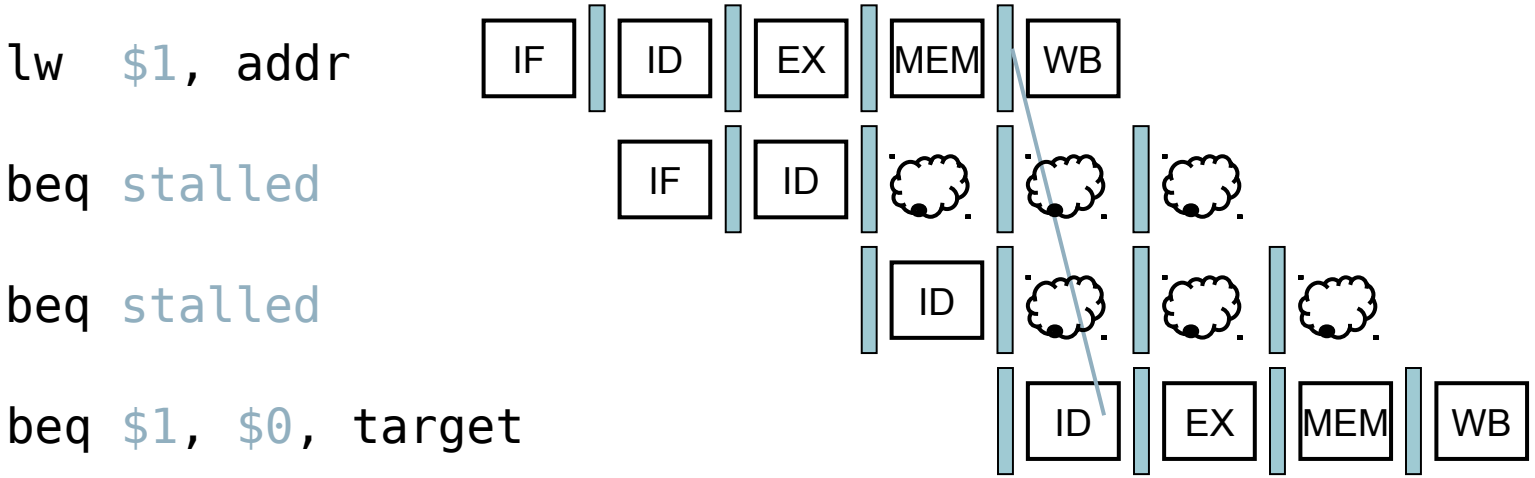
- Jeśli testowany rejestr jest ładowany w poprzedzającej instrukcji ALU lub w instrukcji load dwa kroki wstecz
  - Konieczny jest 1 cykl zatrzymania

```
lw  $1, addr
add $4, $5, $6
beq stalled
beq $1, $4, target
```



Zagrożenia danych a skoki

- Jeśli testowany rejestr jest ładowany w poprzedzającej instrukcji load
  - Konieczne są 2 cykle zatrzymania

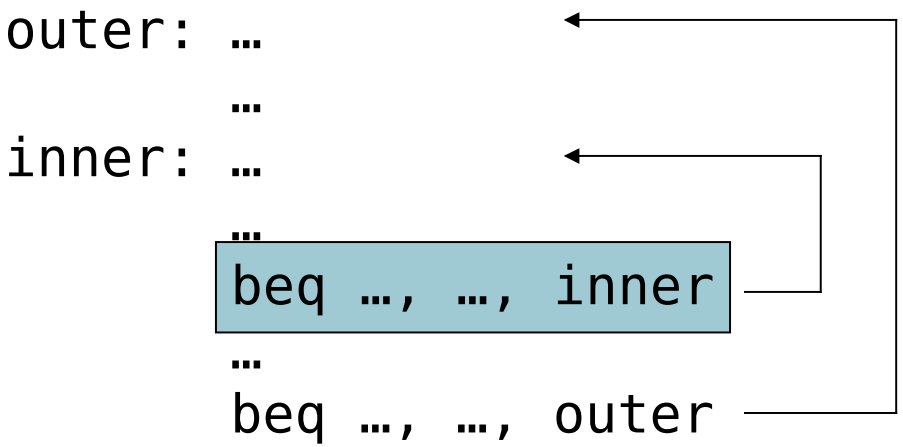


Dynamiczna predykcja skoku

- W głębszych lub superskalarnych potokach koszt zagrożeń przy skoku jest większy
- Użyj predykcji dynamicznej
  - Bufor predykcji skoków (ew. tablica historii skoków)
  - Indeksowany przez adresy ostatnich instrukcji skoku
  - Zapamiętuje wynik (skok wykonany/nie wykonany)
  - W celu wykonania skoku
    - Sprawdź tablicę, spodziewaj się tego samego wyniku
    - Ładuj następne instrukcje zgodnie z predykcją
    - Jeśli jest nieprawidłowa, spłucz potok i odwróć predykcję (w tablicy)

Predyktor 1-bit.: ograniczenia

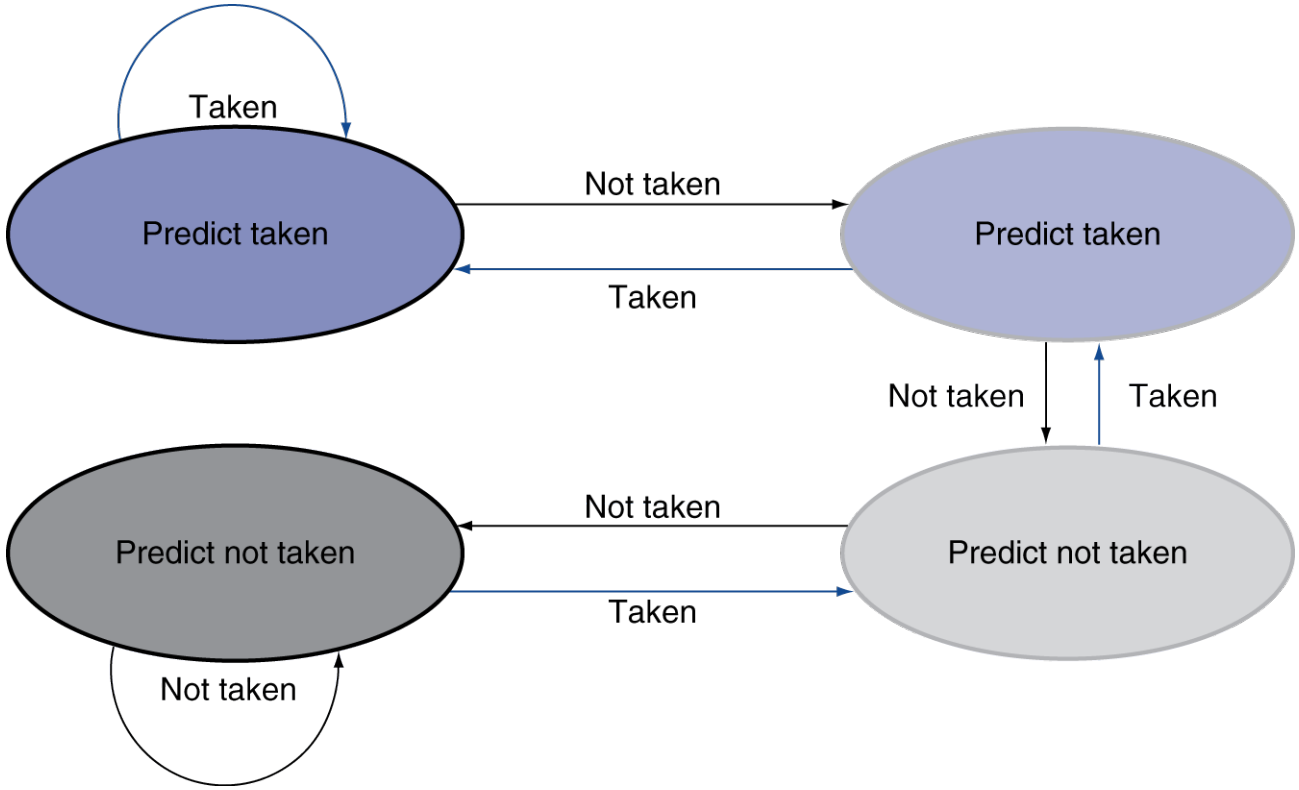
- Skoki w wewnętrznej pętli dwukrotnie błędnie przewidziane!



  - Błędna predykcja skok nastąpi w ostatniej iteracji pętli wewnętrznej
  - Po czym błędna predykcja skok nie nastąpi w kolejnej pierwszej iteracji tej pętli

Predyktor 2-bitowy

- Zmień predykcję dopiero po dwóch błędnych predykcjach



Obliczanie adresu skoku

- Nawet przy predyktorze konieczność obliczania adresu docelowego skoku
  - Utrata jednego cyklu przy wykonaniu skoku
- Bufor adresów docelowych
  - Pamięć podręczna adresów skoku
  - Indeksowana przez PC przy ładowaniu instrukcji
    - Jeśli trafienie i przewidziane jest: skok nastąpi, to adres docelowy jest natychmiast ładowany

Wyjątki i przerwania

- “Niespodziewane” wydarzenia wymagające zmiany przebiegu programu
  - Różne ISA trochę różnie te pojęcia rozumieją
- Wyjątek (Exception)
  - Pojawia się w ramach CPU
    - np. niezdefiniowana instrukcja, przepełnienie, wywołanie systemowe (syscall), …
- Przerwanie
  - Z zewnętrznego sterownika WE/WY (I/O)
- Ich obsługa bez obniżania wydajności jest trudna

Obsługa (Handling) wyjątków

- W MIPS wyjątki są zarządzane przez System Control Coprocessor (CP0)
- Zachowaj PC wadliwej (lub przerwanej) instrukcji
  - W MIPS: Exception Program Counter (EPC)
- Zachowaj przyczynę problemu
  - W MIPS: Cause register
  - Tu przyjmujemy 1-bitowy opis
    - 0 dla niezdefiniowanej instrukcji, 1 dla przepełnienia
- Skocz do obsługi pod adres 8000 00180

Mechanizm alternatywny

- Przerwania wektorowe
  - Adres obsługi zdeterminowany przez przyczynę
- Przykładowo:
  - Niezdefiniowana instrukcja: C000 0000
  - Przepełnienie: C000 0020
  - …: C000 0040
- Instrukcje albo
  - Radzą sobie z przerwaniem, albo
  - Skok do jego obsługi

Akcje obsługi przerwań: MIPS

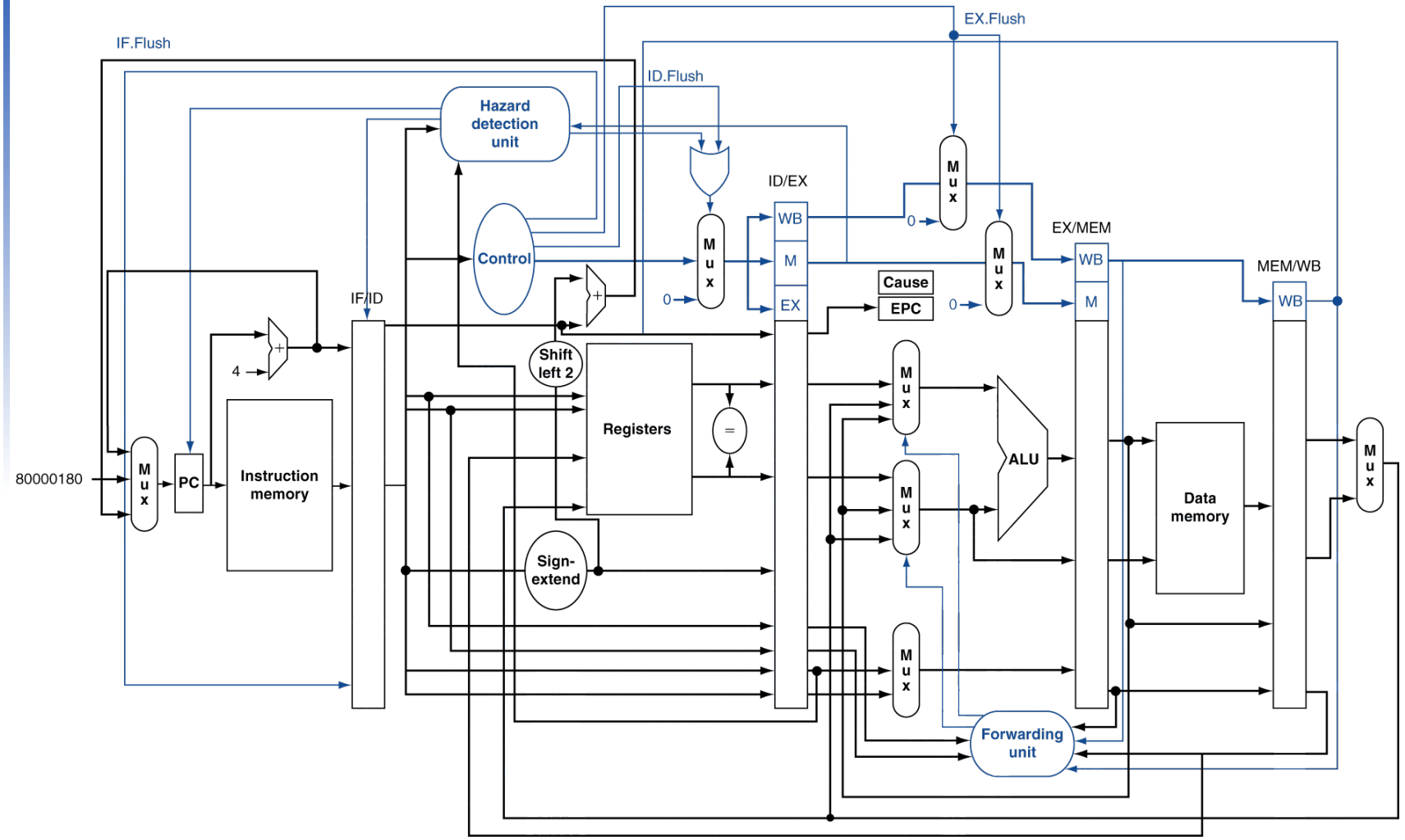
- Czytaj przyczynę i dokonuj skoku do właściwego podprogramu obsługi
- Określ wymaganą akcję
- Jeśli przyczyna naprawialna
  - Wykonaj akcję korygującą
  - użyj EPC w celu powrotu do programu
- Jeśli nie
  - Zakończ program
  - Raportuj błąd podając EPC, przyczynę, …

Wyjątki a potok

- Inna forma zagrożenia sterowania
- Rozważmy przepełnienie w stanie EX

  `add $1, $2, $1`
  - Chroń $1 przed zniszczeniem zawartości
  - Dokończ poprzednie instrukcje
  - Spłucz add i następne instrukcje
  - Ustaw wartości rejestrów Cause i EPC
  - Przekaż sterowanie do obsługi przerwań
- Podobne do źle przewidzianego skoku
  - Użyj wspólnego dla obu przypadków sprzętu

Potok z wyjątkami



Właściwości wyjątków

- Wyjątki naprawialne (restartable)
  - Potok może spłukać instrukcję
  - Po wykonaniu obsługi powrót do instrukcji
    - Ponowne jej załadowanie i wykonanie
- PC zachowany w rejestrze EPC
  - Identyfikuje instrukcję sprawiającą problem
  - Faktycznie zachowane jest PC + 4
    - Obsługa wyjątku musi to skorygować

Przykład wyjątku

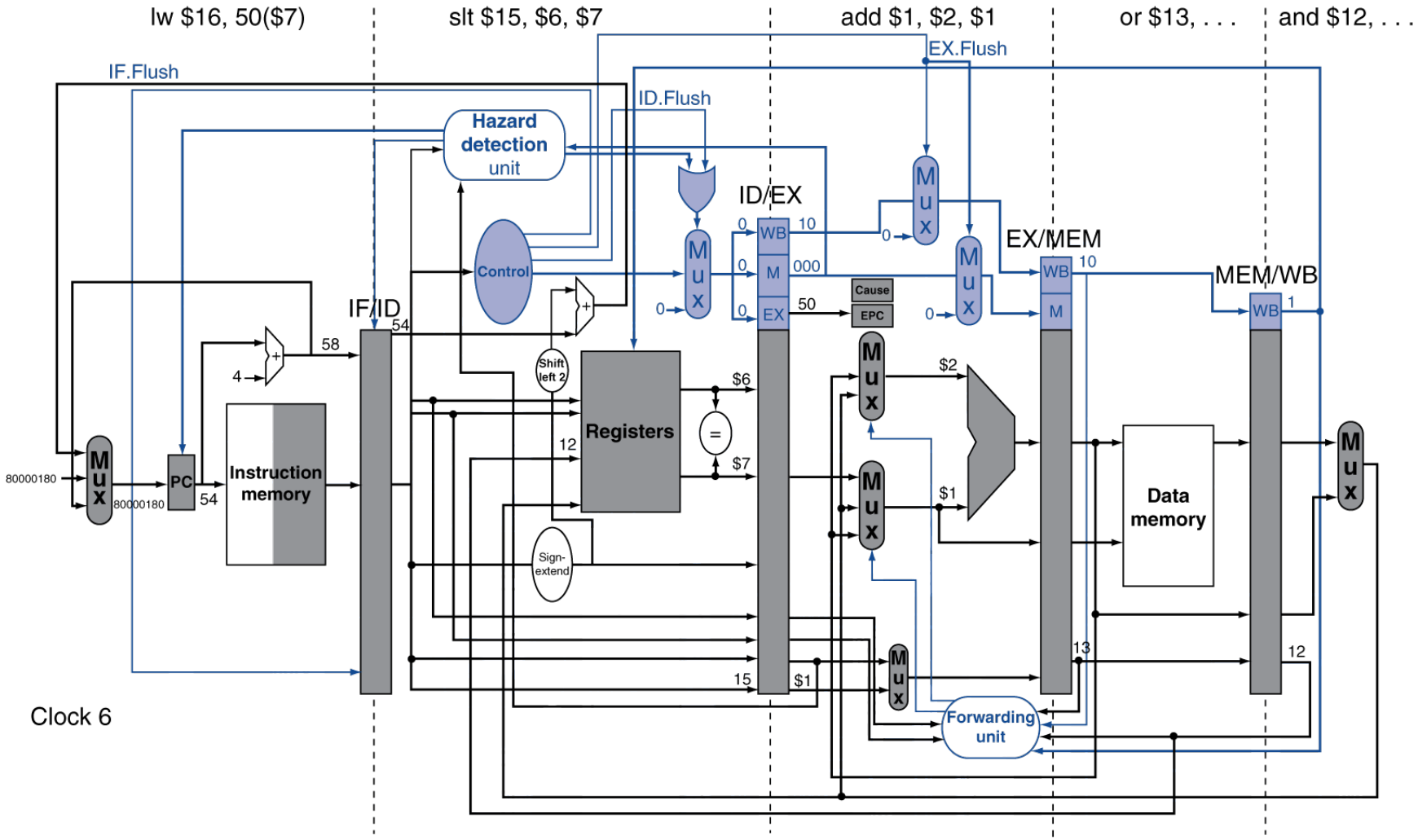
- Wyjątek w add w

```
40      sub   $11, $2, $4
44      and   $12, $2, $5
48      or    $13, $2, $6
4C      add   $1,  $2, $1
50      slt   $15, $6, $7
54      lw    $16, 50($7)
…
```

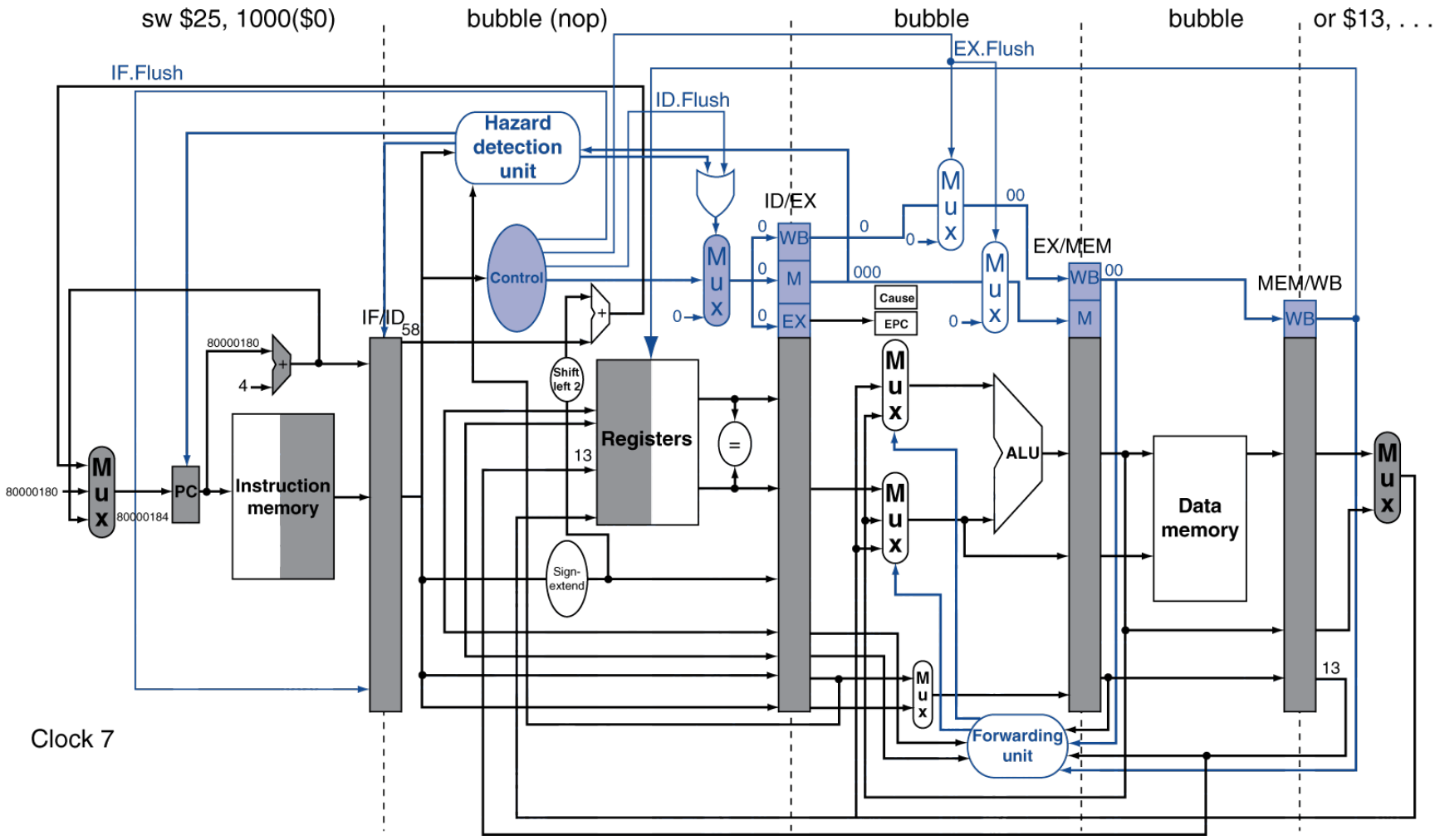
- Obsługa

```
80000180    sw    $25, 1000($0)
80000184    sw    $26, 1004($0)
…
```

Przykład wyjątku



Przykład wyjątku



Wyjątki wielokrotne

- W potoku instrukcje nakładają się
  - Może wystąpić wiele wyjątków naraz
- Proste podejście: opracuj wyjątek dla najwcześniejszej instrukcji
  - Spłucz następne instrukcje
  - Wyjątki “dokładne” (precise exceptions)
- W złożonych potokach
  - W jednym cyklu uruchamianych jest wiele instrukcji
  - Kończenie instrukcji nie po kolei (Out-of-order completion)
  - Gwarantowanie dokładnych wyjątków jest trudne!

Niedokładne wyjątki

- Po prostu zatrzymaj potok i zachowaj stan
  - Włączając w to przyczynę(y) wyjątku(ów)
- Niech obsługa wypracuje
  - Która(e) instrukcja(e) ma(ją) wyjątek
  - Które dokończyć i które spłukać
    - Może wymagać dokańczania “ręcznego”
- Upraszcza sprzęt, ale bardziej złożony program obsługi
- Niewykonalne dla złożonych potoków z równoległym uruchamianiem wielu instrukcji (multiple-issue) i kończeniem ich nie po kolei

Instruction-Level Parallelism (ILP)

- Potok: wykonywanie wielu instrukcji równolegle
- W celu zwiększenia równoległości na poziomie instrukcji (ILP)
  - Głębszy potok
    - Mniej pracy na stan ⇒ krótszy cykl zegara
  - Multiple issue – równoległe uruchamianie wielu instrukcji
    - Powiel stany potoku ⇒ wiele potoków
    - Startuj wiele instrukcji w jednym cyklu zegara
    - CPI < 1, tak więc nowy termin: Instructions Per Cycle (IPC)
    - np. 4GHz 4-krotne „multiple-issue”
      - 16 BIPS, szczytowe CPI = 0.25, szczytowe IPC = 4
    - W praktyce zależności obniżają ten wynik

Multiple Issue

- Statyczne (Static multiple issue)
  - Kompilator grupuje instrukcje do równoczesnego wykonania
  - Pakuje je w “issue slots”
  - Kompilator wykrywa i eliminuje zagrożenia
- Dynamiczne (Dynamic multiple issue)
  - CPU bada strumień instrukcji i wybiera instrukcje do równoczesnego wykonania
  - Kompilator może pomóc przez reorganizowanie instrukcji
  - CPU oddala zagrożenia w czasie rzeczywistym stosując zaawansowane techniki

Spekulowanie (Speculation)

- “Zgadnij” co zrobić z instrukcją
  - Startuj wykonanie najszybciej jak się da
  - Sprawdź czy zgadywanie było poprawne
    - Jeśli tak, dokończ wykonanie
    - Jeśli nie, wycofaj się i rób to co trzeba
- Wspólne dla obu rodzajów multiple issue
- Przykłady
  - Spekuluj co do wyniku instrukcji branch
    - Wycofaj się jeśli wybrana ścieżka jest inna
  - Spekuluj co do instrukcji load
    - Wycofaj się jeśli miejsce w pamięci jest uaktualnione

Spekulowanie

- Kompilator może przeorganizować instrukcje
  - np. przesunąć load przed branch
  - Może wstawiać instrukcje “naprawiające” w celu wycofania się z nieudanego zgadywania
- Sprzęt może analizować instrukcje czekające na wykonanie
  - Buforuje wyniki do momentu kiedy stają się potrzebne
  - Spłukuje bufory gdy niepoprawna spekulacja

Spekulowanie i wyjątki

- Co jeśli wyjątek wystąpi przy spekulacyjnym wykonaniu instrukcji?
  - np. spekulacyjny load przed testem czy wskaźnik nie jest null
- Spekulowanie statyczne
  - Można dodać wsparcie ISA dla odraczania wyjątków
- Spekulowanie dynamiczne
  - Można buforować wyjątki aż do dokończenia wykonania instrukcji (co może nie nastąpić)

Static Multiple Issue

- Kompilator grupuje instrukcje w “issue packets” („pakiety wykonania”)
  - Grupuje instrukcje które mogą być uruchomione w jednym wspólnym cyklu
  - Określone przez niezbędne zasoby potoku
- Myśl o „issue packet” jak o bardzo długiej instrukcji
  - Określa wiele równoczesnych operacji
  - ⇒ Very Long Instruction Word (VLIW)

Planowanie Static Multiple Issue

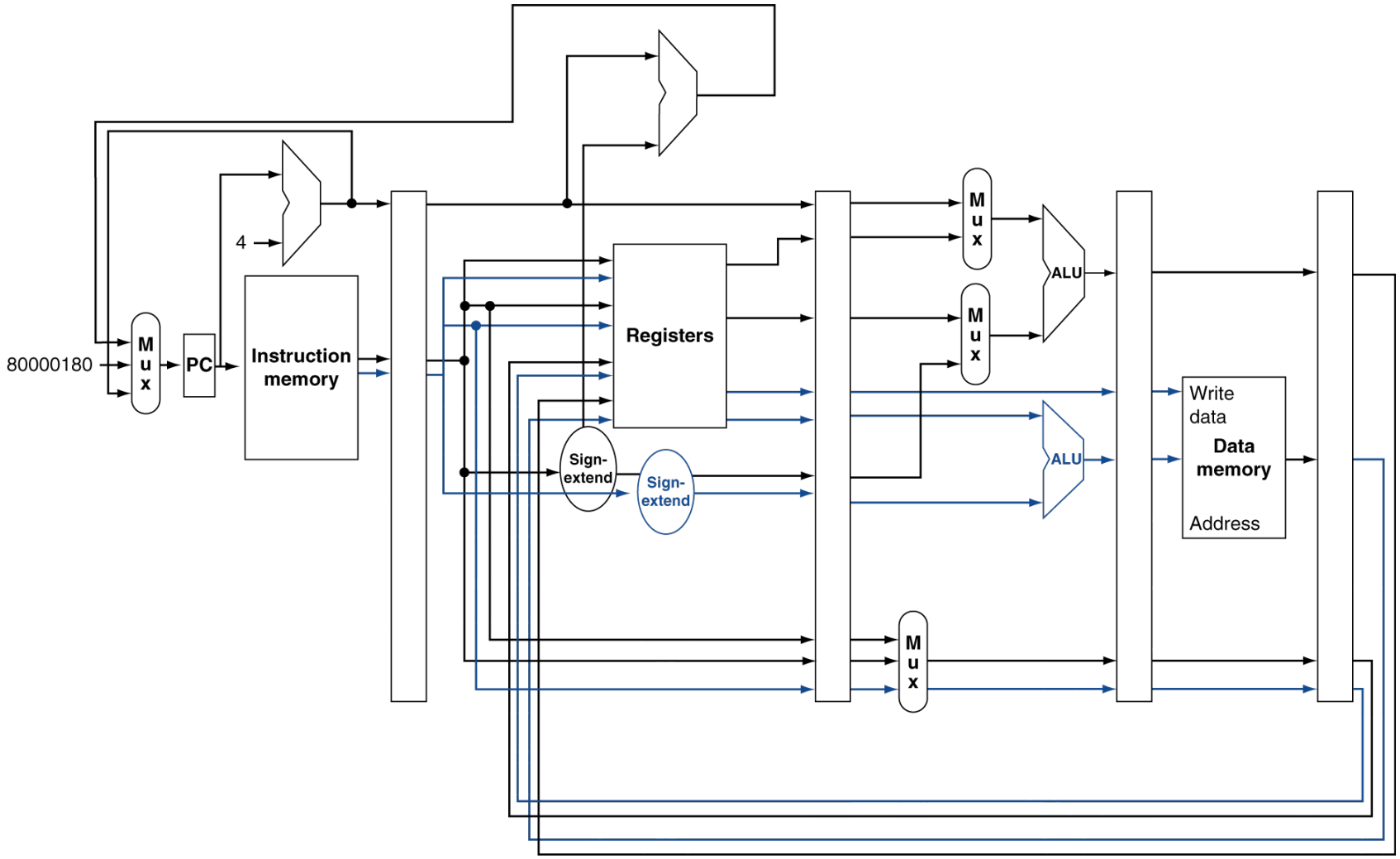
- Kompilator musi usunąć niektóre/wszystkie zagrożenia
  - Przeorganizować instrukcje w „issue packets”
  - Brak zależności w pakiecie
  - Możliwe zależności pomiędzy pakietami
    - Różnie to wygląda dla różnych ISA; kompilator musi to wiedzieć!
  - Jeśli konieczne, wypełnić nop’ami

MIPS z Static Dual Issue

- Pakiety dwumiejscowe (Two-issue packets)
  - Jedna instrukcja ALU/branch
  - Jedna instrukcja load/store
  - Wyrównanie do 64 bitów
    - ALU/branch, po czym load/store
    - Wypełnij nop'ami niewykorzystane miejsca

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

MIPS z Static Dual Issue



Zagrożenia w Dual-Issue MIPS

- Więcej instrukcji wykonujących się równolegle
- Zagrożenie danych w EX
  - Dotychczas przekierowanie pozwalało unikać zatrzymań
  - Teraz nie można użyć wyniku ALU w load/store z tego samego pakietu
    - ```
      add  $t0, $s0, $s1
      load $s2, 0($t0)
      ```
    - Podziel na dwa pakiety, faktyczne zatrzymanie
- Zagrożenie przy Load
  - Nadal zatrzymanie na cykl, ale teraz to dwie instrukcje
- Wymagane bardziej dogłębne planowanie
      ```

Przykład planowania

- Planuj dla dual-issue MIPS

```
Loop: lw   $t0, 0($s1)      # $t0=element tablicy
      addu $t0, $t0, $s2    # dodaj skalar z $s2
      sw   $t0, 0($s1)      # schowaj wynik
      addi $s1, $s1,–4      # zmniejsz wskaźnik
      bne  $s1, $zero, Loop # branch $s1!=0
```

	ALU/branch	Load/store	cykl
Loop:	nop	lw $t0, 0($s1)	1
	addi $s1, $s1,–4	nop	2
	addu $t0, $t0, $s2	nop	3
	bne $s1, $zero, Loop	sw $t0, 4($s1)	4

- IPC = 5/4 = 1.25 (teoretycznie IPC = 2)

Rozwijanie pętli

- Replikuj ciało pętli w celu ułatwienia większej równoległości
  - Zmniejsza koszt sterowania pętli
- Użyj różnych rejestrów dla różnych replikacji
  - Określane jako “register renaming”
  - Unikamy wynikających z pętli “antyzależności”
    - Po store następuje load tego samego rejestru
    - Czyli “zależności nazwy” (name dependence)
      - Ponowne użycie nazwy rejestru

Przykład rozwijania pętli

	ALU/branch	Load/store	cykl
Loop:	addi $s1, $s1,–16	lw $t0, 0($s1)	1
	nop	lw $t1, 12($s1)	2
	addu $t0, $t0, $s2	lw $t2, 8($s1)	3
	addu $t1, $t1, $s2	lw $t3, 4($s1)	4
	addu $t2, $t2, $s2	sw $t0, 16($s1)	5
	addu $t3, $t4, $s2	sw $t1, 12($s1)	6
	nop	sw $t2, 8($s1)	7
	bne $s1, $zero, Loop	sw $t3, 4($s1)	8

- IPC = 14/8 = 1.75
  - Bliżej 2, ale kosztem rejestrów i rozmiaru kodu

Dynamic Multiple Issue

- Procesory “Superskalarne”
- CPU decyduje ile instrukcji startuje w każdym cyklu: 0, 1, 2, …
  - Unikając zagrożeń strukturalnych i danych
- Planowanie kolejności instrukcji przez kompilator zbędne
  - Choć może pomóc
  - Sens kodu ustalany przez CPU

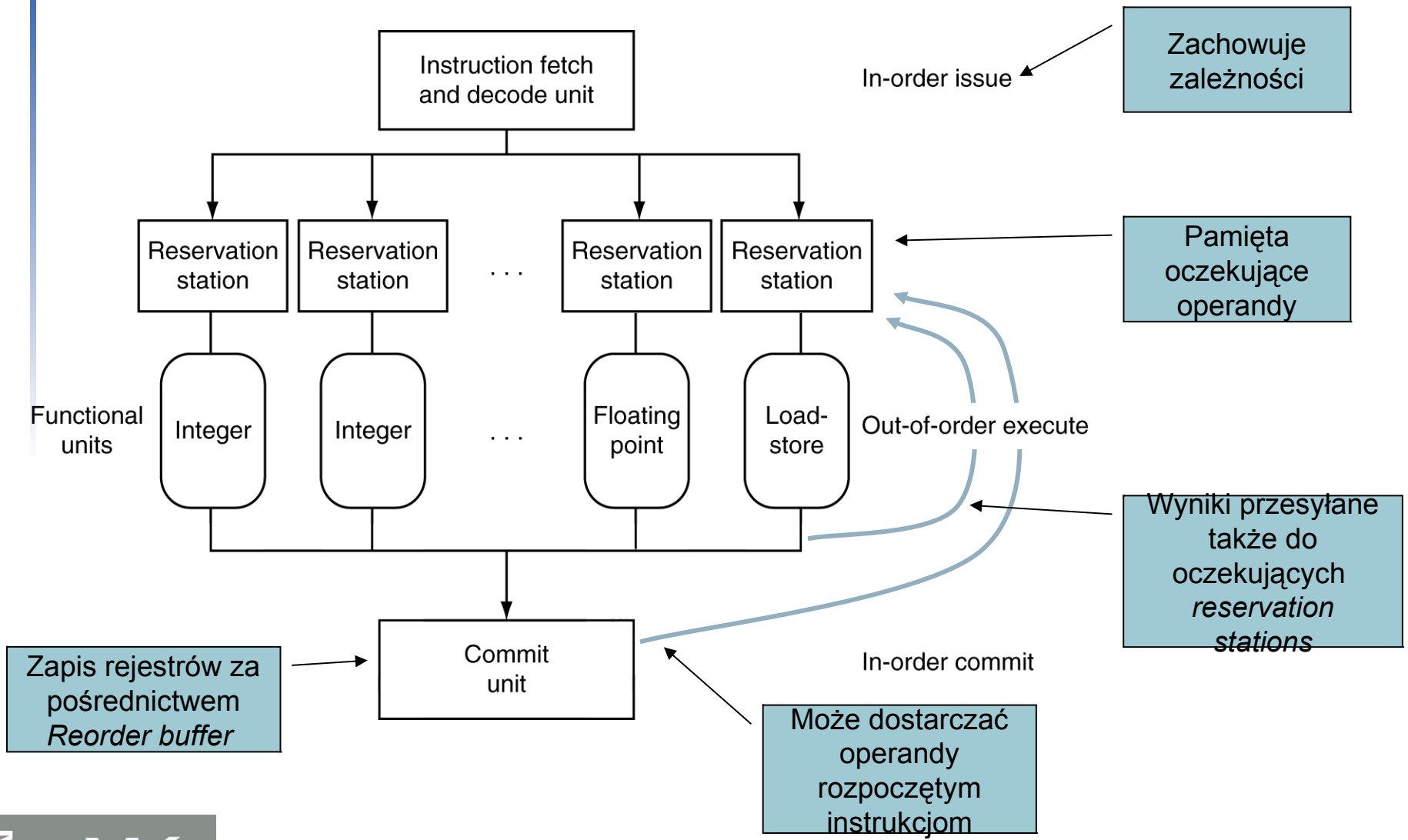
Dynamiczny plan potoku

- Pozwala by CPU wykonywało instrukcje nie po kolei by uniknąć zatrzymań
  - Ale umieszcza wyniki w rejestrach po kolei (in order – out of order)
- Przykład

```
lw    $t0, 20($s2)
addu  $t1, $t0, $t2
sub   $s4, $s4, $t3
slti  $t5, $s4, 20
```

  - Można rozpocząć sub gdy addu czeka na lw

Dynamicznie planowana CPU



Register Renaming

- Register renaming (zmiany rejestrów) są efektywnie wykonywane przez *reservation stations* i *reorder buffer*
- Przesłanie instrukcji do *reservation station*
  - Jeśli operand jest dostępny w rejestrach lub w *reorder buffer*
    - Jest kopiowany do *reservation station*
    - Rejestr można nadpisać jeśli operand niepotrzebny
  - Jeśli operand jeszcze nie jest dostępny
    - Będzie dostarczony do *reservation station* przez jednostkę wykonawczą (function unit)
    - Uaktualnienie rejestru może nie być konieczne

Spekulowanie

- Przewiduj efekt branch i kontynuuj
  - Nie kończ instrukcji po branch dopóki jego efekt nieznany
- Spekulowanie przy load
  - Unikaj load z opóźnieniem przy nietrafieniu pamięci podręcznej (cache miss delay)
    - Przewiduj adres efektywny
    - Przewiduj ładowaną wartość
    - Dokonaj load przed ukończeniem zaległych stores
    - Omijaj przechowywane wartości w load unit
  - Nie kończ load przed wyjaśnieniem spekulacji

Po co dynamiczne planowanie

- Dlaczego nie dać kompilatorowi planować kod (to schedule code)?
- Nie zawsze zatrzymanie da się przewidzieć
  - np. nietrafienia pamięci podręcznej
- Nie zawsze da się planować skoki
  - Wynik branch jest określany dynamicznie
- Różne implementacje ISA mają różne opóźnienia i zagrożenia

Czy Multiple Issue działa?

The BIG Picture

- Tak, ale nie w pełni jak byśmy chcieli
- W programach są zależności ograniczające ILP
- Niektóre zależności są trudne do usunięcia
  - np. zależności wskaźników
- Niektóre równoległości trudno ujawnić
  - Ograniczony zakres analizowanych instrukcji
- Opóźnienia i ograniczone pasmo pamięci
  - Trudno całkowicie wypełnić potok
- Jeśli dobrze robione spekulowanie może pomóc

Efektywność mocowa

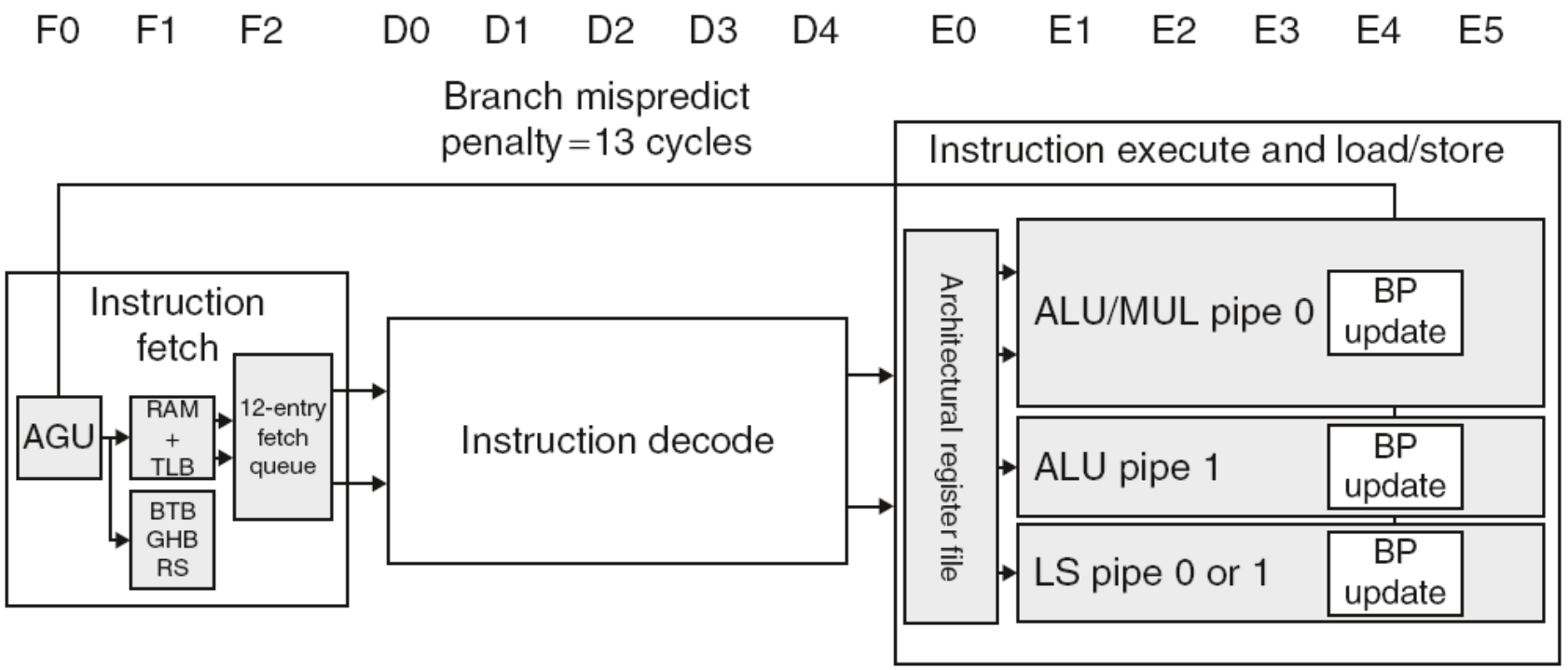
- Planowanie dynamiczne i spekulowanie zwiększa pobór mocy
- Wiele prostszych rdzeni może być lepsze

Mikroprocesor	Rok	Zegar	Etapy potoku	Ile instr. jednocz.	Out-of-order/ Speculation	Rdzenie	Pobór mocy
i486	1989	25MHz	5	1	Nie	1	5W
Pentium	1993	66MHz	5	2	Nie	1	10W
Pentium Pro	1997	200MHz	10	3	Tak	1	29W
P4 Willamette	2001	2000MHz	22	3	Tak	1	75W
P4 Prescott	2004	3600MHz	31	3	Tak	1	103W
Core	2006	2930MHz	14	4	Tak	2	75W
UltraSparc III	2003	1950MHz	14	4	Nie	1	90W
UltraSparc T1	2005	1200MHz	6	1	Nie	8	70W

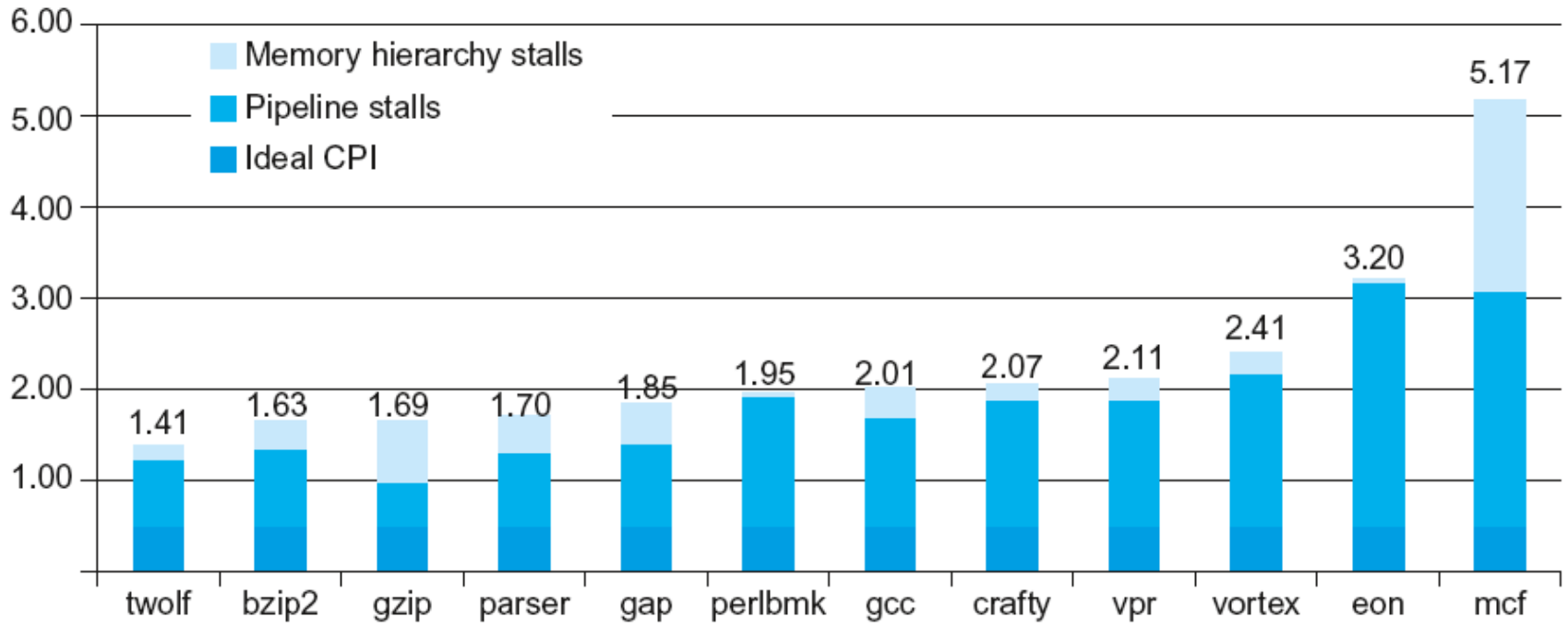
ARM Cortex A8 i Intel i7

Processor	ARM A8	Intel Core i7 920
Market	Personal Mobile Device	Server, cloud
Thermal design power	2 Watts	130 Watts
Clock rate	1 GHz	2.66 GHz
Cores/Chip	1	4
Floating point?	No	Yes
Multiple issue?	Dynamic	Dynamic
Peak instructions/clock cycle	2	4
Pipeline stages	14	14
Pipeline schedule	Static in-order	Dynamic out-of-order with speculation
Branch prediction	2-level	2-level
1st level caches/core	32 KiB I, 32 KiB D	32 KiB I, 32 KiB D
2nd level caches/core	128-1024 KiB	256 KiB
3rd level caches (shared)	-	2- 8 MB

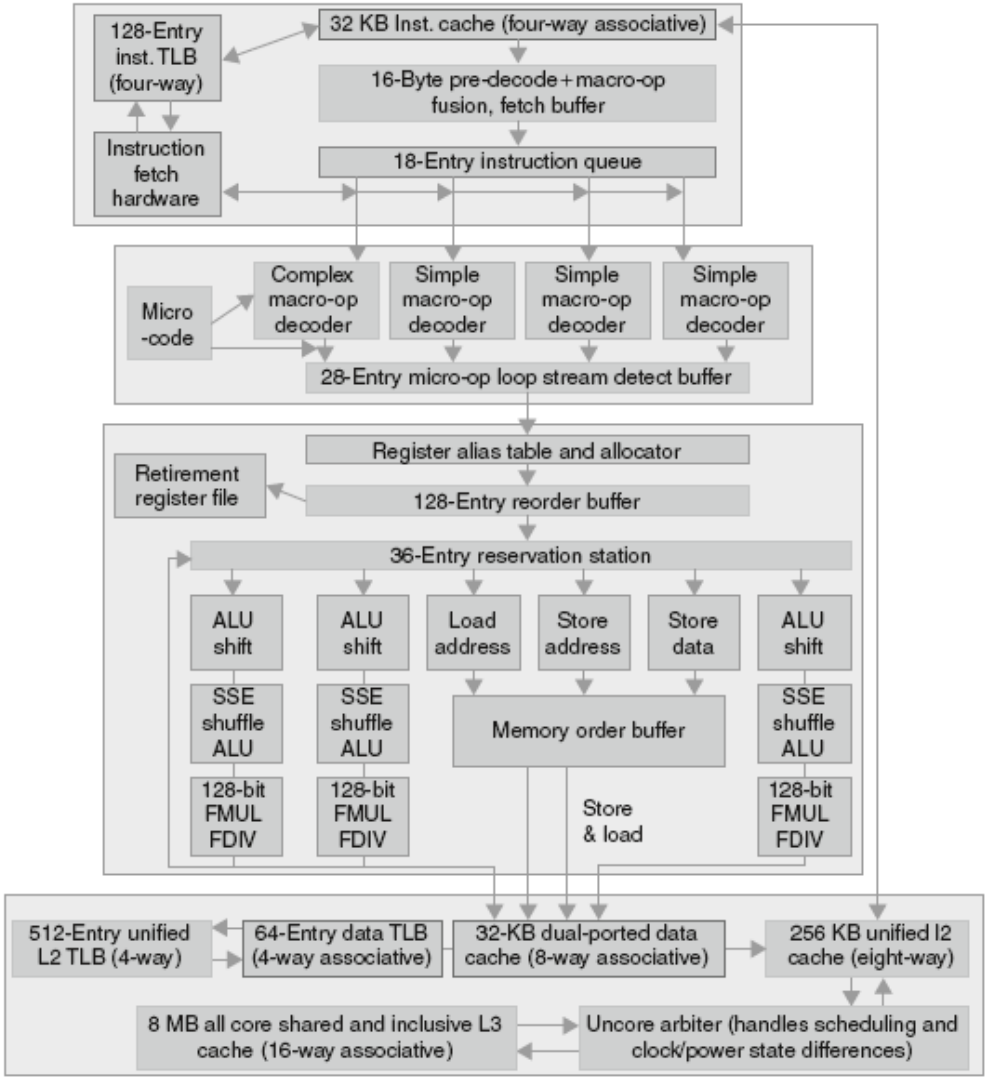
Potok ARM Cortex-A8



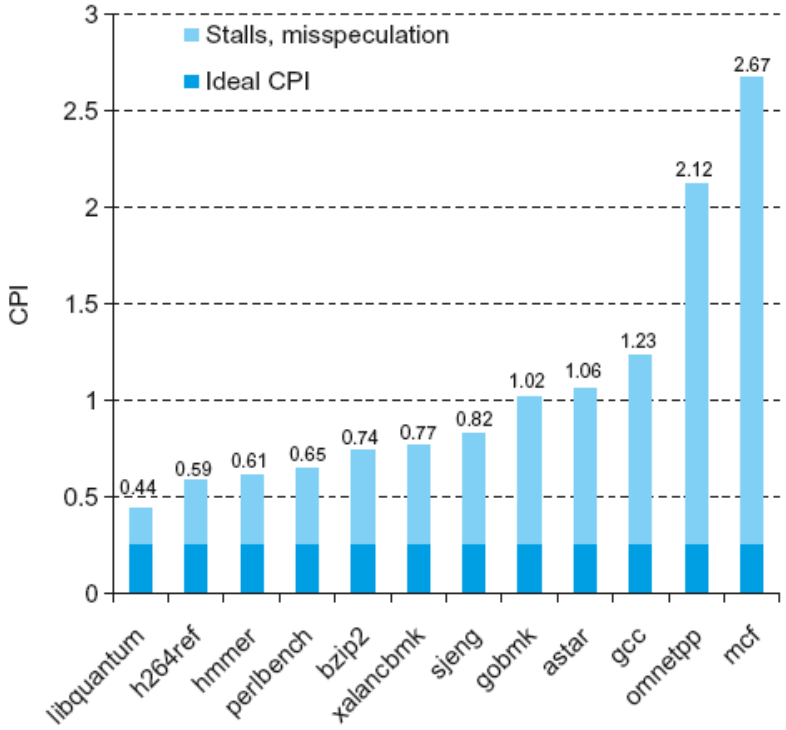
Wydajność ARM Cortex-A8

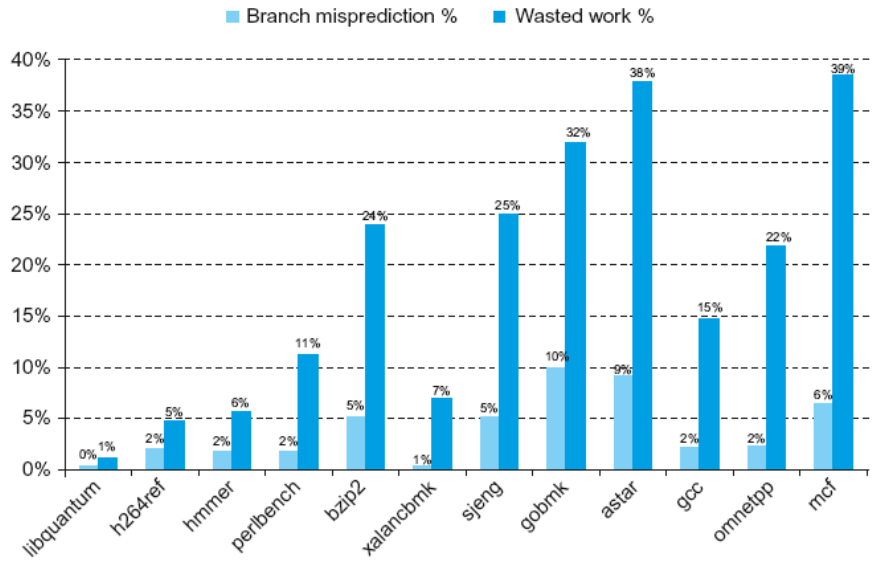


Potok Core i7



Wydajność Core i7





Mnożenie macierzy

- Rozwinięty kod C

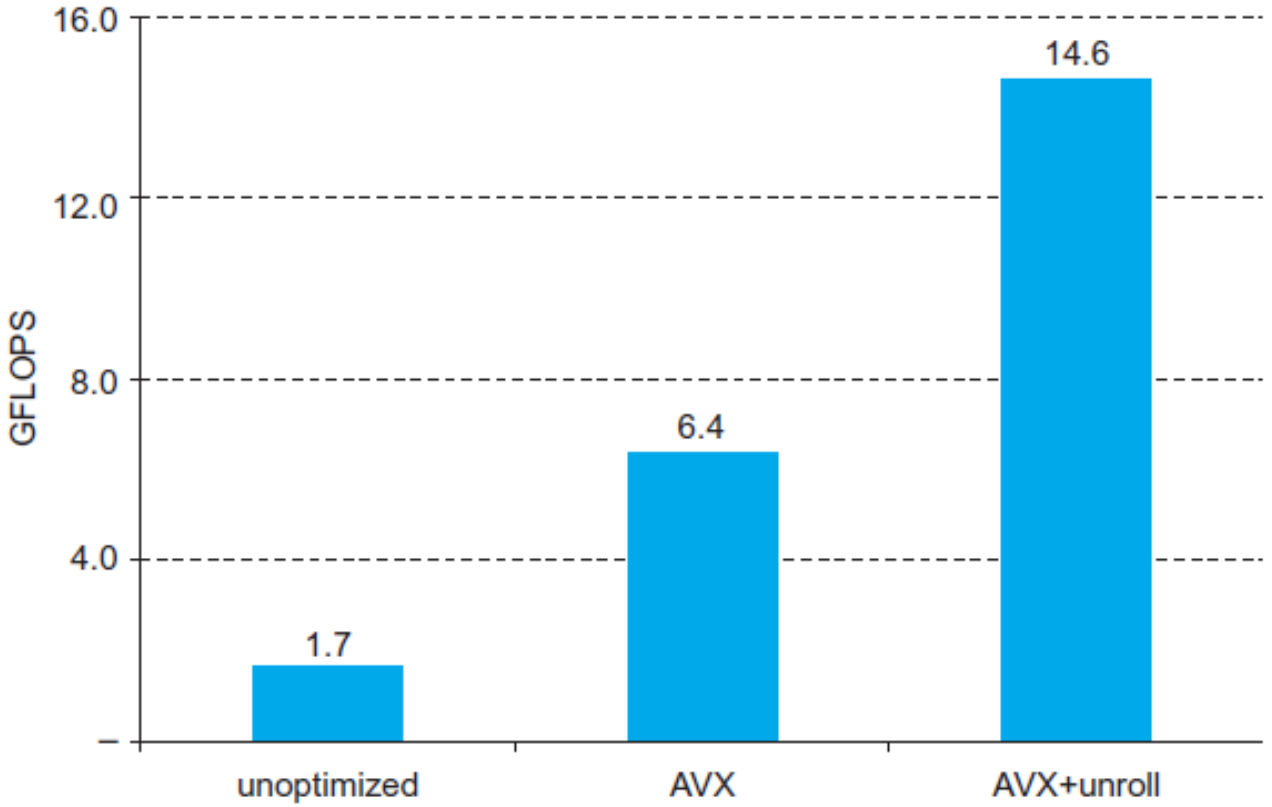
```
1 #include <x86intrin.h>
2 #define UNROLL (4)
3
4 void dgemm (int n, double* A, double* B, double* C)
5 {
6  for ( int i = 0; i < n; i+=UNROLL*4 )
7   for ( int j = 0; j < n; j++ ) {
8    __m256d c[4];
9    for ( int x = 0; x < UNROLL; x++ )
10    c[x] = _mm256_load_pd(C+i+x*4+j*n);
11
12   for( int k = 0; k < n; k++ )
13   {
14    __m256d b = _mm256_broadcast_sd(B+k+j*n);
15    for (int x = 0; x < UNROLL; x++)
16    c[x] = _mm256_add_pd(c[x],
17                         _mm256_mul_pd(_mm256_load_pd(A+n*k+x*4+i), b));
18   }
19
20    for ( int x = 0; x < UNROLL; x++ )
21     _mm256_store_pd(C+i+x*4+j*n, c[x]);
22  }
23 }
```

Mnożenie macierzy

- Kod asemblera:

```
1 vmovapd (%r11),%ymm4                  # Load 4 elements of C into %ymm4
2 mov %rbx,%rax                         # register %rax = %rbx
3 xor %ecx,%ecx                         # register %ecx = 0
4 vmovapd 0x20(%r11),%ymm3              # Load 4 elements of C into %ymm3
5 vmovapd 0x40(%r11),%ymm2              # Load 4 elements of C into %ymm2
6 vmovapd 0x60(%r11),%ymm1              # Load 4 elements of C into %ymm1
7 vbroadcastsd (%rcx,%r9,1),%ymm0       # Make 4 copies of B element
8 add $0x8,%rcx # register %rcx = %rcx + 8
9 vmulpd (%rax),%ymm0,%ymm5             # Parallel mul %ymm1,4 A elements
10 vaddpd %ymm5,%ymm4,%ymm4             # Parallel add %ymm5, %ymm4
11 vmulpd 0x20(%rax),%ymm0,%ymm5        # Parallel mul %ymm1,4 A elements
12 vaddpd %ymm5,%ymm3,%ymm3             # Parallel add %ymm5, %ymm3
13 vmulpd 0x40(%rax),%ymm0,%ymm5        # Parallel mul %ymm1,4 A elements
14 vmulpd 0x60(%rax),%ymm0,%ymm0        # Parallel mul %ymm1,4 A elements
15 add %r8,%rax                         # register %rax = %rax + %r8
16 cmp %r10,%rcx                        # compare %r8 to %rax
17 vaddpd %ymm5,%ymm2,%ymm2             # Parallel add %ymm5, %ymm2
18 vaddpd %ymm0,%ymm1,%ymm1             # Parallel add %ymm0, %ymm1
19 jne 68 <dgemm+0x68>                  # jump if not %r8 != %rax
20 add $0x1,%esi                        # register % esi = % esi + 1
21 vmovapd %ymm4,(%r11)                 # Store %ymm4 into 4 C elements
22 vmovapd %ymm3,0x20(%r11)             # Store %ymm3 into 4 C elements
23 vmovapd %ymm2,0x40(%r11)             # Store %ymm2 into 4 C elements
24 vmovapd %ymm1,0x60(%r11)             # Store %ymm1 into 4 C elements
```

Wpływ na wydajność



Błędny poglądy

- Implementacja potoku jest łatwa (!)
  - Sama idea jest prosta
  - Diabeł tkwi w szczegółach
    - np. wykrywanie zagrożeń danych
- Implementacja potoku nie zależy od technologii
  - To dlaczego nie stosowano potoku od zawsze?
  - Więcej tranzystorów dało możliwość realizacji bardziej zaawansowanych technik
  - Projektowanie ISA z potokiem wymaga wzięcia pod uwagę trendów technologicznych
    - np. kluczowych instrukcji

Pułapki

- Niedoskonały projekt ISA może prowadzić do trudnej realizacji potoku
  - np. złożone instrukcje (VAX, IA-32)
    - Istotne utrudnienie realizacji potoku
    - Micro-op w realizacji instrukcji IA-32
  - np. złożone tryby adresowania
    - Efekty uboczne aktualizacji rejestrów, pośredni dostęp do pamięci
  - np. skoki opóźnione (delayed branches)
    - Zaawansowane potoki mają wiele etapów

Uwagi końcowe

- ISA wpływa na projekt ścieżki danych i sterowania
- Odwrotna zależność jest też prawdziwa
- Potok zwiększa przepustowość instrukcji przez wykorzystanie równoległości
  - Więcej instrukcji wykonanych w sekundzie
  - Czasy wykonywania instrukcji bez zmian
- Zagrożenia: strukturalne, danych, sterowania
- *Multiple issue* i planowanie dynamiczne (ILP)
  - Zależności ograniczają osiągalną współbieżność
  - Złożoność prowadzi do „ściany mocowej”